

CSCI E-25

Introduction to Convolutional Neural Networks

Steve Elston

1.0 Introduction to convolutional neural networks

This lesson introduces you to a powerful neural network architecture, known as **convolutional neural networks**. Convolutional neural networks operate by **learning a set of filters** or **convolution kernels**. Using a process, known as **convolution**, these filters extract a **feature map** from the raw data. The feature map is a **lower dimensional** map of the raw input features. This map is learned in a supervised machine learning process employing back propagation. You can think of convolutional neural networks as a powerful and flexible **dimensionality reduction** method.

Convolutional neural networks are used to build feature maps from any type of data with coherency in one or more dimensions. This includes time series data, and text data as one dimensional cases, and image data as a two dimensional case.

Convolutional neural networks have a reasonably long history. The first known commercial application was for automated processing of check images by LeCun et. al. (1998). For unclear reasons, convolutional neural networks were relegated to specialized applications such as hand writing recognition until Krizhevsky et. al. (2012) used them to decisively win an ImageNet object recognition competition. Curiously, other teams had previously used convolutional neural networks to win competitions, but somehow this was not widely recognized.

In this lesson you will learn:

1. The basics of convolutional neural network architecture.
2. How convolutional neural networks learn the filters to create the feature map.
3. Feature extraction for creating the feature map.

4. Visualization of the feature map and the filters.
5. Commonly used convolutional operator design.
6. How to use the feature map in supervised learning.
7. Application of various regularization methods for improving the behavior of learning.

Note: The formulation of convolution filters used in deep learning is somewhat idiosyncratic. If you are familiar with the standard approach as applied in say electrical engineering or control engineering you may find the approach used here a bit different.

It is time to import the packages required to execute the code in this notebook.

```
In [1]: #!pip install pydot
```

```
In [2]: from keras.api.datasets import mnist
import tensorflow as tf
from tensorflow import convert_to_tensor
import keras.api.utils as ku
from keras.api.utils import plot_model
import keras.api.models as models
import keras.api.layers as layers
from keras.api.optimizers import RMSprop
from keras.api.optimizers.schedules import ExponentialDecay
from keras import regularizers, callbacks
from keras.layers import Dropout
from keras.metrics import CategoricalAccuracy, TopKCategoricalAccuracy
import numpy as np
import numpy.random as nr
import pandas as pd
from math import exp
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import rankdata
import sklearn.metrics as metrics
import math
```

```
%matplotlib inline
```

2025-03-08 21:20:26.225006: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

✓ Number of epochs to the notebook code integrity check run. Should be **None** for a full production run and 2 for test

```
In [3]: # TEST_EPOCHS = 2
TEST_EPOCHS = None
```

1.1 Applying 2-d convolution

Let's review how we can apply convolution operations in two dimensions. Figure 1.1 illustrates a simple example of applying a convolution operator to a 2-d input array. The 2-d input array can be a gray-scale image, for example.

A 3X3 convolutional operator is illustrated in Figure 1.1. This operator is applied to a small image of dimension of 4X4 pixels. Within the 4X4 image the operator is only applied four times, where the complete operator lies on the image. Convolution where the operator is completely contained within the input sample is sometimes called **valid convolution**.

In this case the 4X4 pixel input results in a 2X2 pixel output. You can think of the convolutional operator mapping from the center of the operator input to pixel in the output. Similar to the 1-d case, for a convolution operator with span s in both dimensions the output array will be reduced in each dimension by $\frac{s+1}{2}$

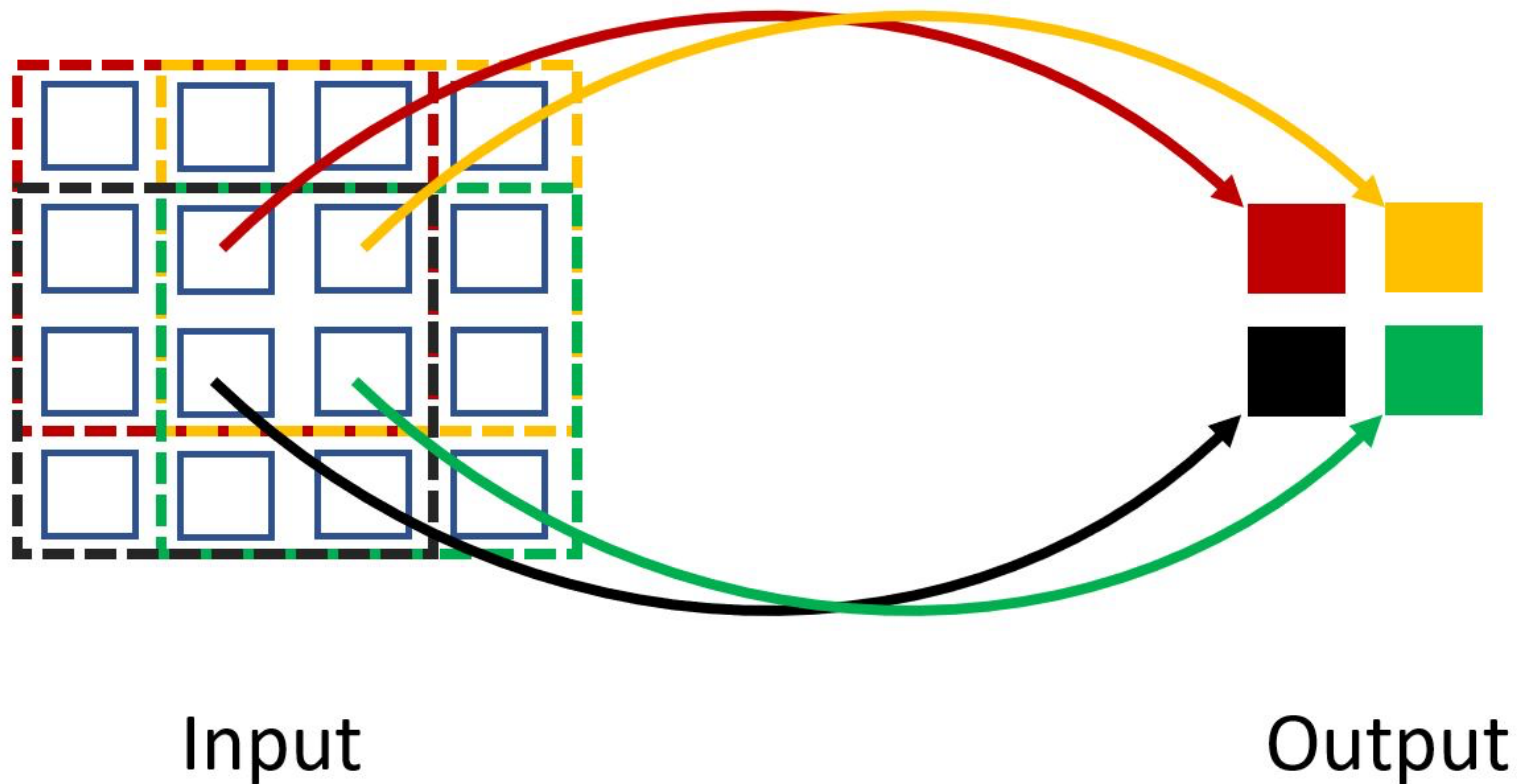


Figure 1.1. Simple 2-d convolution operation

For a convolution with the identical span s in both dimensions we can write the convolution operations as:

$$S(i, j) = (I * K)(i, j) = \sum_{m=i-a}^{i+a} \sum_{n=j-a}^{j+a} I(i, j) K(i - m, j - n)$$

Notice that S , I and K are all tensors. Given this fact, the above formula is easily extended to higher dimensional problems.

The convolutional kernel K and the image I are commutative so there is an alternative representation by applying an operation known as **kernel flipping**. Flipped kernel convolution can be represented as follows:

$$s(i, j) = (I * K)(i, j) = \sum_{m=i-a}^{i+a} \sum_{n=j-a}^{j+a} I(i-m, j-n)K(m, n)$$

1.2 Convolution in higher dimensions

In deep convolutional neural networks the input typically have several **input channels** with the convolution operation resulting in multiple **output channels**.

So far, we have only looked at single channel convolution. Here are some examples of input tensors with different numbers of input channels:

- A gray-scale image has a single input channel of pixels.
- A color image typically has 3 input channels, {Red, Green, Blue}.
- A color movie has a large number of input channels. For n images in the video there are $3 * n$ images counting the Red, Green, Blue channels for each image.

The purpose of the a convolutional neural network is to create a feature map. To create the most general feature map convolution operators need not be restricted to a single input channel.

Further, the feature map can can many channels, resulting in multiple output channels. Each channel is a set of features extracted from the input tensor. This concept is illustrated in Figure 1.2 below. In this figure a set of convolution operators transform the input tensors to multiple channels of the feature map. Each convolution operator maps to a specific output channel in the feature map.

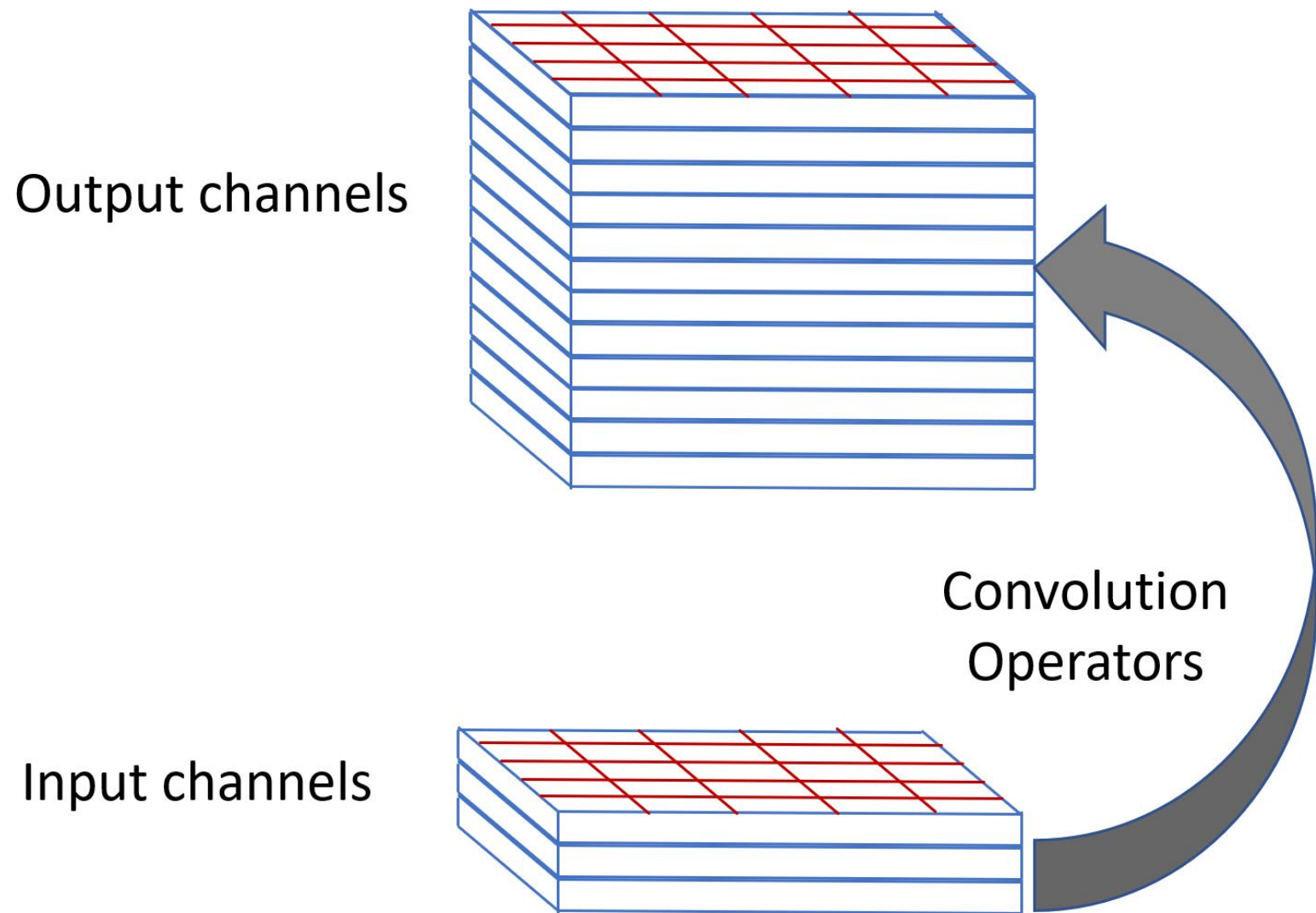


Figure 1.2. Example of multichannel convolution

Let's investigate the case of a 3-d input tensor V , a 3-d output tensor Z and 4-d convolution tensor K . In this case the convolution equation becomes:

$$Z_{i,j,k} = (V * Z)(i, j, k, l) = \sum_l \sum_{m=-a}^a \sum_{n=-a}^a V_{i,j,l} \cdot K_{i-m,j-n,k,l}$$

where,

i, j are the spatial dimensions,

k is the index of the output channel,

l is the index of the input channel,

$K_{i,j,k,l}$ is the kernel connecting the l th channel of the input to the k th channel of the output for pixel offsets i and j

$V_{i,j,l}$ is the i, j input pixel offsets from channel l of the input,

$Z_{i,j,k}$ is the i, j pixel offsets from channel k of the output,

$a = \frac{1}{2}(kernel_span + 1)$, for an odd length kernel.

Notice that the summation in the equation above is over input channels l as well as the spatial dimensions m, n of the convolutional operator. This operation is applied for each spatial pixel coordinate i, j . This generalization allows the convolutional kernel to operate over multiple input channels. This property is highly desirable since features should be extracted from the entire input tensor.

The entire foregoing operation is performed for each output channel k . Therefore, you can think of the k th dimension of the 4-d kernel tensor as indexing multiple 3-d convolution kernels.

1.3 Parameter sharing and sparse interactions

Parameter sharing and **sparse interactions** are two powerful aspects of convolutional neural networks. To understand what these terms mean and why they are important, consider the fully connected neural network in Figure 1.3 below.

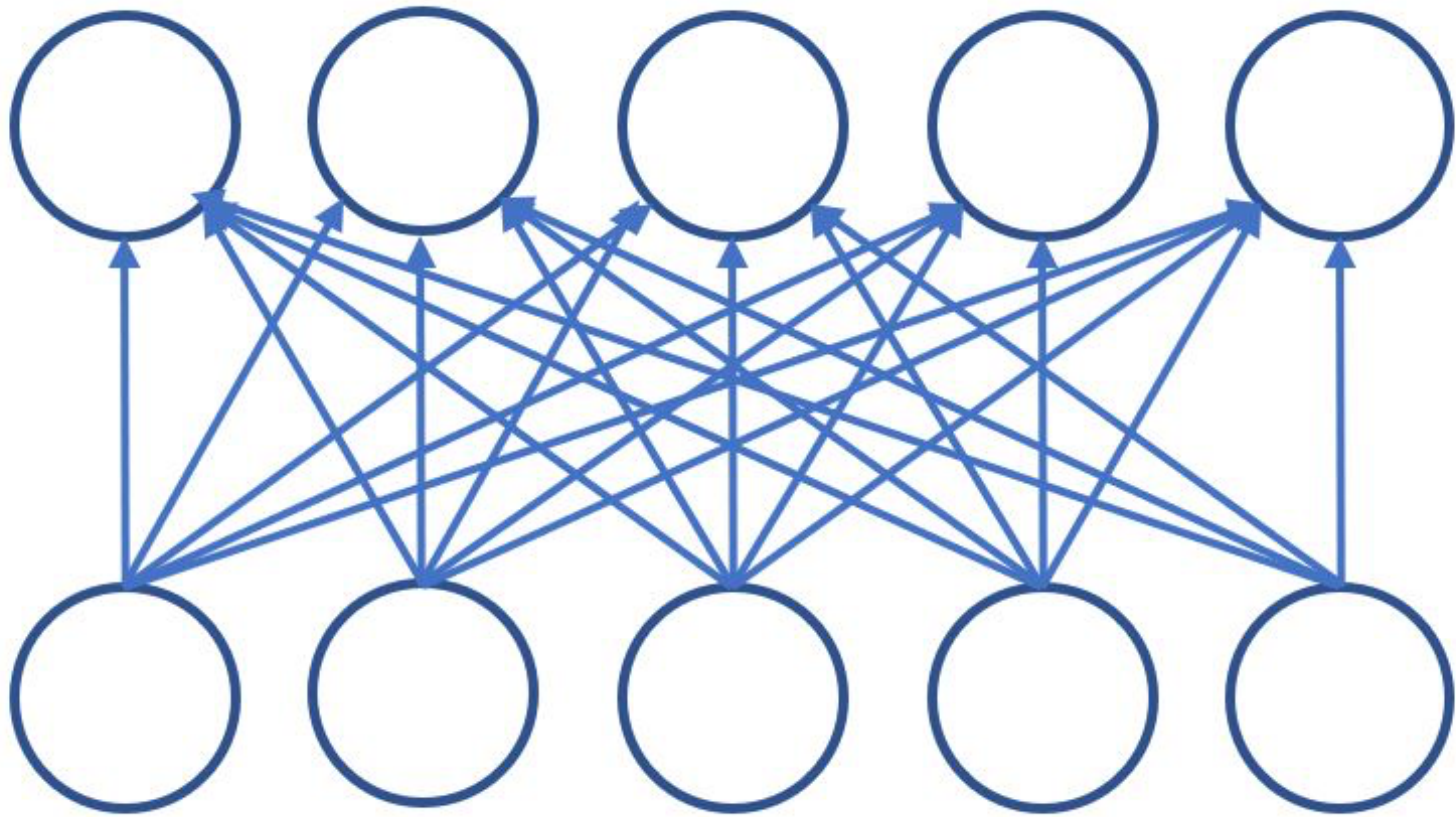


Figure 1.3. A fully connected network

The fully connected neural network has a large number of weights, $5^2 = 25$ in this case. Compare this network to the convolutional layer with 2×2 operator will have only $2 * 2 = 4$ weights. For a more realistic case, the fully connected network might have millions of weights. The convolutional network using **weight sharing** across the entire layer. For a 5×5 CovNet layer there are only $5 * 5 = 25$ weights. Parameter sharing is also referred to as **tied weights**. This is an example of a **sparse interaction**.

The shared nature of the parameters in convolutional neural networks is a significant reduction in complexity. Statistically, such a sparse model is said to **share statistical strength**. This statistical strength results in model weights with less uncertainty and a model less likely to be over-fit.

The computational efficiency of convolutional neural networks is not just in training. Convolutional neural networks are also efficient for creating **rich feature maps** which lead to high levels of accuracy in production.

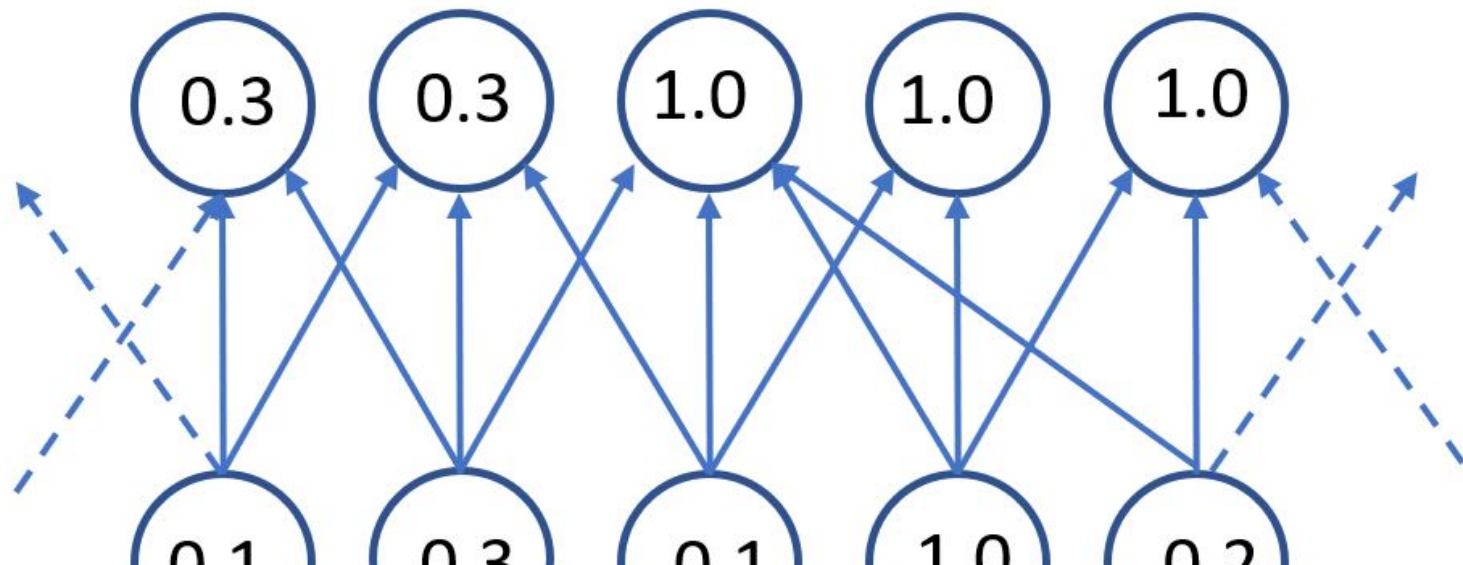
1.4 Pooling and invariance

To further reduce dimensionality and to make the feature map robust to small shifts in the input, **pooling** is applied following the nonlinear detection stage. Pooling involves applying an aggregation function to patches of the output from the nonlinear detector of a convolution layer.

Researchers have tried a number of pooling schemes. In practice **max pooling** (Zhou and Chellappa, 1988) has proved to be both robust and versatile. In max pooling the output value is just the maximum of the input values in each patch.

Unlike convolutional layers, there are no parameters to learn in pooling layers.

Max pooling has the nice property of being robust to small shifts in the input values. For example, Figure 1.4 shows the response of max pooling to a right shift of a single pixel for an operator with a span of 3. Notice that the response itself is simply shifted one pixel to the right as well. The pattern of maximum values in the feature map (the output) remains the same.



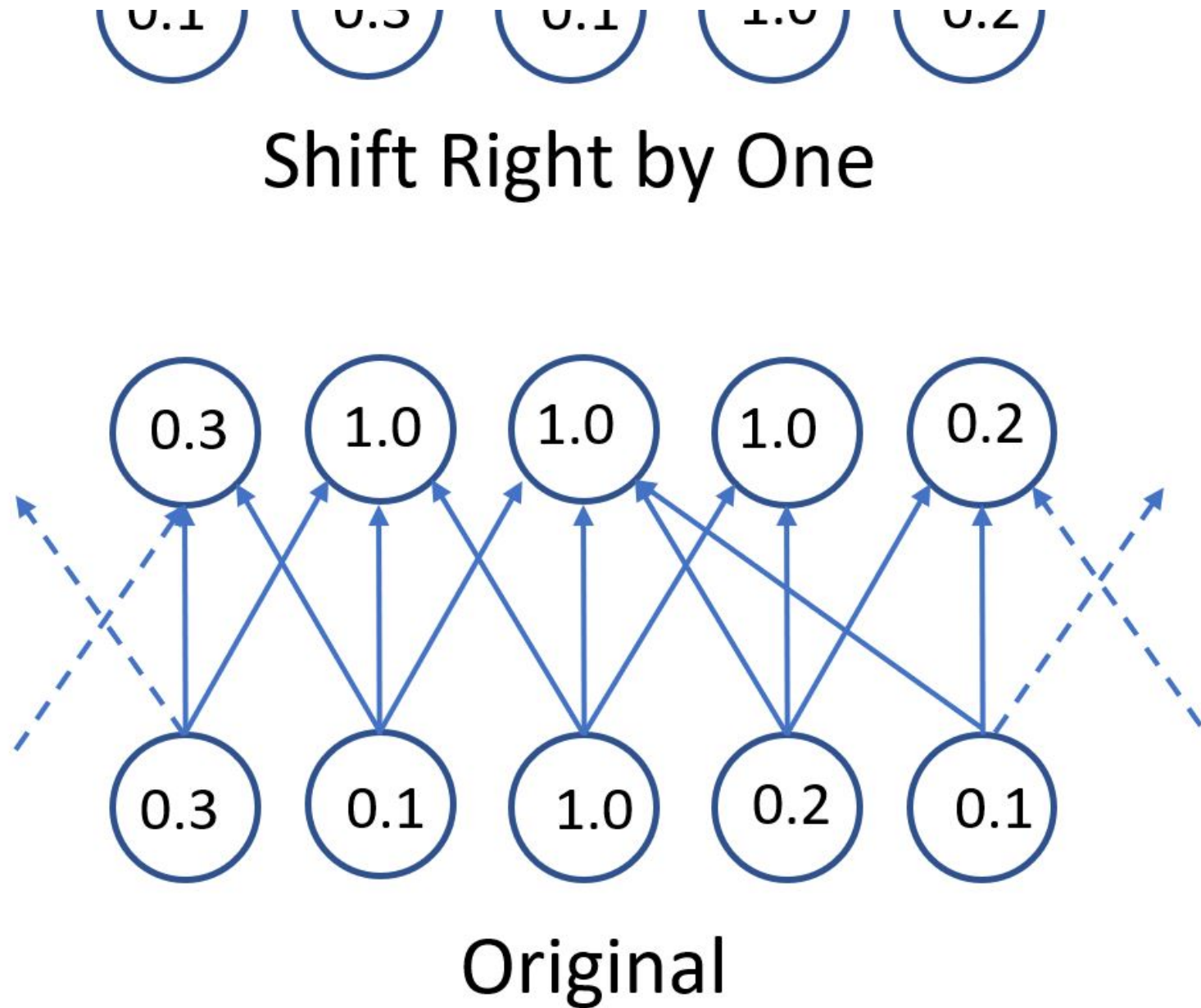


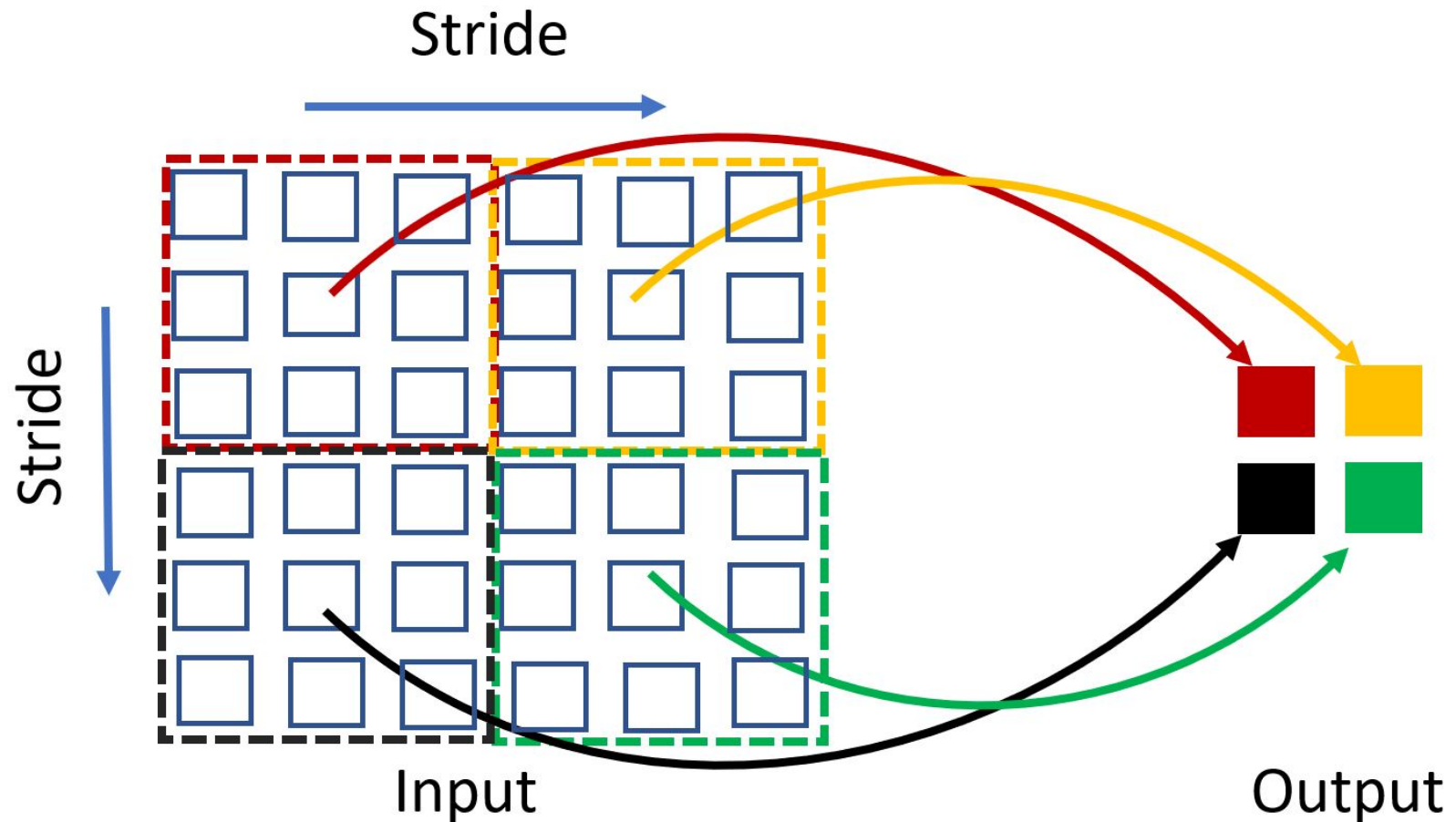
Figure 1.4. Example of max pooling responses to one pixel shift right

1.5 Stride and tiling

Up until now, we have only moved the convolution operator one pixel at a time. There is no restriction that requires this type of step, however. In fact, a convolutional operator or pooling operator can be moved by several pixels in any direction. This step size is known as the **stride** of the operator.

Operators with stride greater than one **decimate** the number of pixels in the output. This can be useful when resizing images of different dimensions so they have constant input dimensions for a neural network.

Using an operator with $stride = span$ is a special case known as **tiling**. Tiled operators are layed out without any overlap. Figure 1.6 shows an example of tiling. In this case, an input of dimensions 6×6 is tiled by operators with dimension 3×3 with stride of 3. Only 4 tiles fit on the input in this case, resulting in a 4×4 output array.



****Figure 1.6. Example of a tiled convolution operator****

► **Exercise 6-1:** You will now undertake an exercise to better understand the effects of max pooling. To simplify this exercise you will work with 1-dimensional data and **edge detection** operators. Do the following:

1. Create a function named `max_pool` that returns maximum of the absolute value kernel operator with a stride of 1 over the input series. The kernel span should be an argument to the function, with default value of 3. For simplicity start the operator at the beginning of the series.

2. Create two edge detection convolutional kernels, $kernel1 = [-0.5, 1.0, -0.5]$ and $kernel2 = [0.5, -1.0, 0.5]$.
3. Apply both of the convolution kernels to the `series1` provided using the `numpy.convolve` function. Assign these two results to separate variables.
4. Apply your `max_pool` function to the convolution result series. Assign these two results to different variables.
5. Display the results comparing the convolution series to the max-pooled series using the `plot_pool` function provided.

```
In [35]: def plot_series(series, pool, x_pool, label, ax):
    x = list(range(series.shape[0]))
    ax.plot(x, series, label = 'Detected series')
    ax.plot(x_pool, pool, color = 'red', label = label)
    ax.set_title('Series of raw values and max pool values')
    ax.legend()
    ax.set_xlabel('Index')
    ax.set_ylabel('Value')

def plot_pool(series1, series2, pool1, pool2):
    x = list(range(series1.shape[0]))
    end = series1.shape[0]
    end_pool = pool1.shape[0]
    end_remain = end % end_pool
    pool_stride = int((end_pool - end_remain)/end_pool) + 1
    x_pool = list(range(end_remain, end_pool + end_remain, pool_stride))
    print('length of the input series = ' + str(len(x)))
    print('length of the max pool series = ' + str(len(x_pool)))

    ## Display the results
    fig,ax = plt.subplots(2, 1, figsize=(8,6))
    ## First channel
    plot_series(series1, pool1, x_pool, 'First channel max pool', ax[0])
    ## Second channel
    plot_series(series2, pool2, x_pool, 'Second channel max pool', ax[1])

    plt.tight_layout()
```

```

nr.seed(12233)
series1 = np.concatenate((
    np.zeros((20,)),
    np.zeros((5,)) + 1.0,
    np.zeros((5,)) - 1.0,
    np.zeros((20,))
))

def max_pool(series, span=3, stride=1):
    '''Performs simple 1d max pooling with
    an operator over span and stride 1'''
    ## Your code goes below

    w = np.lib.stride_tricks.sliding_window_view(series, span)
    n = len(w) - len(w)%stride
    w = w[np.linspace(0, n-stride, n//stride).astype(int)]

    return np.abs(w).max(axis=1)

# Convolution kernels
k_edge1 = [-0.5, 1.0, -0.5]
k_edge2 = [0.5, -1.0, 0.5]

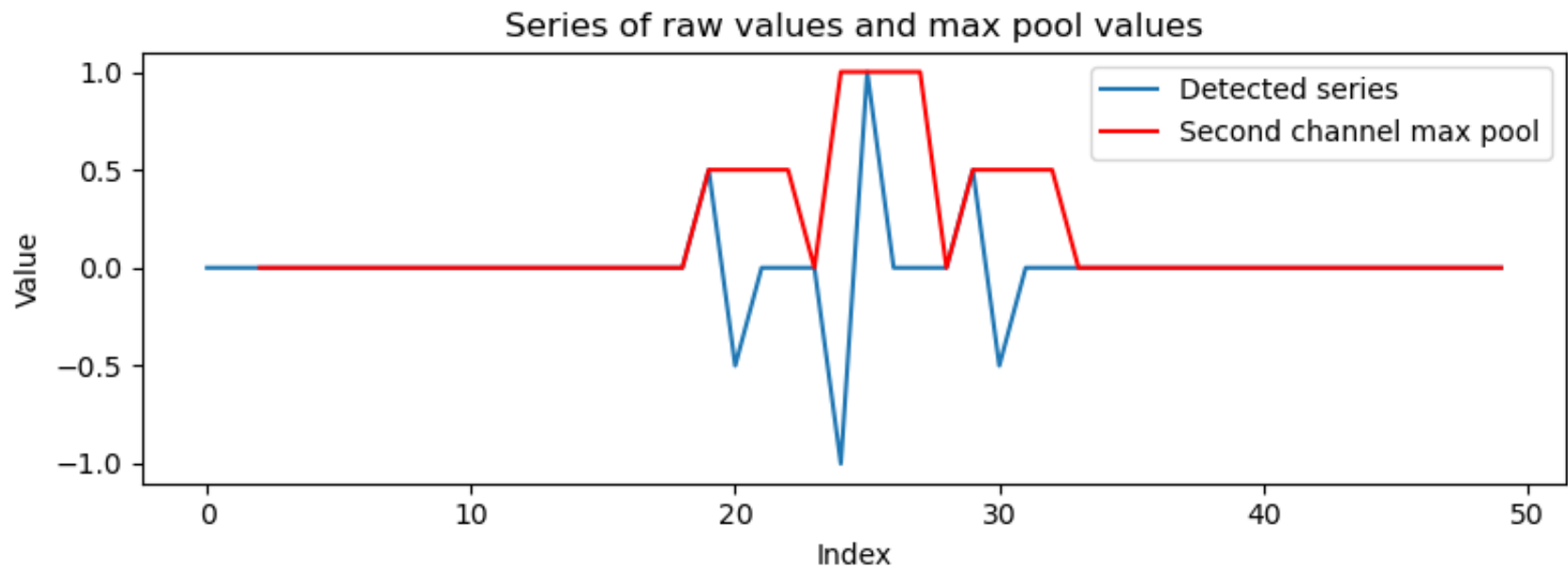
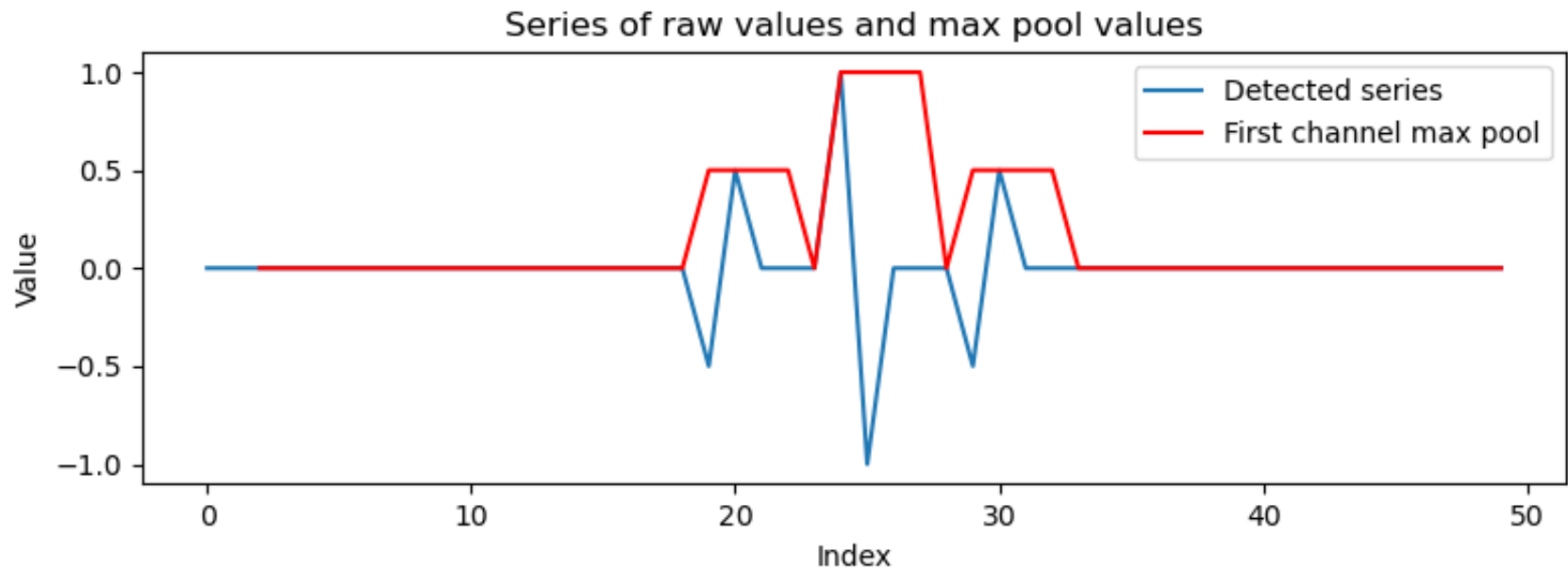
# Apply convolution
s_edge1 = np.convolve(series1, k_edge1, mode='same')
s_edge2 = np.convolve(series1, k_edge2, mode='same')

# Apply maxpool
s_edge1_mp = max_pool(s_edge1)
s_edge2_mp = max_pool(s_edge2)

# Display the results
plot_pool(s_edge1, s_edge2, s_edge1_mp, s_edge2_mp)

```

length of the input series = 50
length of the max pool series = 48



6. Define and apply new 5-point convolution kernels for the edge detection,

$[-0.25, -0.25, 1.0, -0.25, -0.25]$ and $[0.25, 0.25, -1.0, 0.25, 0.25]$.

7. Apply the max pooling with kernel span of 5 to both of the edge detection series.
8. Display the results.

```
In [5]: # Convolution kernels

k_edge1 = [-0.25, -0.25, 1, -0.25, -0.25]
k_edge2 = [0.25, 0.25, -1, 0.25, 0.25]

# Apply convolution

s_edge1 = np.convolve(series1, k_edge1)
s_edge2 = np.convolve(series1, k_edge2)

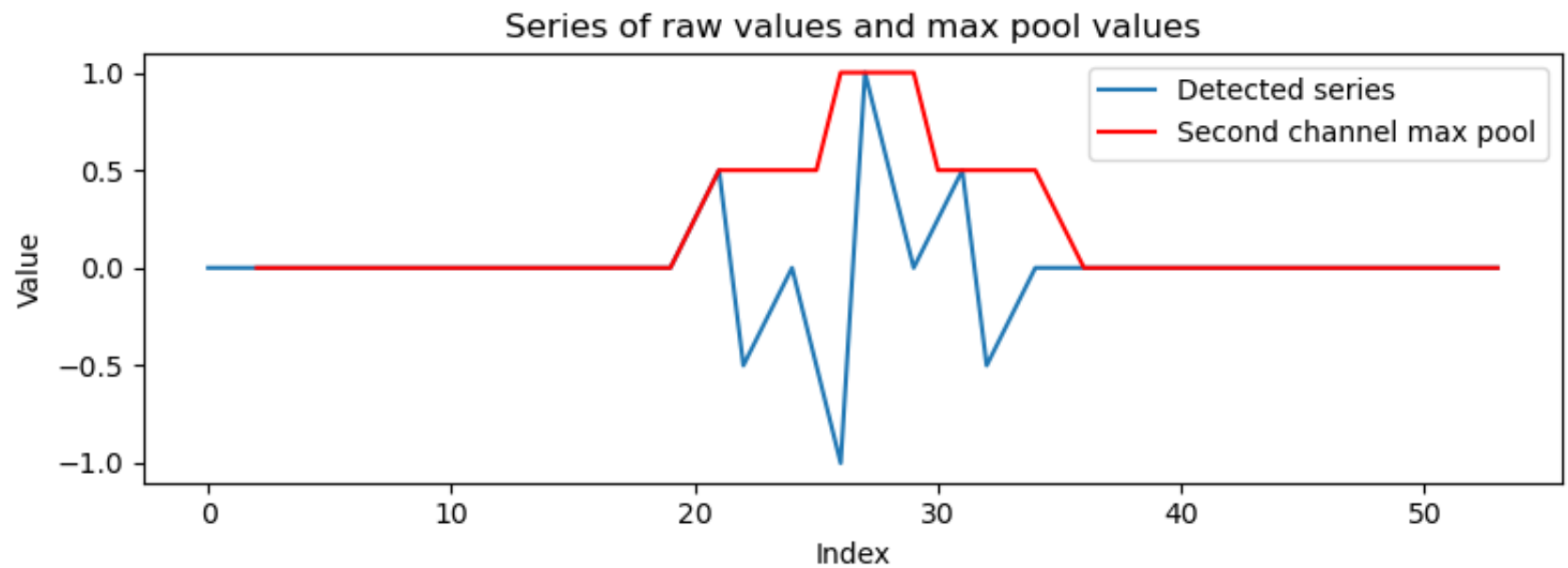
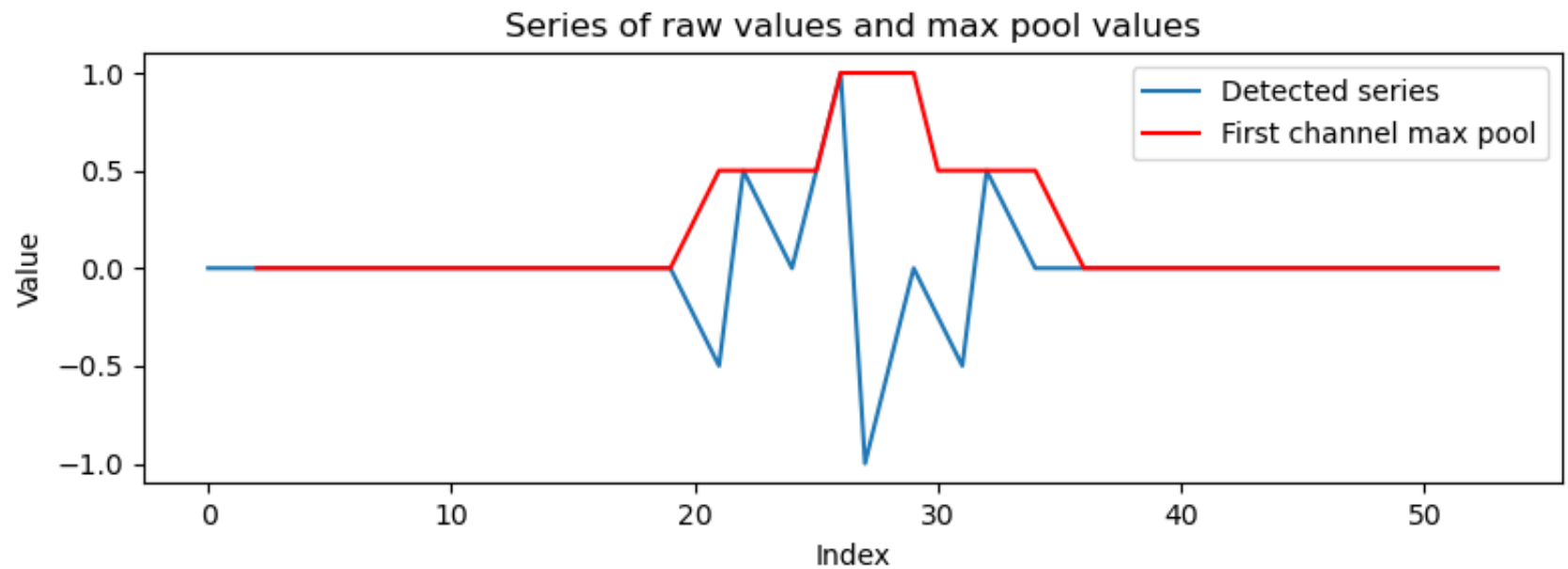
# Apply maxpool

s_edge1_mp = max_pool(s_edge1)
s_edge2_mp = max_pool(s_edge2)

# Display the results
plot_pool(s_edge1, s_edge2, s_edge1_mp, s_edge2_mp)
```

length of the input series = 54

length of the max pool series = 52



? Examine these results and answer these questions in one or a few sentences:

1. How does the max-pooling operator with span of preserve edges in the convolution output series?
2. How can you explain the invariance of the max-pooling results to the sign of the input series.
3. How can you explain the difference in the edge detection series and the max-pooled values between the two kernel spans in terms of the receptive field of the operator?

End of exercise.

 Answers:

1. The max-pooling operator preserves edges in the convolution output series by capturing both positive and negative peaks, taking their absolute value. These peaks arise from applying a Laplacian-like kernel to the time series edges, which functions as an edge detection filter.
2. The invariance of the max-pooling results to the sign of the input series can be explained by the Laplacian kernel generating two opposite-sign peaks at each edge location. Since max-pooling selects absolute values, it ensures that the representation remains invariant to whether the peak before an edge is positive or negative.
3. The difference between the two kernels can be explained by their receptive field sizes. That is 3 for the first kernel versus 5 for the second. As a result, the detected edge signals are wider for the second kernel. This proportionally increases the resulting receptive field of the max-pooling layer from 5 and 7, respectively (for input to max pooling output data propagation).

2.0 Convolutional neural networks with Keras

With the foregoing theory and simple examples, let's try constructing and testing a convolutional neural network using Keras. In this case, we will test classification of the MNIST dataset. The neural network will have the following layers:

1. An input layer for the 28X28 images.
2. A multi-layer convolutional neural network to create a feature map.
3. A fully-connected hidden layer to perform the classification.

4. A output layer to indicate which digit is most likely.

2.1 Preparing the dataset

The preparation of this dataset follows nearly identical steps to the process followed in an earlier lesson. In summary, three preparation steps are performed:

1. Load the training and test image data and labels.
2. Reshape the image tensors so the neural network views them as a 4-d tensor. For the training images the tensor has a shape of (60000,28,28,1) corresponding to 60000 images of dimension 28×28 with a single channel (these are gray-scale images). At the same time the pixel values are converted to `float` type in the range $\{0, 1\}$.
3. The labels are recoded as dummy variables using the Keras `to_categorical` function.

```
In [6]: (train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

```
In [7]: train_images = train_images.reshape((60000, 28, 28, 1)).astype('float')/255
print(train_images.shape)
print(train_images.dtype)
```

```
(60000, 28, 28, 1)
float64
```

```
In [8]: print(test_images.shape, test_labels.shape)
test_images = test_images.reshape((10000, 28, 28, 1)).astype('float')/255
print(test_images.shape)
```

```
(10000, 28, 28) (10000,)
(10000, 28, 28, 1)
```

```
In [9]: train_labels = ku.to_categorical(train_labels)
print(train_labels[5:,])
```

```
[[0. 0. 1. ... 0. 0. 0.]
 [0. 1. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 1. 0.]]
```

```
In [10]: test_labels = ku.to_categorical(test_labels)
```

Notice the one-hot encoding of the labels. There is one dummy variable (column) for each label value, $\{0, \dots, 9\}$.

2.2 Defining the model.

It is now time to define the model.

Early stopping regularization is an old idea in machine learning. The concept is simple, stop learning at the point the model starts to over-fit. Early stopping measures learning with a single selected validation metric. The early stopping algorithm can be describe as follows, using a validation metric like validation error rate or validation loss rate.

Proceedure Early_Stopping patience = initialPatience patienceCount = 0 maxSteps = initialMaxSteps steps = 0 bestMetric = 1.0 model = Instantiate_Model(trainData, validationData, hyperparameters) while patience >= patienceCount and steps <= maxSteps: model = Train(model) metric = Validate(model) if metric > bestMetic: patienceCount += 1 else bestMetric = metric bestModel = model steps += 1 return bestModel

As a first step, the callbacks required to implement early stopping regularization are defined. The steps to define the [early stopping call backs with Kera](#) are:

1. Define a path to an HDF5 file used to save the training history.
2. The first callback stops the training when the validation loss does not improve within 4 epochs, the patience.
3. The second callback saves the model weights resulting from the epoch with the best validation loss.

```
In [11]: ## Set up and call-backs for early stopping
patience = 2
filepath = 'my_model_file.keras' # define where the model is saved
callbacks_list = [
```

```

callbacks.EarlyStopping(
    monitor = 'val_loss', # Use accuracy to monitor the model
    patience = patience # Stop after patience steps with lower accuracy
),
callbacks.ModelCheckpoint(
    filepath = filepath, # file where the checkpoint is saved
    monitor = 'val_loss', # Don't overwrite the saved model unless val_loss is worse
    save_best_only = True # Only save model if it is the best
)
]

```

► **Exercise 6-2:** You will now use Keras to define the CovNet model named `nn`. This model uses both convolutional layers and max pooling layers. You will define this model by completing the function in the code cell below. You can find [documentation on the Keras convolutional layers here](#). You can find [documentaton on Keras pooling layers here](#). The architecture of the model you will create is:

1. A 3X3 convolutional layer with 32 output channels using the `Conv2D` function, with `activation='relu'`. Recall that this operation creates a feature map with 32 channels from the single gray-scale input channel. This is the input layer of the model, so make sure you define the input shape.
2. A 2X2 max pooling layer using the `MaxPooling2D` function.
3. A dropout layer is used to improve regularization of the model using the `Dropout` function with argument 0.2.
4. A 3X3 convolution layer with 64 output channels, with `activation='relu'`. The 32 channels of the input layer are mapped to a 64 channel feature map.
5. Another 2X2 max pooling layer.
6. A dropout layer to improve regularization of the model with argument 0.2.
7. A final 3X3 convolution layer with 64 output channels, with `activation='relu'`.
8. A dropout layer to improve regularization of the model with argument 0.2.
9. Flatten the 64 channel feature map to a vector.
10. A Dense (fully-connected) hidden layer with 64 units is the first classifier layer, with `activation='relu'`. A limited amount of regularization is applied using Keras [layer regularizer](#)

- 12 as a `kernel_regularizer`, with hyperparameter 0.1. This layer operates on the flattened feature map.
11. A dropout layer to improve regularization of the model with argument 0.2.
 12. The output Dense layer has 10 units with `activation='softmax'` to indicate the classification of the digits.
 13. A summary of the model is printed.
 14. The model object is returned. Execute your code and examine the summary of the model.

```
In [12]: INPUT_SHAPE = train_images.shape[1:]
```

```
In [13]: def create_model(verbose=1):

    inputs = layers.Input(INPUT_SHAPE)

    # Feature extraction

    x = layers.Conv2D(32, (3,3), activation='relu')(inputs)
    x = layers.MaxPool2D((2,2))(x)
    x = layers.Dropout(0.2)(x)

    x = layers.Conv2D(64, (3,3), activation='relu')(x)
    x = layers.MaxPool2D((2,2))(x)
    x = layers.Dropout(0.2)(x)

    x = layers.Conv2D(64, (3,3), activation='relu')(x)
    x = layers.Dropout(0.2)(x)

    # Inference

    x = layers.Flatten()(x)
    x = layers.Dense(64)(x)
    x = layers.Dropout(0.2)(x)

    outputs = layers.Dense(10, activation='softmax')(x)

    # Build and compile the model
```

```
model = models.Model(inputs=inputs, outputs=outputs)

# Preview

if verbose:
    model.summary()

return model

nn = create_model()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
dropout (Dropout)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
dropout_1 (Dropout)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 64)	36,928
dropout_2 (Dropout)	(None, 3, 3, 64)	0
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 64)	36,928
dropout_3 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 10)	650

Total params: 93,322 (364.54 KB)

Trainable params: 93,322 (364.54 KB)

Non-trainable params: 0 (0.00 B)

? There is quite bit of interesting information in the summary of this model. In one or a few sentences answer the following questions:

1. How does the dimensionality of the feature map change at each convolution and max pooling layer?

2. Compare the number of parameters of your CovNet model to the fully connected model you created for exercise 5-4. How different is the overall number of parameters and layers?
3. What does the difference in the numbers of parameters between the CovNet and the fully connected model tell you about the ability to train these models with a finite amount of data?
4. How many learnable parameters do the max-pooling layers have, and why?

End of exercise.

✓ Answers:

1. We're using 3x3 convolutions with no padding preserving layer input size. Each such convolution reduces both image dimensions by 2 pixels. At the same time, each 2x2 max pooling reduces both dimensions by a factor of 2.
2. The overall number of parameters in a Exercise "5-4" FFNN is 4 times greater than in the convolutional model, while the CNN above number of layers is opposite 6x times greater than FFNN. Not all of these layer have trainable params, but still.
3. The difference in the number of parameters between the CNN and the FFNN models suggests that, when training with a finite amount of data, the CNN will be less prone to overfitting and faster to train and infer.
4. Max pooling layers have no learnable parameters. They traverse across each feature map with a specified stride, and pick a maximum value within their sliding window for each step.

2.3 Train and test the model

Now, it is time to compile and fit your model. This particular model appears to require only limited regularization. The table below shows the limited heuristic search performed on the hyperparamters.

Final Accuracy	Dropout	weight decay	Learning Rate
0.9923	0.2	0.00	0.001
0.9923	0.2	0.001	0.001

0.9890 0.5 0.001 0.001

The code in the cell below compiles the model using the chosen optimizer with learning rate and weight decay hyperparameters selected. The model is then fit and the history is captured. Run the code in the cell below, which may take some time.

```
In [14]: ## Compile the model
nn.compile(optimizer=RMSprop(learning_rate=0.001, weight_decay=0.001),
           loss = 'categorical_crossentropy', metrics = ['accuracy', 'top_k_categorical_accuracy'])

## Now fit the model
history = nn.fit(train_images, train_labels,
                 epochs = TEST_EPOCHS or 80, batch_size = 256,
                 validation_data = (test_images, test_labels),
                 callbacks = callbacks_list, # Call backs argument here
                 verbose = 1)
```

Epoch 1/80

235/235 ————— **23s** 94ms/step – accuracy: 0.7451 – loss: 0.7751 – top_k_categorical_accuracy: 0.9449 – val_accuracy: 0.9722 – val_loss: 0.0933 – val_top_k_categorical_accuracy: 0.9997

Epoch 2/80

235/235 ————— **21s** 89ms/step – accuracy: 0.9597 – loss: 0.1331 – top_k_categorical_accuracy: 0.9992 – val_accuracy: 0.9839 – val_loss: 0.0517 – val_top_k_categorical_accuracy: 1.0000

Epoch 3/80

235/235 ————— **21s** 90ms/step – accuracy: 0.9749 – loss: 0.0824 – top_k_categorical_accuracy: 0.9995 – val_accuracy: 0.9828 – val_loss: 0.0526 – val_top_k_categorical_accuracy: 0.9999

Epoch 4/80

235/235 ————— **21s** 91ms/step – accuracy: 0.9805 – loss: 0.0632 – top_k_categorical_accuracy: 0.9997 – val_accuracy: 0.9902 – val_loss: 0.0325 – val_top_k_categorical_accuracy: 1.0000

Epoch 5/80

235/235 ————— **41s** 90ms/step – accuracy: 0.9834 – loss: 0.0521 – top_k_categorical_accuracy: 0.9999 – val_accuracy: 0.9894 – val_loss: 0.0293 – val_top_k_categorical_accuracy: 0.9999

Epoch 6/80

235/235 ————— **24s** 100ms/step – accuracy: 0.9864 – loss: 0.0435 – top_k_categorical_accuracy: 0.9998 – val_accuracy: 0.9907 – val_loss: 0.0267 – val_top_k_categorical_accuracy: 1.0000

Epoch 7/80

235/235 ————— **22s** 94ms/step – accuracy: 0.9868 – loss: 0.0437 – top_k_categorical_accuracy: 0.9999 – val_accuracy: 0.9906 – val_loss: 0.0288 – val_top_k_categorical_accuracy: 0.9999

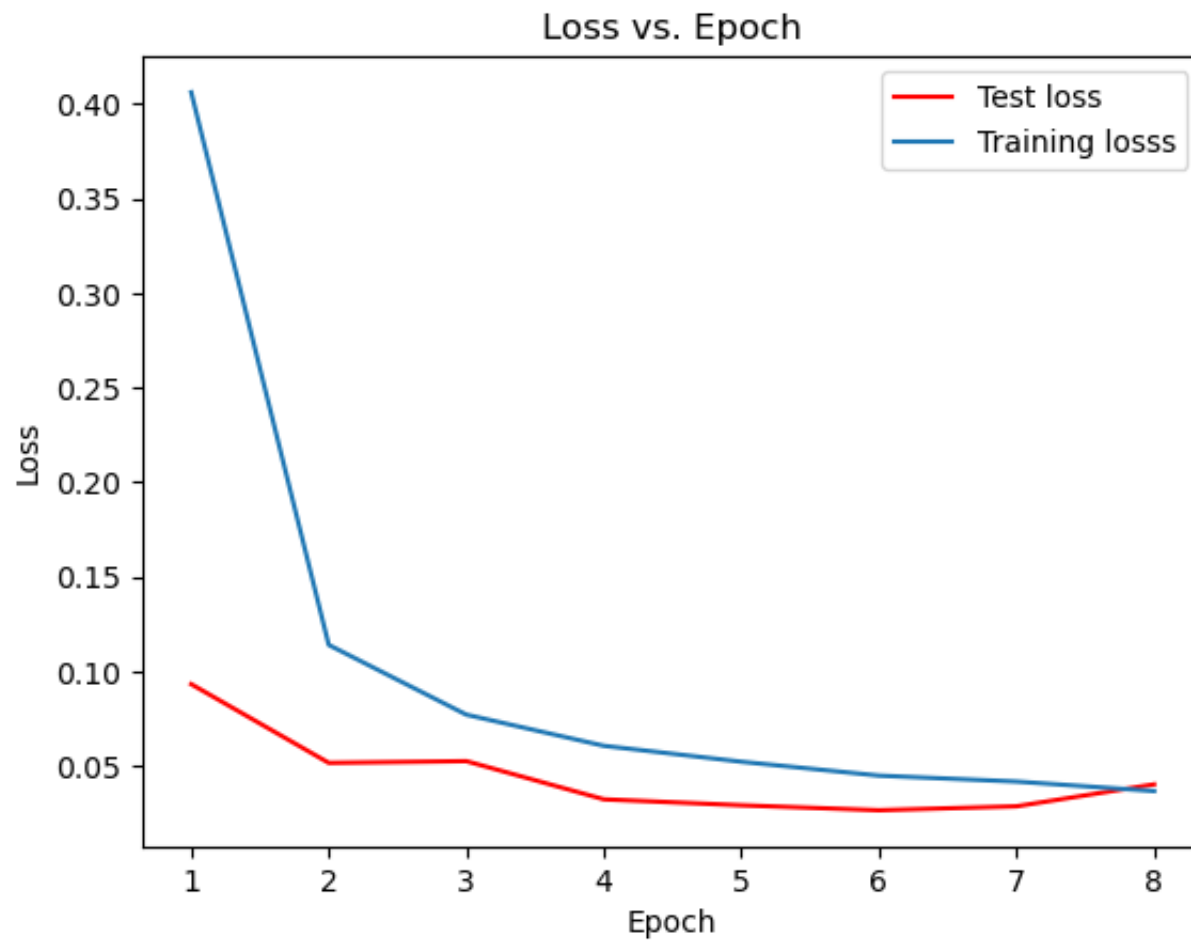
Epoch 8/80

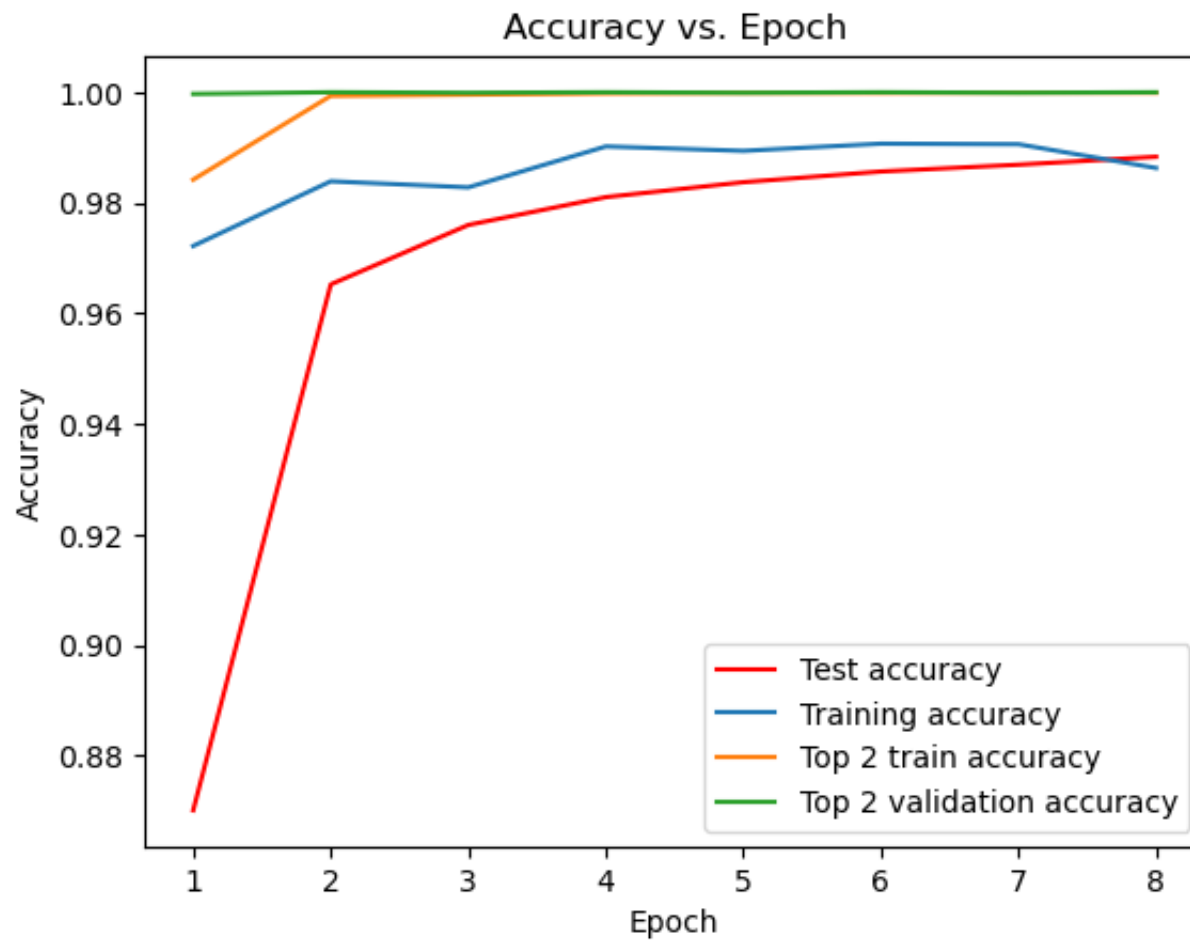
235/235 ————— **23s** 98ms/step – accuracy: 0.9876 – loss: 0.0392 – top_k_categorical_accuracy: 0.9999 – val_accuracy: 0.9863 – val_loss: 0.0404 – val_top_k_categorical_accuracy: 1.0000

Run the code in the cells below to plot the loss and accuracy history vs. epoch.

```
In [15]: def plot_loss(history):
    '''Function to plot the loss vs. epoch'''
    train_loss = history.history['loss']
    test_loss = history.history['val_loss']
    x = list(range(1, len(test_loss) + 1))
    plt.plot(x, test_loss, color = 'red', label = 'Test loss')
    plt.plot(x, train_loss, label = 'Training loss')
    plt.legend()
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Loss vs. Epoch')
    plt.show()
```

```
def plot_accuracy(history):  
    x = list(range(1, len(history.history['accuracy']) + 1))  
    plt.plot(x, history.history['accuracy'], color = 'red', label = 'Test accuracy')  
    plt.plot(x, history.history['val_accuracy'], label = 'Training accuracy')  
    plt.plot(x, history.history['top_k_categorical_accuracy'], label = 'Top 2 train accuracy')  
    plt.plot(x, history.history['val_top_k_categorical_accuracy'], label = 'Top 2 validation accuracy')  
    plt.legend()  
    plt.xlabel('Epoch')  
    plt.ylabel('Accuracy')  
    plt.title('Accuracy vs. Epoch')  
    plt.show()  
  
plot_loss(history)  
plot_accuracy(history)
```





► ? **Exercise 6-3:** Examine these results and answer the following questions:

1. How does the accuracy and error rate of your CovNet model compare to the regularized fully connected model of Exercise 5-10 of the previous assignment?
2. How can you characterize the convergence or learning rate of the CovNet model after the first 10 epochs.

✓ **Answers:**

1. The CNN accuracy is 98.63%, which is ~0.5% higher than that of the FFNN, resulting in an error rate that is approximately 30% lower than the FFNN's.
2. The model reaches its asymptotic convergence around the epoch 10, indicating that the model is no longer learning. With this effective learning rate can be considered reduced to zero.

Now you will continue with the evaluation of the model. One can compute several possible accuracy metrics.

First, you will use conventional or **top-1** accuracy along with **top-k** accuracy, in this case top-2. Top-1 accuracy measures the fraction of cases where the correct category is predicted. Top-k accuracy measures the fraction of cases where the correct category is in the top k probabilities of categories. Since there are only 10 categories for this problem we are limiting to $k = 2$. For cases with many categories it is common to use top-5 accuracy.

You will also compute and display the mean average precision and mean average recall.

Execute this code in the cell below to compute and display these accuracy metrics along with average precision and average recall.

```
In [16]: def evaluate_model(model, test_imgs, test_labels):
    ## Compute class predictions
    class_probabilities = model.predict(test_images)
    class_predictions = class_probabilities.argmax(axis=1)

    ## Compute the top-1 categorical accuracy.
    categorical_accuracy = CategoricalAccuracy()
    categorical_accuracy.update_state(test_labels, class_probabilities)

    # Compute the top-2 accuracy.
    top_2_accuracy = TopKCategoricalAccuracy(k=2, name='top_2_categorical_accuracy')
    top_2_accuracy.update_state(test_labels, class_probabilities)

    ## Summarize and print the result.
    print('Categorical accuracy = ' + str(categorical_accuracy.result().numpy()))
    print('Top 2 accuracy = ' + str(top_2_accuracy.result().numpy()))

    ## Get the class labels, counts and compute the class-specific recall and precision.
```

```

test_labels_vector = test_labels.argmax(axis=1)
unique_labels, label_counts = np.unique(test_labels_vector, return_counts=True)
class_precision = metrics.precision_score(test_labels_vector, class_predictions, labels=unique_labels)
class_recall = metrics.recall_score(test_labels_vector, class_predictions, labels=unique_labels)

## Compute and print the mean average precision and recall accounting for class frequency weights
sum_label_counts = np.sum(label_counts)
weighted_average = lambda x: round(np.sum(np.divide(x * label_counts, sum_label_counts)), 4)
print('Average precision = ' + str(weighted_average(class_precision)))
print('Average recall = ' + str(weighted_average(class_recall)))

## Compute, display and plot the confusion matrix.
confusion_matrix = metrics.confusion_matrix(test_labels_vector, class_predictions)
print('\nConfusion matrix')
print(pd.DataFrame(confusion_matrix))
p = plt.imshow(np.log(np.divide(confusion_matrix + 1.0, np.sum(confusion_matrix, axis=1))))
cb = plt.colorbar(p)
cb.set_label('Log count')

evaluate_model(nn, test_images, test_labels)

```

313/313  1s 3ms/step

Categorical accuracy = 0.9863

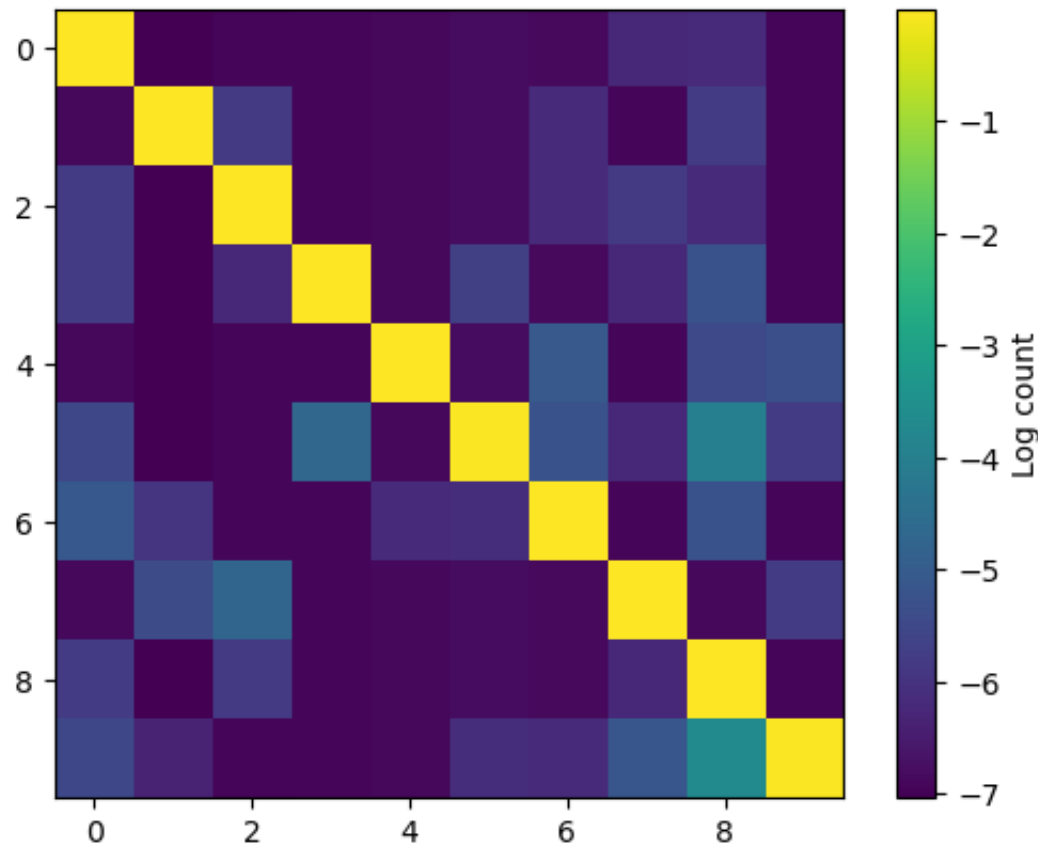
Top 2 accuracy = 0.9981

Average precision = 0.9866

Average recall = 0.9863

Confusion matrix

	0	1	2	3	4	5	6	7	8	9
0	978	0	0	0	0	0	0	1	1	0
1	0	1130	2	0	0	0	1	0	2	0
2	2	0	1026	0	0	0	1	2	1	0
3	2	0	1	1000	0	2	0	1	4	0
4	0	0	0	0	970	0	5	0	3	4
5	3	0	0	8	0	857	4	1	17	2
6	5	2	0	0	1	1	945	0	4	0
7	0	4	8	0	0	0	0	1014	0	2
8	2	0	2	0	0	0	0	1	969	0
9	3	1	0	0	0	1	1	5	24	974



► ? **Exercise 6-4:** Examine the evaluation metrics of the model. Notice that the accuracy, mean average precision and mean average recall are all high. Now, answer these questions:

1. Consider the differences between the top-1 (categorical) accuracy and top-2 accuracy. Is the difference expected for this model and why?
2. Why are average precision and average recall values computed expected for this model?

✓ **Answers:**

1. The difference between TOP1 and TOP2 accuracy for this model makes sense. The softmax output

probabilities reflect digits images similarities within their classes. So, if a particular data point falls out of its correct TOP1 category, the next highest probable category (TOP2) is likely to capture the true value with high probability, still placing it in TOP2.

2. The average precision and recall make sense due to the well-balanced dataset classes distribution.

3.0 What features does the convolutional neural network learn?

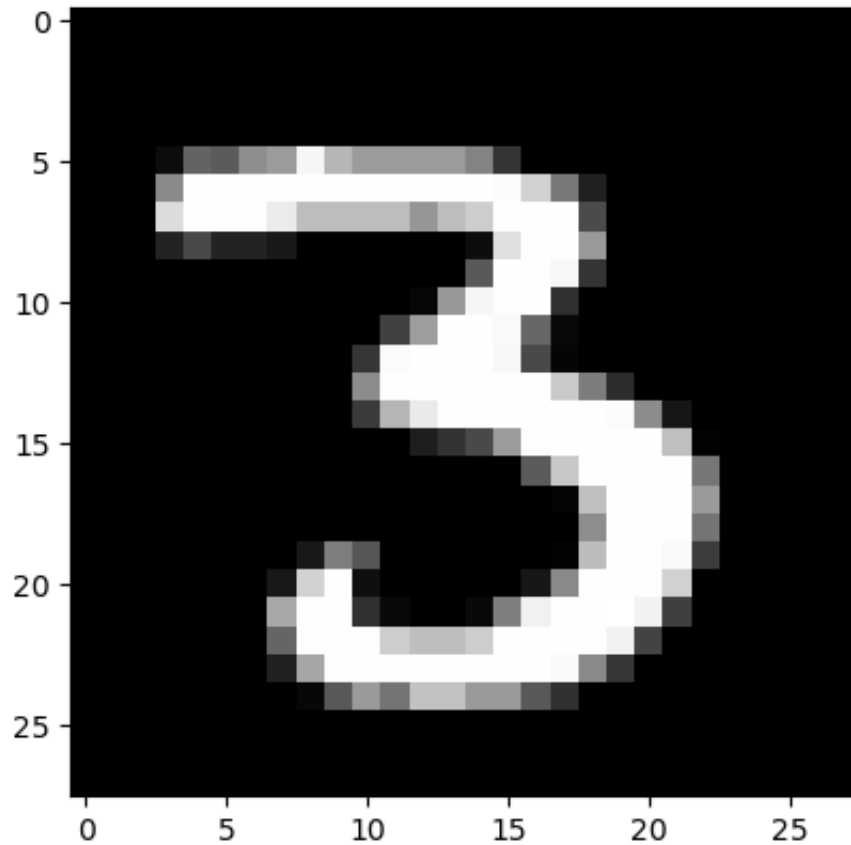
You might well ask, what is in the various layers of the feature map? It turns out, we can actually visualize the output from each layer of the convolutional neural network to get a feel for the learned features. This can be done for both convolutional and max pooling layers.

As a first step we must pick an image used as the test case. For this example, we arbitrarily choose the 13th image in the training tensor. Execute the code in the cell below to visualize this image.

```
In [17]: img = train_images[12,:,:,:]
print(img.shape)
plt.imshow(img.reshape((28,28)), cmap='gray')
```

(28, 28, 1)

```
Out[17]: <matplotlib.image.AxesImage at 0x180c55410>
```



Our sample image turns out to be a number 3.

The code in cell below does the following:

1. Extracts the layers from the model object into a list.
2. Creates an activation map for each layer using the original model object and the list of layer outputs.
3. Computes the activations for each layer by applying the predict method to the activation map.

Note: In some combinations of environment and versions of Keras and Tensorflow, you may need to change the name of the model object attribute, `nn.input` to `nn.inputs`.

We will now extract the activations from the trained model by the following steps.

3.1 - Extract the layers from the model

The code below creates a list of the output layers we want to visualize. Execute this code to create the list of these layers.

```
In [18]: layer_names = np.array([layer.name for layer in nn.layers]) # create list of layer names
layer_indx = np.array([0,1,3,4,6]) + 1 # indices of layers we want to visualize
layer_outputs = [nn.get_layer(layer_names[i]).output for i in layer_indx]
layer_outputs
```

```
Out[18]: [<KerasTensor shape=(None, 26, 26, 32), dtype=float32, sparse=False, name=keras_tensor_1>,
<KerasTensor shape=(None, 13, 13, 32), dtype=float32, sparse=False, name=keras_tensor_2>,
<KerasTensor shape=(None, 11, 11, 64), dtype=float32, sparse=False, name=keras_tensor_4>,
<KerasTensor shape=(None, 5, 5, 64), dtype=float32, sparse=False, name=keras_tensor_5>,
<KerasTensor shape=(None, 3, 3, 64), dtype=float32, sparse=False, name=keras_tensor_7>]
```

```
In [19]: print(f'Layers to preview:\n\n-', '\n- '.join(layer_names[layer_indx]))
```

Layers to preview:

```
- conv2d
- max_pooling2d
- conv2d_1
- max_pooling2d_1
- conv2d_2
```

3.2 - Create and compute an activation model

With the layers we want to visualize the activations defined, we can now build a model and extract the activations. This temporary model inherits the weights already learned for the full model. The activations are computed by applying the predict method to the model, with a correctly shaped image tensor as input. Execute the code in the cell below.

```
In [20]: activation_model = models.Model(inputs=nn.inputs, outputs=layer_outputs)

img_tensor = np.expand_dims(img, axis=0) # shape becomes (28, 28, 1, )
```

```
activations = activation_model.predict(img_tensor)
```

1/1 ————— 0s 97ms/step

```
/usr/local/anaconda3/envs/e25/lib/python3.11/site-packages/keras/src/models/functional.py:237: User
Warning: The structure of `inputs` doesn't match the expected structure.
Expected: ['keras_tensor']
Received: inputs=Tensor(shape=(1, 28, 28, 1))
warnings.warn(msg)
```

3.3 Display the activations

Now that the activations have been computed for the test image, it is time to display them. The code in the cell below does the following for each layer of interest:

1. A plot grid is layed out for each channel in the layer.
2. The activations for each channel of the layer is plotted on the grid.

Execute the code in the cell and examine the resuling activation plots.

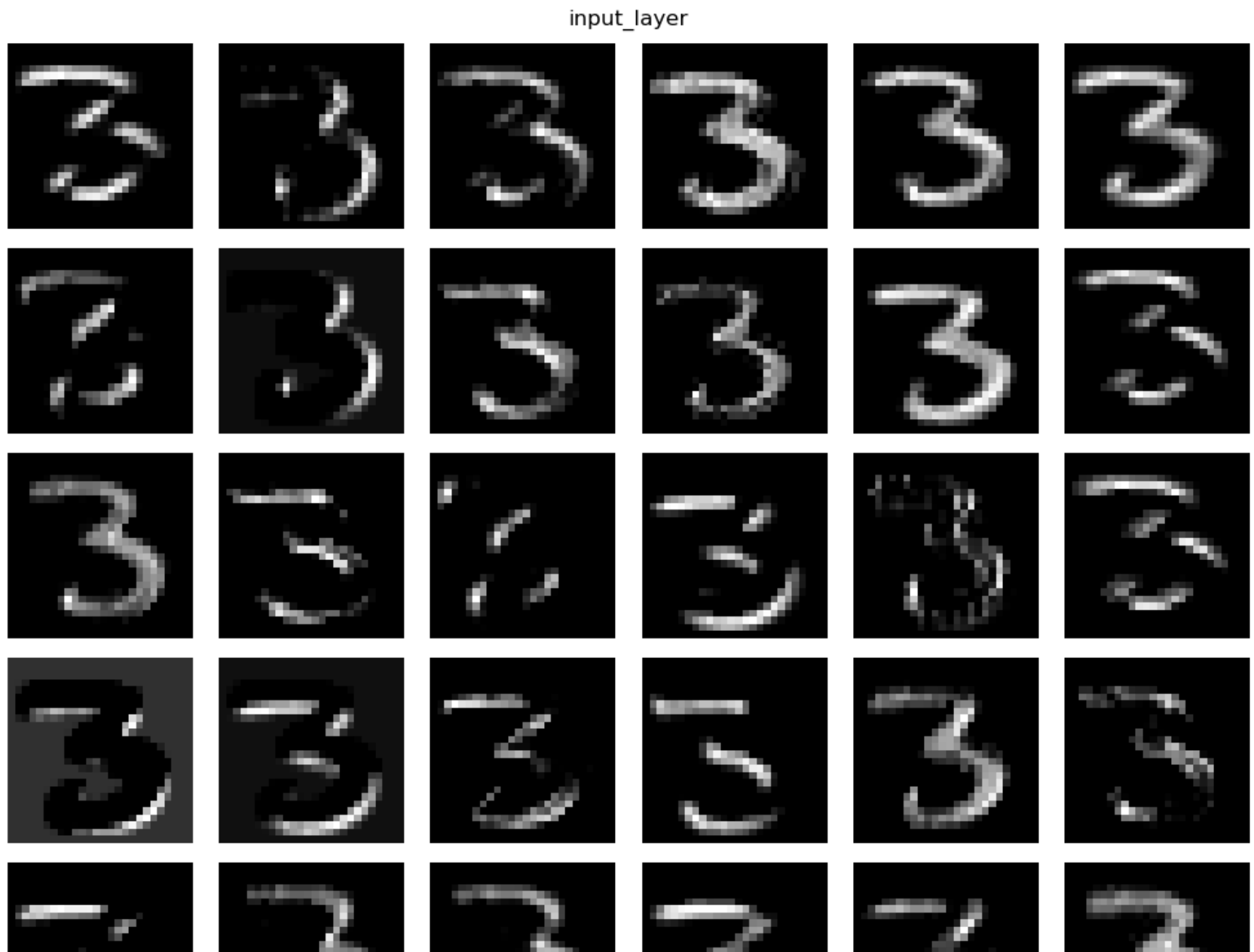
```
In [21]: # Visualize each layer's activations
for layer_name, layer_activation in zip(layer_names, activations):
    # Drop the batch dimension: shape (height, width, channels)
    activation_maps = layer_activation[0]
    num_channels = activation_maps.shape[-1]

    # Compute a grid size so all channels fit in a square
    grid_size = int(math.ceil(math.sqrt(num_channels)))

    fig, axes = plt.subplots(grid_size, grid_size, figsize=(10, 10))
    axes = axes.flatten()
    fig.suptitle(layer_name)

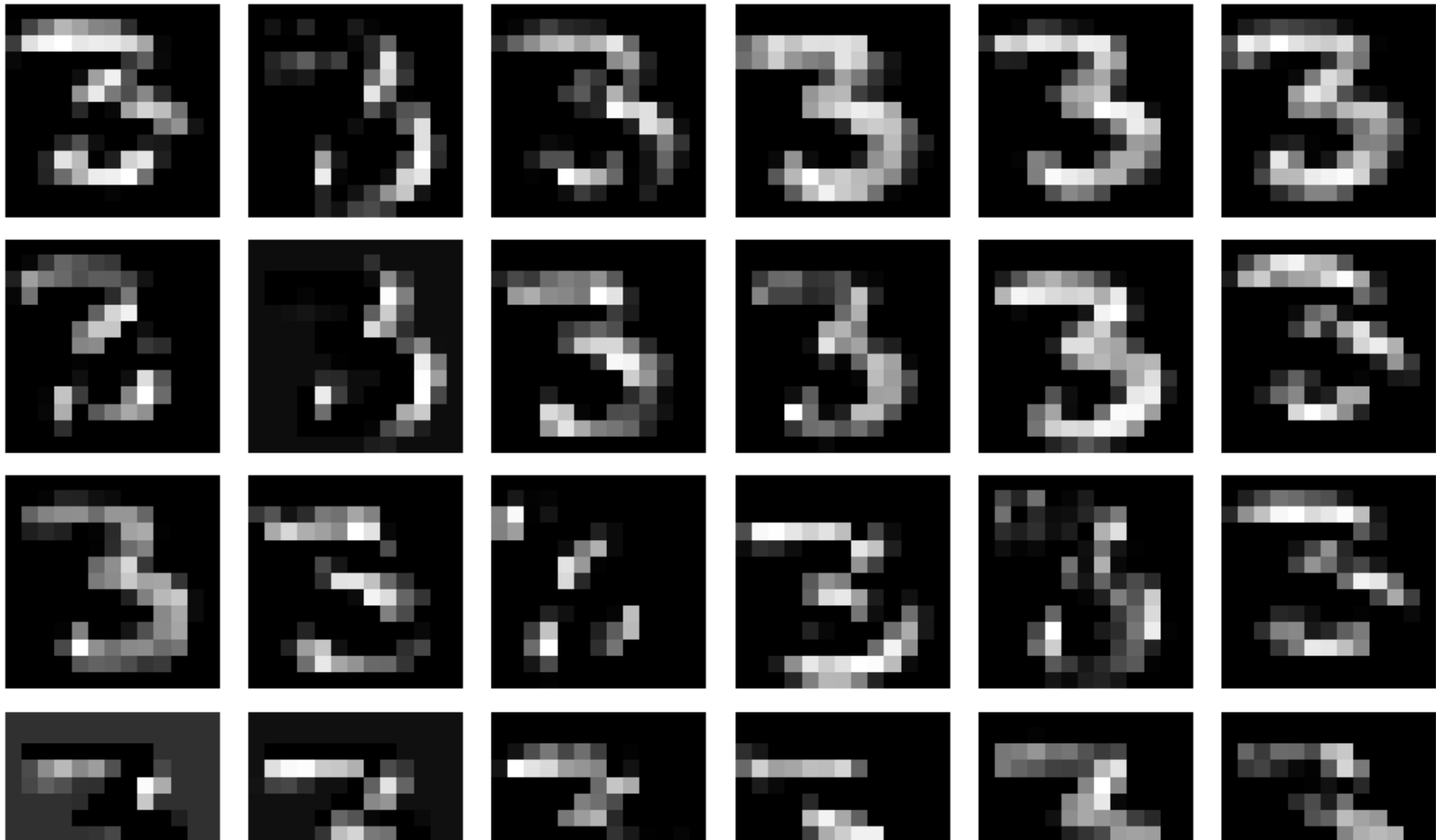
    # Plotting each channel
    for i in range(num_channels):
        axes[i].imshow(activation_maps[...], i], cmap='gray')
        axes[i].axis('off')
```

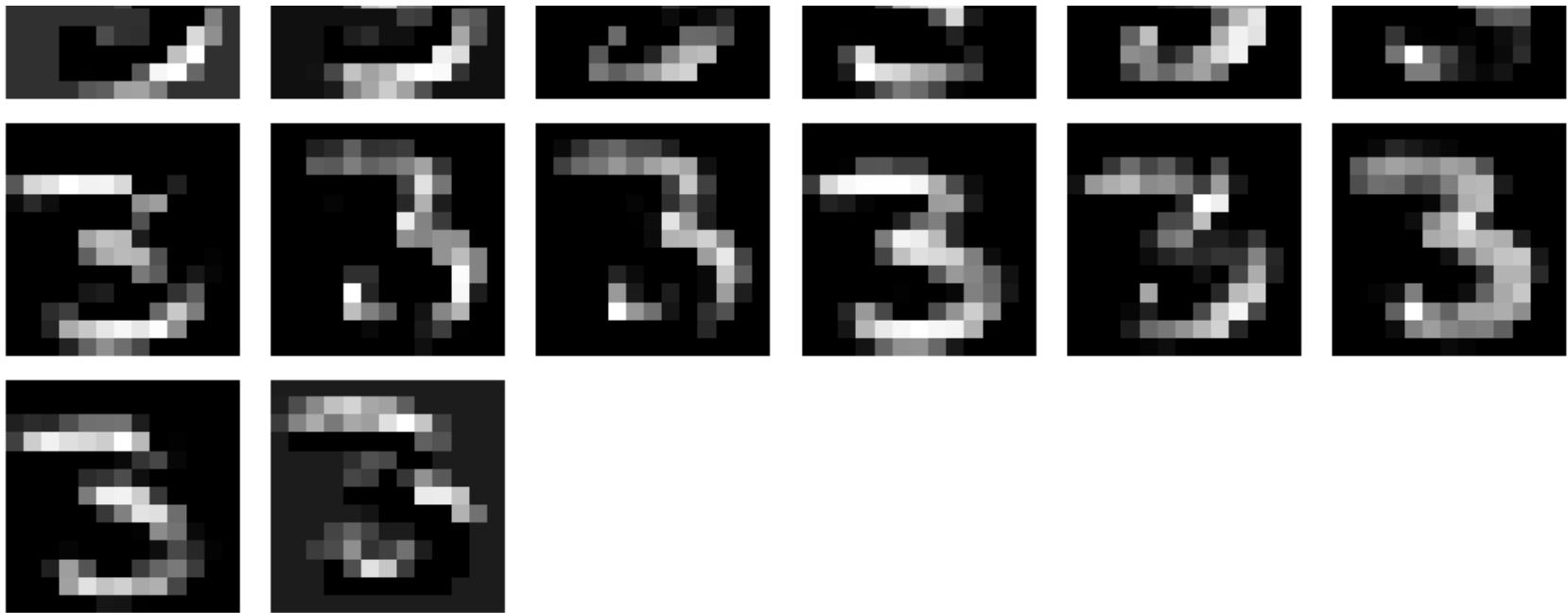
```
for i in range(num_channels, grid_size * grid_size):  
    axes[i].axis('off')  
plt.tight_layout()  
plt.show()
```



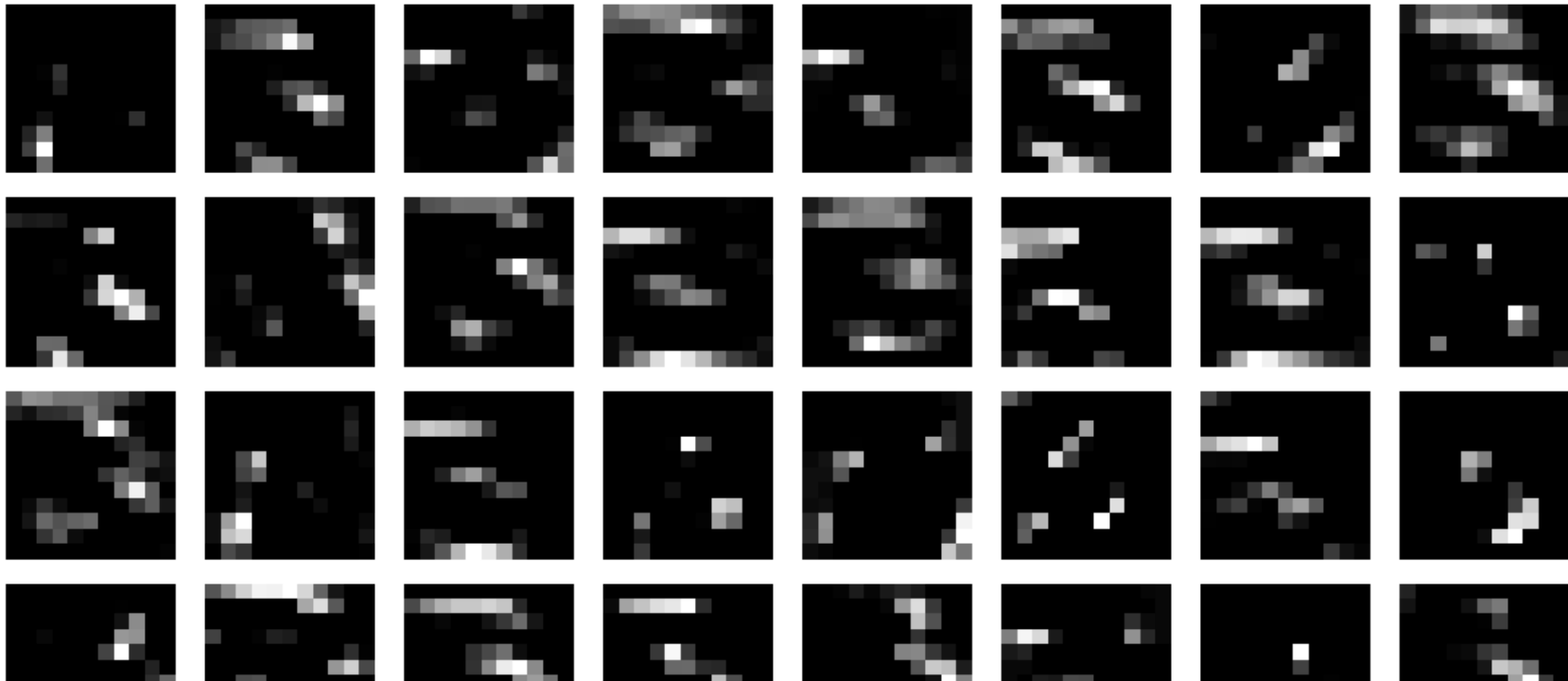


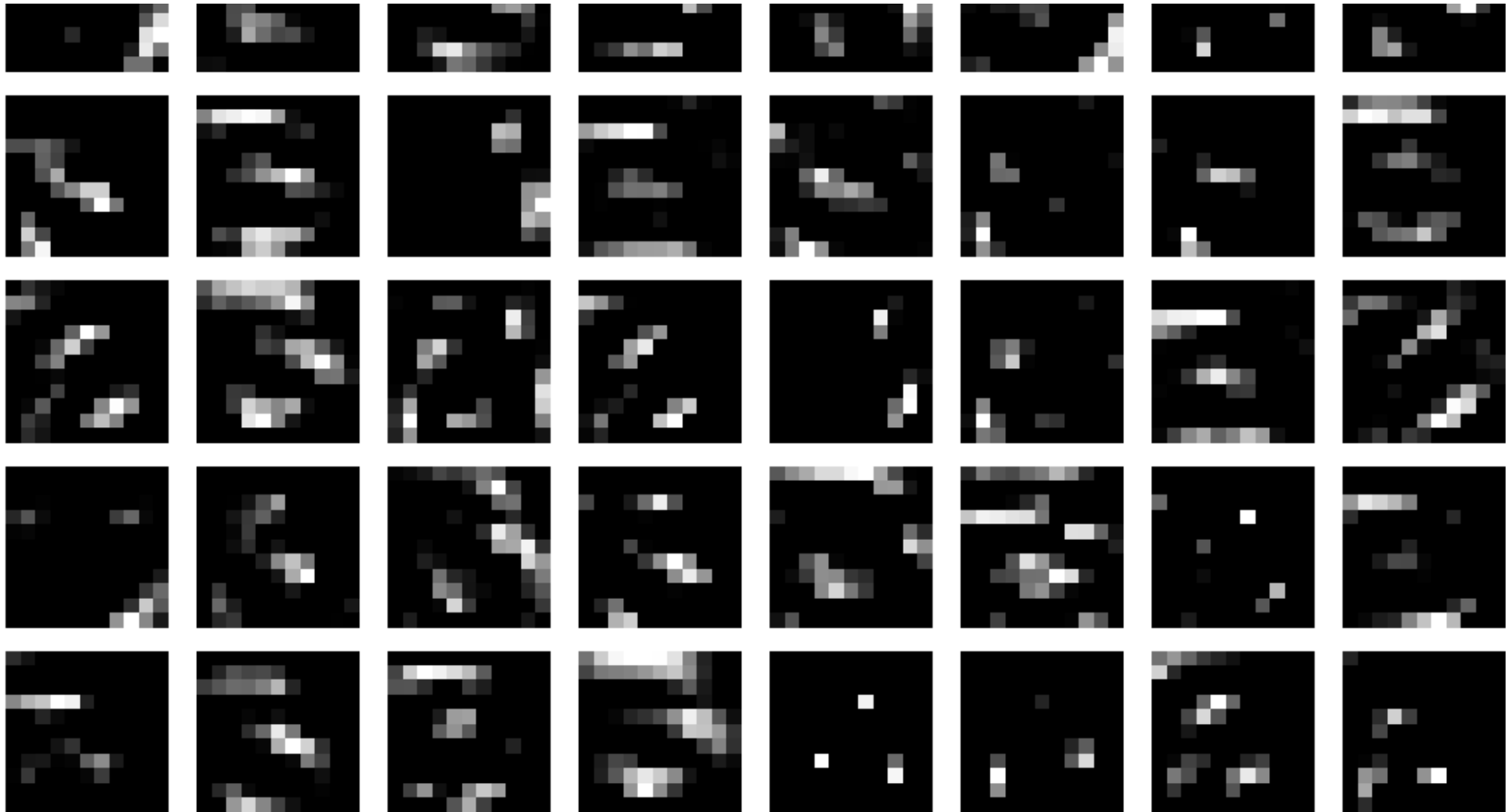
conv2d



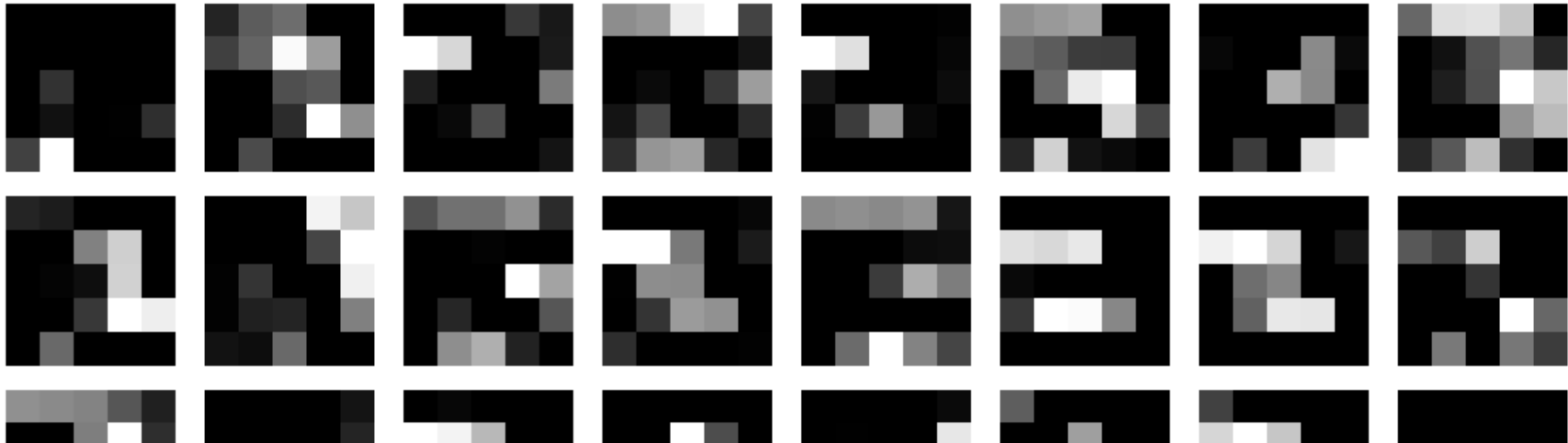


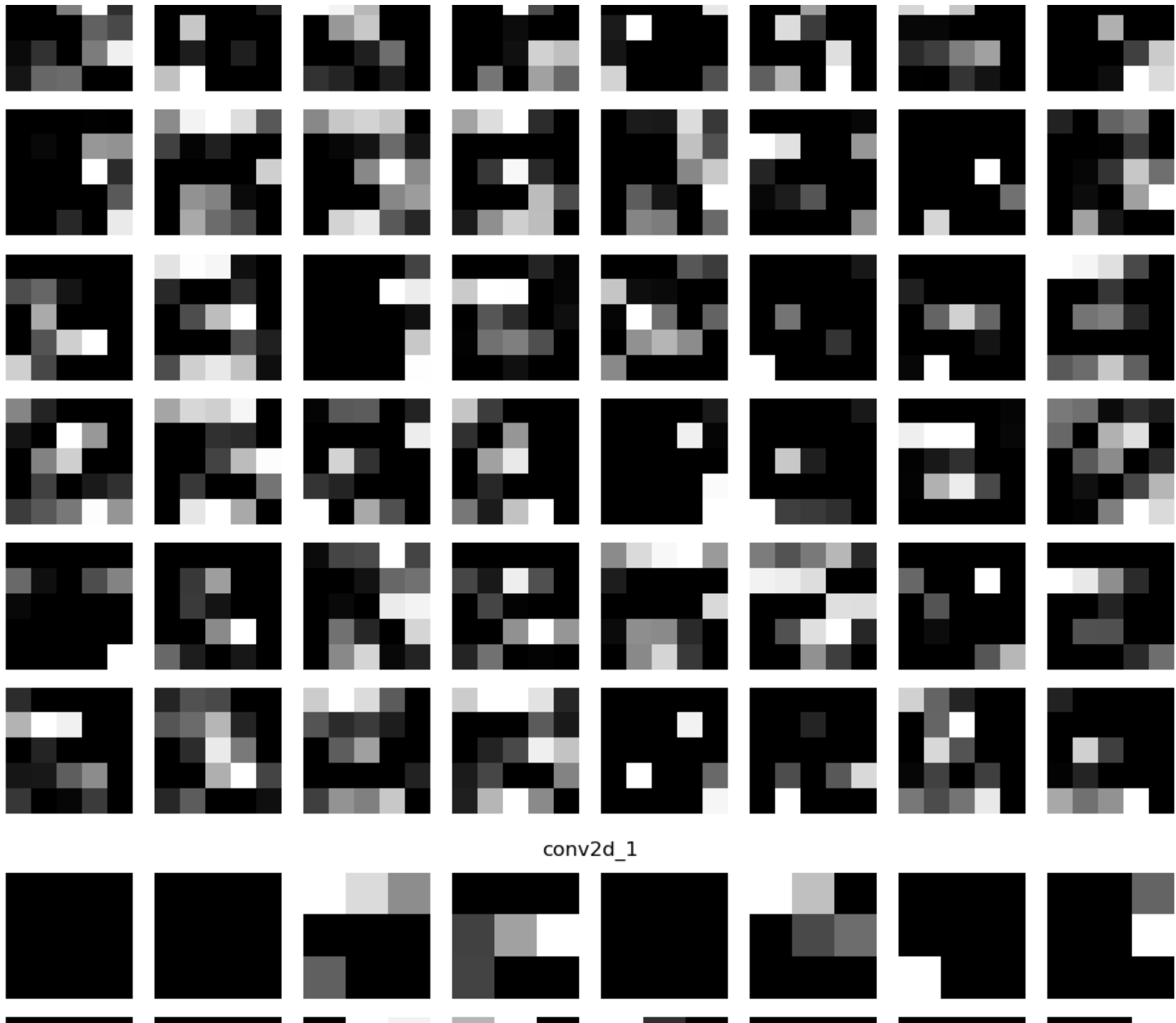
max_pooling2d

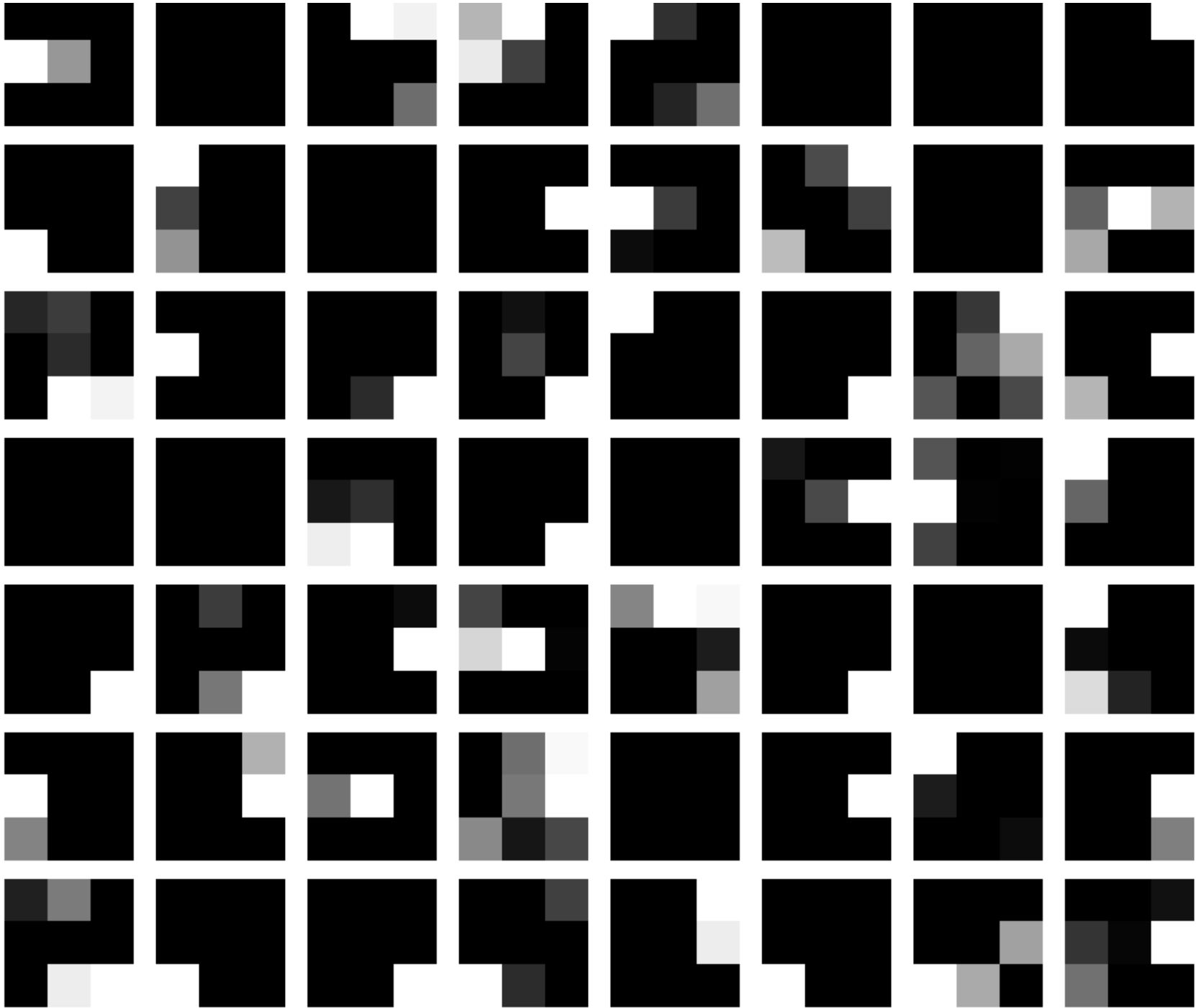




dropout







► ? **Exercise 6-5:** Examine the above sets of images. There are 5 sets of images, one for each of the output of the first 5 layers of the model. For each layer, there is an image showing each channel of the feature map. Keeping in mind that CNNs are known to be effective as finding find-grain features, in one or a few sentences, answer the following questions:

1. The initial single channel $28 \times 28 \times 1$ input image tensor becomes a $32 \times 26 \times 26 \times 1$, 32 channel feature map after the first convolution. The second $32 \times 13 \times 13 \times 1$ output is from the max-pooling layer. In general terms, describe how much these feature maps resemble the original image, and why do you think this might be the case given the receptive field of the kernels?
2. Examine the feature maps created by the forth (convolution), fith (max-pooling), and seventh (convolution) layers. How can you best describe the level of abstraction of these layers and the the resemblance to the original image?
3. What effect can you observe of using the max-pooling layer other than dimensionality reduction, and why?
4. WHat relationship can you identify between the spatial dimensionality of the feature map and the level of abstraction and localization of the features? **End of exercise.**

✓ **Answers:**

1. The first convolution + max pooling transformation layers produce features that still resemble the original image quite well. Each pixel in the max pooling output has 4×4 pixel receptive field in the input. This is a significant but still recognizable transformation. Also, the first convolution layer features aren't too abstract yet (compared to deeper layers).
2. The second and third convolution layers, along with the following max pooling layers add another level of abstraction on to of the first convolution's + max pool features. These 2-nd and 3-rd level abstraction features no longer resemble the original image for a human eye.
3. Max pooling makes the extracted features bolder simultaneously suppressing small artifacts, or in many cases - noise.
4. The relationship between the spatial dimensionality of the feature map and the level of abstraction

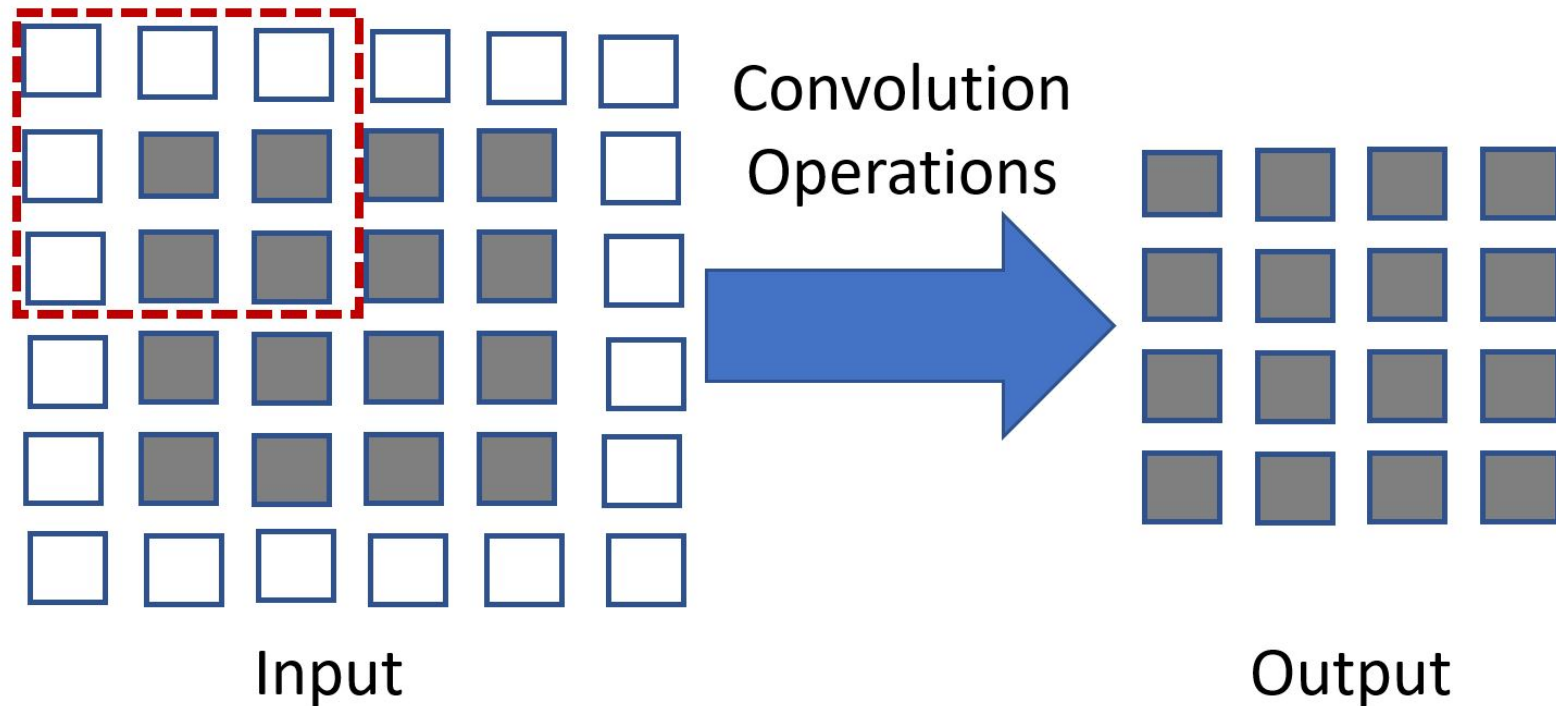
is inversely proportional. The deeper the level and smaller the spatial dimensionality of the feature map, the more abstract it would be.

4.0 Padding convolution

As you have observed, each convolution layer in a neural network reduces the spatial dimensions by $\frac{span+1}{2}$ for odd span kernels. The coverage of the convolution operator can be expanded by **zero padding** in the spatial dimension. With the zero padding added, the convolution operator covers the entire spatial dimension of the input tensor. This type of convolution is sometime referred to as **same convolution**, since the output tensor has the same spatial dimensions.

In some situations, adding zero padding produces a better feature map. Zero padding allows each pixel to be visited the same number of times by the convolution operator. Without padding pixels at the edge, are not full represented in the feature map. More importantly, the larger feature map will have bigger model capacity.

2-d convolution with zero padding is illustrated in Figure 4.1 below. The original input 2-d input tensor is shown in gray. The span of the kernel is 3X3 so one additional column of zeros is added to each sided along with a row of zeros top and bottom. As illustrated, the kernel can operate on the edge pixels. The result is an output tensor with the same spatial dimension as the input.



****Figure 4.1. 2-d convolution with zero padding****

► **Exercise 6-6:** With Keras, adding zero padding to convolutional layers is easy. The `padding = same` argument adds zero padding. The default is `padding = valid` adds no padding. Create a new model, with a new name 'nns', with the same layers as the foregoing model, and a dropout rate of 0.2, but with the `padding = same` argument for the convolutional layers. Then print the summary of the model.

```
In [22]: ## Your code goes below
nns = models.Sequential()
dropout=0.2

## Add the convolutional layers to extract features
nns.add(layers.Input(INPUT_SHAPE))
```

```
nns.add(layers.Conv2D(32, (3,3), activation='relu', padding='same'))
nns.add(layers.MaxPool2D((2,2)))
nns.add(layers.Dropout(0.2))

nns.add(layers.Conv2D(64, (3,3), activation='relu', padding='same'))
nns.add(layers.MaxPool2D((2,2)))
nns.add(layers.Dropout(0.2))

nns.add(layers.Conv2D(64, (3,3), activation='relu', padding='same'))
nns.add(layers.Dropout(0.2))

## Now flatten the output of the convolutional layers so
## a fully connected network can be applied.
nns.add(layers.Flatten())

## Finally, fully connected layers to classify the digits
## Using the features extracted by the convolutional layers.
nns.add(layers.Dense(64, activation = 'relu',
                    kernel_regularizer=regularizers.l2(0.1)))
nns.add(layers.Dropout(0.5))
nns.add(layers.Dense(10, activation = 'softmax'))

nns.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 32)	0
dropout_4 (Dropout)	(None, 14, 14, 32)	0
conv2d_4 (Conv2D)	(None, 14, 14, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 64)	0
dropout_5 (Dropout)	(None, 7, 7, 64)	0
conv2d_5 (Conv2D)	(None, 7, 7, 64)	36,928
dropout_6 (Dropout)	(None, 7, 7, 64)	0
flatten_1 (Flatten)	(None, 3136)	0
dense_2 (Dense)	(None, 64)	200,768
dropout_7 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 10)	650

Total params: 257,162 (1004.54 KB)

Trainable params: 257,162 (1004.54 KB)

Non-trainable params: 0 (0.00 B)

? Examine the summary of your model and answer these questions:

1. The final feature map is $64 \times 7 \times 7$, rather than $64 \times 3 \times 3$ for the model without padding. This feature map has 49 pixels per channel, compared to 9 pixels per channel previously. Do you expect that this feature map will contain more information and why?
2. How has the number of parameters changed for the convolutional layer, and is this expected?

3. Compare the number of learnable parameters for the 64 unit fully connected layer between the models with and without padding. How can you account for this difference? **End of exercise.**

✓ Answers:

1. I do not expect this feature map to contain more information, although it could. Whether it actually contains more information depends on whether all the previous layers are able to extract it. Given that some digits are just 1–2 pixels close to the boundary, zero-padding may affect edge feature extraction, but I don't think it will be significant. Another nuance I would like to point out is large feature map, like 7x7 VS 3x3 even if contains more information, may require more complex deeper level to extract and leverage that information, which may not always be worth it.
2. The number of trainable parameters in the convolutional layers remains unchanged, which makes sense. The count of input/output features in the convolutional layers has not changed, only their dimensionality. So the number of filters remained the same. The dimensionality of filters, which have all the trainable parameters remained the same too. That's why the convolutional layers parameter count remained unchanged.
3. The number of parameters in the fully connected layer increased by approximately 5x, which is loosely proportional to its input dimensionality raise from 576 to 3136.

Next execute the code in the cell below to compile and fit the model. This process will take some time.

```
In [23]: ## Compile the model
nns.compile(optimizer=RMSprop(learning_rate=0.001, weight_decay=0.001),
            loss = 'categorical_crossentropy', metrics = ['accuracy', 'top_k_categorical_accuracy'])

## Now fit the model
history_nns = nns.fit(train_images, train_labels,
                     epochs = TEST_EPOCHS or 80, batch_size = 256,
                     validation_data = (test_images, test_labels),
                     callbacks = callbacks_list, # Call backs argument here
                     verbose = 1)
```

Epoch 1/80

235/235 ————— **36s** 146ms/step – accuracy: 0.6300 – loss: 3.0451 – top_k_categorical_accuracy: 0.9048 – val_accuracy: 0.9428 – val_loss: 0.3694 – val_top_k_categorical_accuracy: 0.9988

Epoch 2/80

235/235 ————— **34s** 144ms/step – accuracy: 0.9124 – loss: 0.4716 – top_k_categorical_accuracy: 0.9962 – val_accuracy: 0.9701 – val_loss: 0.2442 – val_top_k_categorical_accuracy: 0.9997

Epoch 3/80

235/235 ————— **36s** 153ms/step – accuracy: 0.9332 – loss: 0.3716 – top_k_categorical_accuracy: 0.9980 – val_accuracy: 0.9736 – val_loss: 0.2153 – val_top_k_categorical_accuracy: 0.9999

Epoch 4/80

235/235 ————— **33s** 141ms/step – accuracy: 0.9425 – loss: 0.3282 – top_k_categorical_accuracy: 0.9978 – val_accuracy: 0.9688 – val_loss: 0.2245 – val_top_k_categorical_accuracy: 0.9995

Epoch 5/80

235/235 ————— **33s** 141ms/step – accuracy: 0.9504 – loss: 0.2957 – top_k_categorical_accuracy: 0.9985 – val_accuracy: 0.9780 – val_loss: 0.1947 – val_top_k_categorical_accuracy: 0.9997

Epoch 6/80

235/235 ————— **34s** 144ms/step – accuracy: 0.9545 – loss: 0.2734 – top_k_categorical_accuracy: 0.9988 – val_accuracy: 0.9790 – val_loss: 0.1841 – val_top_k_categorical_accuracy: 0.9999

Epoch 7/80

235/235 ————— **34s** 143ms/step – accuracy: 0.9588 – loss: 0.2550 – top_k_categorical_accuracy: 0.9988 – val_accuracy: 0.9837 – val_loss: 0.1566 – val_top_k_categorical_accuracy: 0.9999

Epoch 8/80

235/235 ————— **33s** 141ms/step – accuracy: 0.9606 – loss: 0.2476 – top_k_categorical_accuracy: 0.9984 – val_accuracy: 0.9787 – val_loss: 0.1676 – val_top_k_categorical_accuracy: 0.9999

Epoch 9/80

235/235 ————— **37s** 156ms/step – accuracy: 0.9633 – loss: 0.2353 – top_k_categorical_accuracy: 0.9989 – val_accuracy: 0.9832 – val_loss: 0.1566 – val_top_k_categorical_accuracy: 0.9998

Epoch 10/80

235/235 ————— **34s** 146ms/step – accuracy: 0.9663 – loss: 0.2220 – top_k_categorical_accuracy: 0.9988 – val_accuracy: 0.9849 – val_loss: 0.1422 – val_top_k_categorical_accuracy: 0.9998

Epoch 11/80

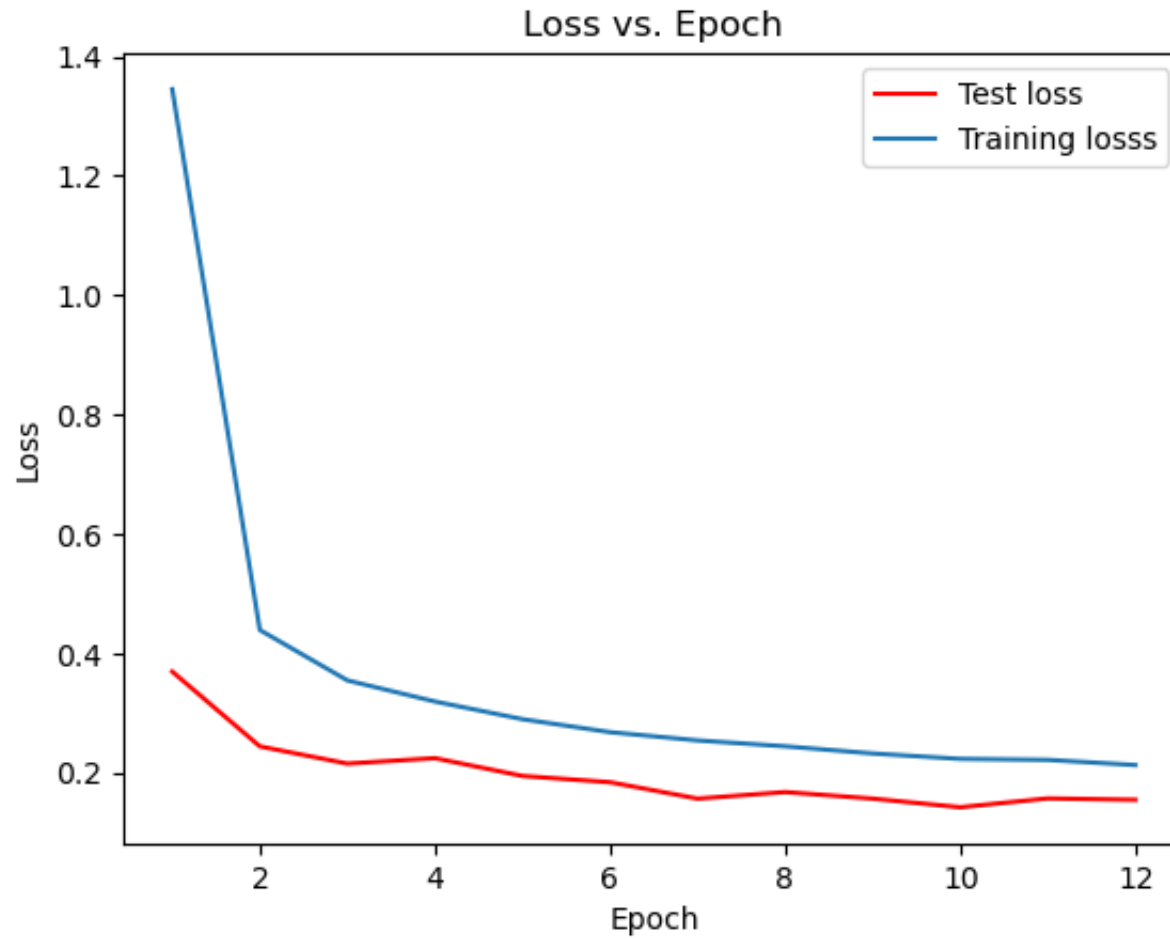
235/235 ————— **40s** 169ms/step – accuracy: 0.9647 – loss: 0.2211 – top_k_categorical_accuracy: 0.9990 – val_accuracy: 0.9808 – val_loss: 0.1570 – val_top_k_categorical_accuracy: 0.9998

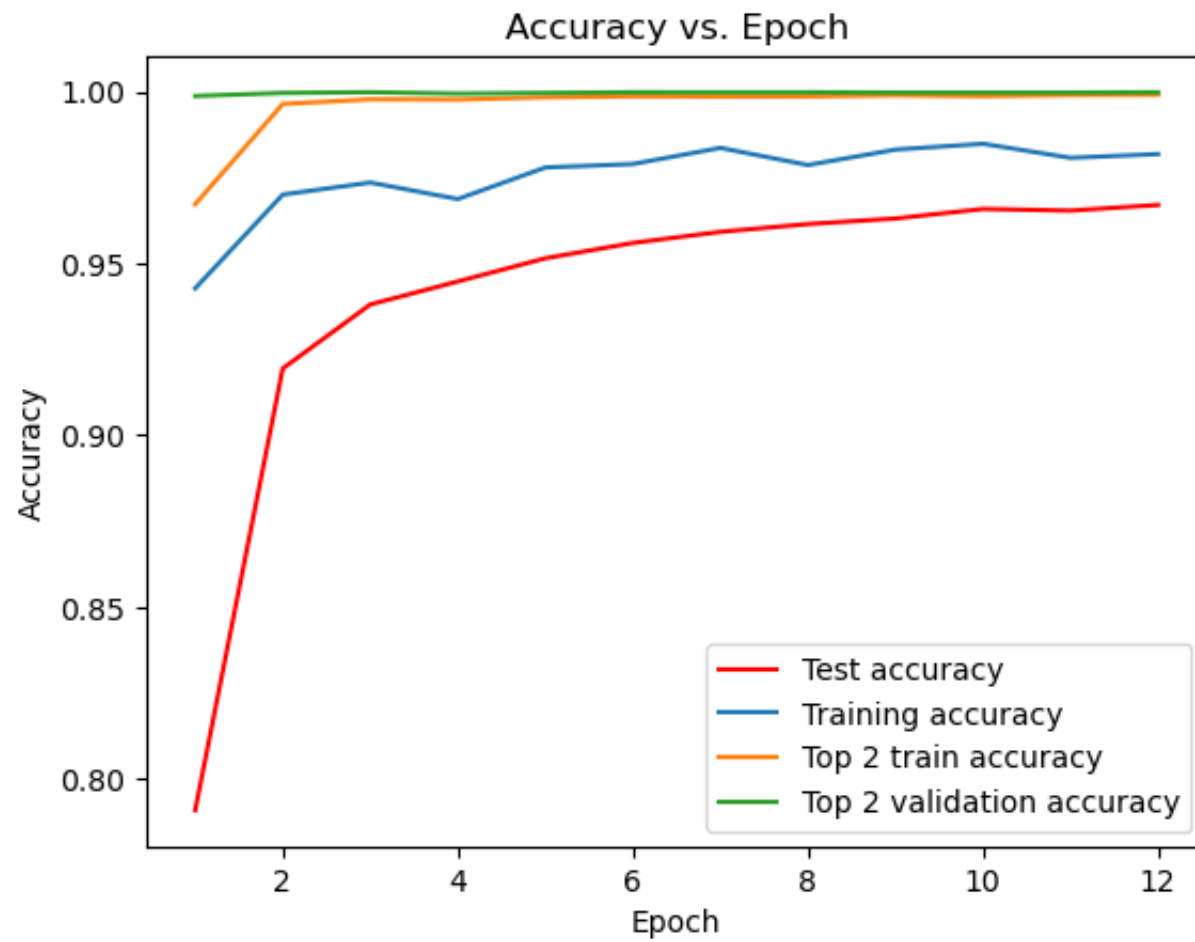
Epoch 12/80

235/235 ————— **34s** 145ms/step – accuracy: 0.9674 – loss: 0.2126 – top_k_categorical_accuracy: 0.9992 – val_accuracy: 0.9819 – val_loss: 0.1547 – val_top_k_categorical_accuracy: 0.9999

Next, execute the code in the cells below to plot the loss vs. epoch and accuracy vs. epoch.

```
In [24]: plot_loss(history_nns)
         plot_accuracy(history_nns)
```





```
In [25]: ## Display accuracy netrics  
evaluate_model(nns, test_images, test_labels)
```

313/313  **3s** 8ms/step

Categorical accuracy = 0.9819

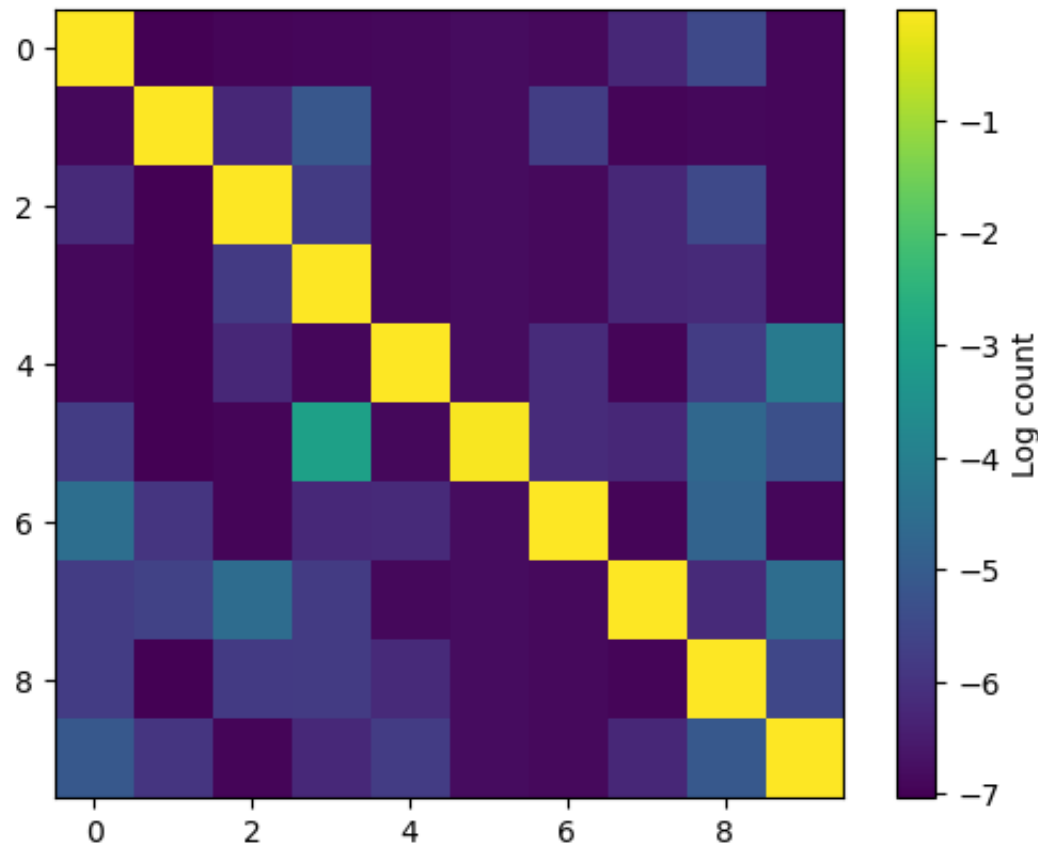
Top 2 accuracy = 0.9965

Average precision = 0.9824

Average recall = 0.9819

Confusion matrix

	0	1	2	3	4	5	6	7	8	9
0	976	0	0	0	0	0	0	1	3	0
1	0	1127	1	5	0	0	2	0	0	0
2	1	0	1025	2	0	0	0	1	3	0
3	0	0	2	1006	0	0	0	1	1	0
4	0	0	1	0	963	0	1	0	2	15
5	2	0	0	48	0	828	1	1	8	4
6	10	2	0	1	1	0	937	0	7	0
7	2	3	10	2	0	0	0	1000	1	10
8	2	0	2	2	1	0	0	0	964	3
9	5	2	0	1	2	0	0	1	5	993



▶ ? **Exercise 6-7:** Keeping in mind that small (3rd digit) differences in metrics are often simply a result of random sampling do these results appear significantly difference than for the unpadded model? It may well be the case that richer feature map may not be important for this relatively simple problem.

✓ **Answer:** I've reinitialized the model three times, and each time the validation accuracy was approximately 0.2% worse. There is still a high chance that this is coincidental, but my current hypothesis is that the new model's performance has at best, remained unchanged.

A More Efficient CNN

As you have likely noticed, training the foregoing CNN models is a rather computationally intensive process. This was even the case for the small networks used for these examples. In large part, the slow training is a result of the large number of parameters that must be learned for 4-D and higher dimensional convolutions. Consider that for a $32 \times 32 \times 3$ to $32 \times 32 \times 3$ 4-D convolution operator the network must learn $3 \times 3 \times 32 \times 32 = 9,216$ parameters.

There is a more efficient formulation of the CNN layer, the **separable convolution operator**. The separable convolution operator consists of two components, a **depth-wise 2-D convolution operator** and a **pointwise convolution operator**. The depthwise convolution learns a set of 2-D weights independently for each channel. The pointwise convolution learns the weights of a 1-D convolution operator that 'mixes between the channels'. Separating the convolutional operator like this greatly reduces the number of parameters that must be learned. For a 3×3 32×32 to 32×32 channel convolution the separable operator has $3 \times 3 + 32 + 32 = 73$ parameters, a large change compared to the 4-D operator.

► **Exercise 6-8:** To see the result of applying separable convolution to the digit classification problem you will do the following. You will use a different method to specify a sequential Keras model than you have used previously. The layers are specified in evaluation order within the list which is the argument to the `keras.models.sequential` function. To do this you can copy the layers already used for Exercise 6-6. You only need to the code that defines the layers, not use the `add` method on the model. Make sure you put comments between the layer specifications in the list. To help you get started the specification of the first two layers is provided. When you have completed the layer list, execute the code in the cell below.

```
In [26]: l2 = regularizers.l2(0.1)

nnsep = models.Sequential(
    [
        layers.Input(shape=(28, 28, 1, )),
        layers.SeparableConv2D(32, (3, 3), padding = 'same', activation = 'relu'),

        ## Put the layer list here
```

```

layers.MaxPool2D((2,2)),
layers.Dropout(0.2),

layers.SeparableConv2D(64, (3, 3), padding = 'same', activation = 'relu'),
layers.MaxPool2D((2,2)),
layers.Dropout(0.2),

layers.SeparableConv2D(64, (3, 3), padding = 'same', activation = 'relu'),
layers.Dropout(0.2),

# Now flatten the output of the convolutional layers so
# a fully connected network can be applied.
layers.Flatten(),

# Finally, fully connected layers to classify the digits
# Using the features extracted by the convolutional layers.
layers.Dense(64, activation = 'relu', kernel_regularizer=l2),

layers.Dropout(0.5),
layers.Dense(10, activation = 'softmax')
]
)

nnsep.summary()

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
separable_conv2d (SeparableConv2D)	(None, 28, 28, 32)	73
max_pooling2d_4 (MaxPooling2D)	(None, 14, 14, 32)	0
dropout_8 (Dropout)	(None, 14, 14, 32)	0
separable_conv2d_1 (SeparableConv2D)	(None, 14, 14, 64)	2,400
max_pooling2d_5 (MaxPooling2D)	(None, 7, 7, 64)	0
dropout_9 (Dropout)	(None, 7, 7, 64)	0
separable_conv2d_2 (SeparableConv2D)	(None, 7, 7, 64)	4,736
dropout_10 (Dropout)	(None, 7, 7, 64)	0
flatten_2 (Flatten)	(None, 3136)	0
dense_4 (Dense)	(None, 64)	200,768
dropout_11 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 10)	650

Total params: 208,627 (814.95 KB)

Trainable params: 208,627 (814.95 KB)

Non-trainable params: 0 (0.00 B)

```
In [28]: ## Compile the model
nnsep.compile(optimizer=RMSprop(learning_rate=0.001, weight_decay=0.001),
              loss = 'categorical_crossentropy', metrics = ['accuracy', 'top_k_categorical_accuracy'])

## Now fit the model
history_nnsep = nnsep.fit(train_images, train_labels,
```

```

epochs = TEST_EPOCHS or 80, batch_size = 256,
validation_data = (test_images, test_labels),
callbacks = callbacks_list, # Call backs argument here
verbose = 1)

## Display learning history
plot_loss(history_nnsep)
plot_accuracy(history_nnsep)

## Display accuracy netrics
evaluate_model(nnsep, test_images, test_labels)

```

Epoch 1/80

235/235  **37s** 148ms/step - accuracy: 0.2108 - loss: 3.8236 - top_k_categorical_accuracy: 0.6240 - val_accuracy: 0.8065 - val_loss: 0.8569 - val_top_k_categorical_accuracy: 0.9888

Epoch 2/80

235/235  **31s** 132ms/step - accuracy: 0.6747 - loss: 1.1381 - top_k_categorical_accuracy: 0.9689 - val_accuracy: 0.8726 - val_loss: 0.5920 - val_top_k_categorical_accuracy: 0.9928

Epoch 3/80

235/235  **31s** 132ms/step - accuracy: 0.7565 - loss: 0.8791 - top_k_categorical_accuracy: 0.9821 - val_accuracy: 0.9041 - val_loss: 0.4505 - val_top_k_categorical_accuracy: 0.9971

Epoch 4/80

235/235  **31s** 132ms/step - accuracy: 0.7881 - loss: 0.7728 - top_k_categorical_accuracy: 0.9859 - val_accuracy: 0.9189 - val_loss: 0.4092 - val_top_k_categorical_accuracy: 0.9958

Epoch 5/80

235/235  **33s** 141ms/step - accuracy: 0.8143 - loss: 0.7034 - top_k_categorical_accuracy: 0.9879 - val_accuracy: 0.9260 - val_loss: 0.3667 - val_top_k_categorical_accuracy: 0.9977

Epoch 6/80

235/235  **32s** 135ms/step - accuracy: 0.8338 - loss: 0.6457 - top_k_categorical_accuracy: 0.9903 - val_accuracy: 0.9366 - val_loss: 0.3489 - val_top_k_categorical_accuracy: 0.9978

Epoch 7/80

235/235  **31s** 132ms/step - accuracy: 0.8442 - loss: 0.6130 - top_k_categorical_accuracy: 0.9895 - val_accuracy: 0.9366 - val_loss: 0.3420 - val_top_k_categorical_accuracy: 0.9979

Epoch 8/80

235/235  **29s** 124ms/step - accuracy: 0.8529 - loss: 0.5853 - top_k_categorical_accuracy: 0.9916 - val_accuracy: 0.9437 - val_loss: 0.3082 - val_top_k_categorical_accuracy: 0.9980

Epoch 9/80

235/235  **31s** 130ms/step - accuracy: 0.8636 - loss: 0.5608 - top_k_categorical_accuracy: 0.9919 - val_accuracy: 0.9545 - val_loss: 0.2734 - val_top_k_categorical_accuracy: 0.9984

Epoch 10/80

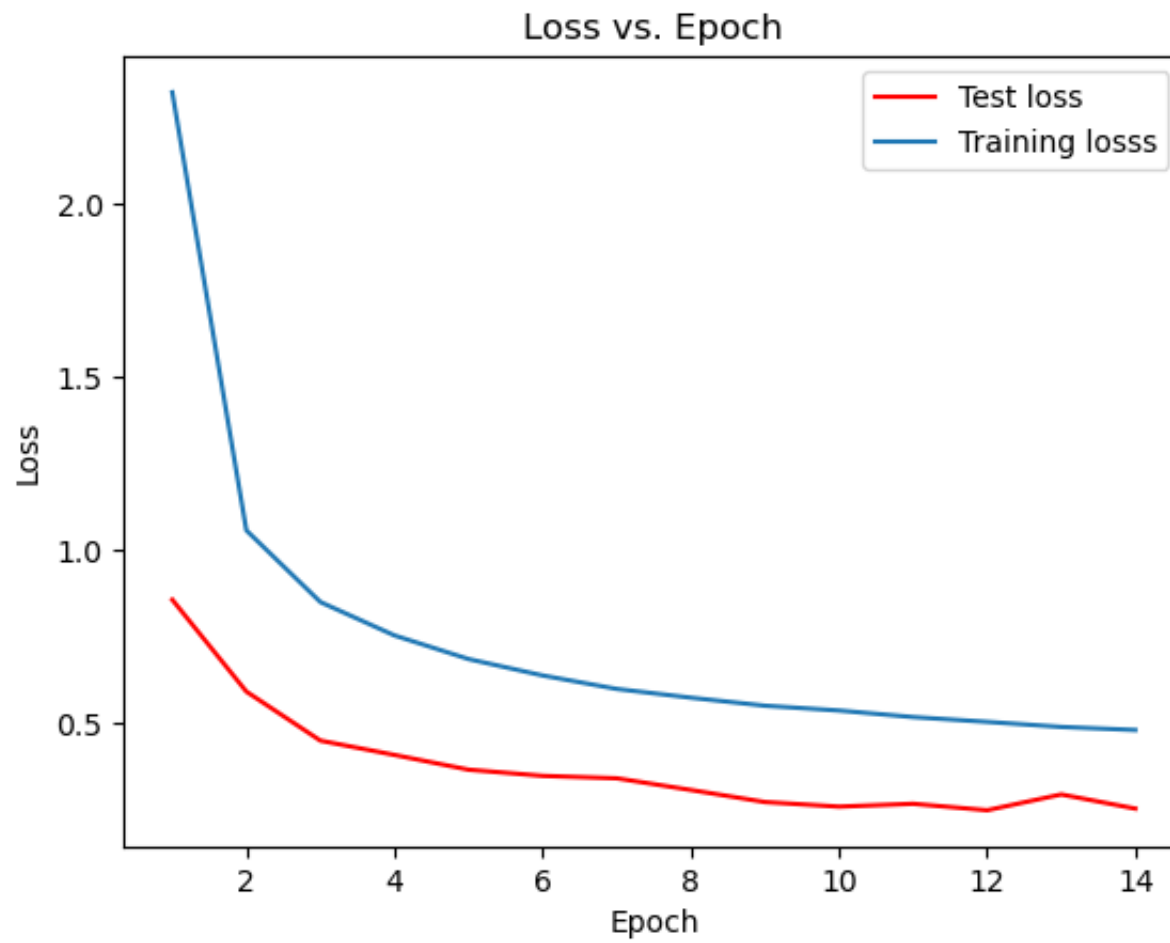
235/235  **34s** 144ms/step - accuracy: 0.8654 - loss: 0.5453 - top_k_categorical_accuracy: 0.9928 - val_accuracy: 0.9588 - val_loss: 0.2603 - val_top_k_categorical_accuracy: 0.9991
Epoch 11/80

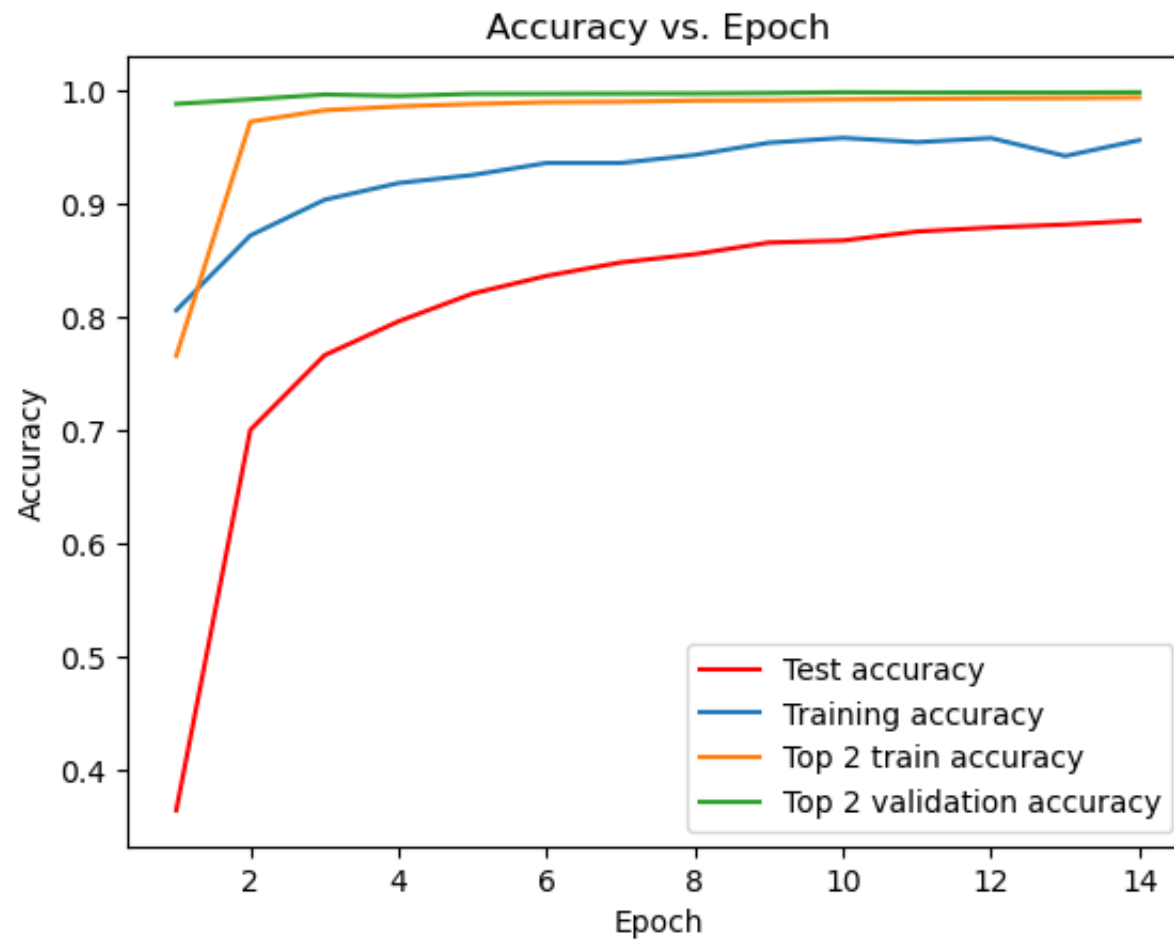
235/235  **30s** 128ms/step - accuracy: 0.8759 - loss: 0.5172 - top_k_categorical_accuracy: 0.9936 - val_accuracy: 0.9550 - val_loss: 0.2685 - val_top_k_categorical_accuracy: 0.9988
Epoch 12/80

235/235  **31s** 130ms/step - accuracy: 0.8781 - loss: 0.5061 - top_k_categorical_accuracy: 0.9939 - val_accuracy: 0.9586 - val_loss: 0.2498 - val_top_k_categorical_accuracy: 0.9989
Epoch 13/80

235/235  **31s** 133ms/step - accuracy: 0.8820 - loss: 0.4902 - top_k_categorical_accuracy: 0.9936 - val_accuracy: 0.9428 - val_loss: 0.2954 - val_top_k_categorical_accuracy: 0.9988
Epoch 14/80

235/235  **32s** 138ms/step - accuracy: 0.8865 - loss: 0.4799 - top_k_categorical_accuracy: 0.9946 - val_accuracy: 0.9568 - val_loss: 0.2545 - val_top_k_categorical_accuracy: 0.9991





313/313  **3s** 10ms/step

Categorical accuracy = 0.9568

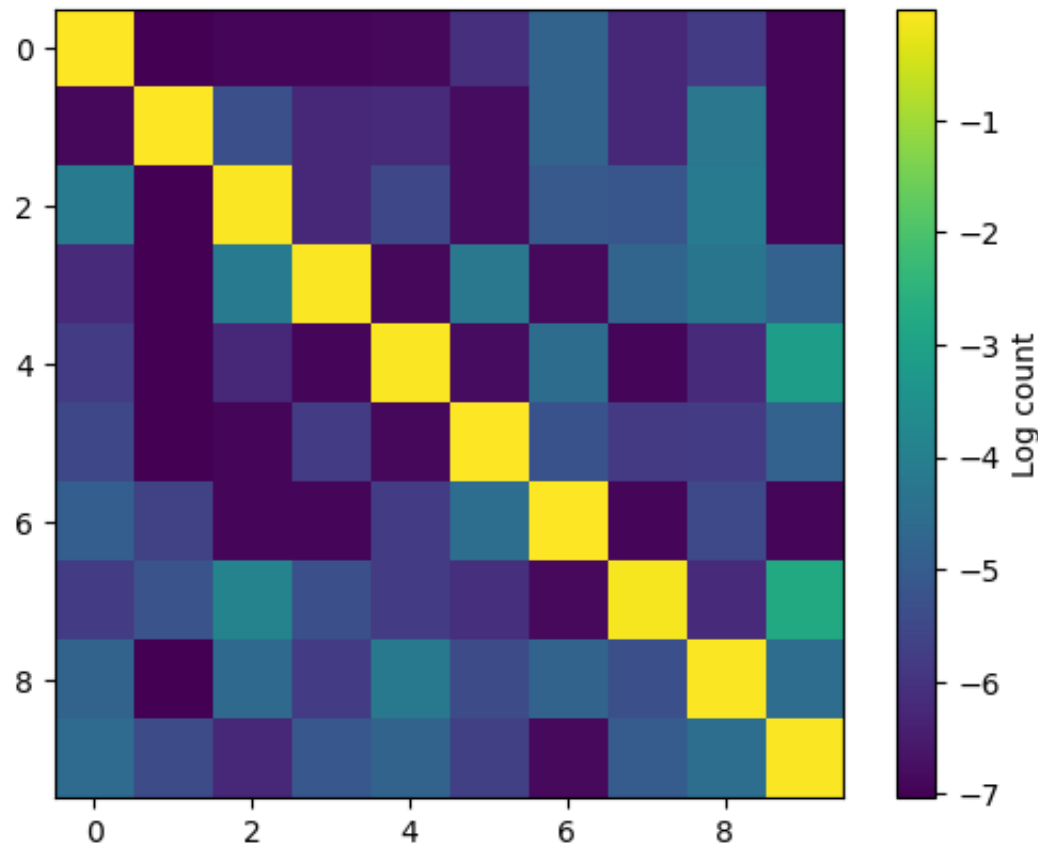
Top 2 accuracy = 0.9876

Average precision = 0.9577

Average recall = 0.9568

Confusion matrix

	0	1	2	3	4	5	6	7	8	9
0	969	0	0	0	0	1	7	1	2	0
1	0	1108	4	1	1	0	7	1	13	0
2	14	0	989	1	3	0	5	5	15	0
3	1	0	15	955	0	12	0	8	12	7
4	2	0	1	0	926	0	9	0	1	43
5	3	0	0	2	0	872	4	2	2	7
6	6	3	0	0	2	9	935	0	3	0
7	2	5	20	4	2	1	0	931	1	62
8	7	0	9	2	14	3	7	4	918	10
9	9	4	1	5	7	2	0	6	10	965



? Examine these results and answer the following questions.

1. How do the performance metrics of the model using the separable convolution compare the the model you constructed and evaluated in Exercises 6-6 and 6-7?
2. Compare the number of model parameters of the separable convolution layers to the full convolution layers in the model you created in Exercise 6-6. What does this difference tell you about separable convolution vs. full convolution?
3. Compare the time required per step to train the model with separable convolution to the time per step for the fully convolution model. Given the reduced number of parameters, is the difference in time required consistent?

 Answers:

1. The performance metrics of the model using separable convolution are worse compared to the original model. Accuracy dropped by approximately 4%.
2. Separable convolution layers should train about ~8x faster. This is because separable convolution layers have a total parameter count of 7,209, compared to 55,744 for standard convolutions, representing an 87% reduction.
3. The reduction in per-step training time is considerable but not directly proportional to the reduction in parameters. The ratio of parameter reduction to speed improvement is roughly 4:1. Specifically, separable convolution reduced training time to 30.8 seconds/epoch compared to 32.5 seconds/epoch for 2D convolution. This is a 5% improvement, while the total model parameter count decreased by approximately 19% (208,267 vs. 257,162). Overall the speed improvement from separable convolution is noticeable but not major. The guilty party is the Dense layer containing ~80% of the model trainable parameters, which remained unchanged.

Variable Learning Rate

The two foregoing models have been trained using a fixed learning rate. In many cases a fixed learning rate is far from optimal. The problem arises since initial learning can take large steps, but small steps are required toward convergence to ensure a stable termination of the algorithm. A fixed low learning rate will learn too slowly at the beginning of training but converge at the end. On the other hand a high learning rate is effective at the beginning of training, but will lead to poor or unstable convergence.

The solution is to use a variable learning rate. The variable learning rate decreases as the training proceeds. In this way, the learning rate can be optimized to ensure the fastest possible convergence of the learning. The variable learning rate is determined by a schedule. Common schedules include linear decreases and exponential decreases. Keras supports a number of [learning rate schedulers](#).

 **Exercise 6-9:** You will now investigate using a variable learning rate for training the CNN model.

The code in the cell bellow does the following:

1. An exponentially decay learning rate schedule is created using `tensorflow.keras.optimizers.schedules.ExponentialDecay`.
2. The `nnv` model is instantiated, using the function you created earlier, and compiled. A learning rate schedule can be incorporated into most Keras optimizers. In this case, the `RMSprop optimizer` has an `learning_rate` argument set to the learning rate just created, and with `decay=0.001`.
3. The model is fit using 80 epochs, or untill the stopping criteria set in the call-back function is reached. The larger number of epochs allow the training to run closer to completion.

Execute this code to traing the model, which will take some time to run.

Note: You may be wondering why a new model needs to be instantiated to run this experiement. The reason is that Keras (nd Tensorflow) model objects will retain any weight values from previous learning. This property can be an advantage if one wishes to do additional training. For example, additional training can be performed when additional data is avaiable, without starting over again. Hoeever, if one wishes to run an independent experiment, as we do here, it is best to instantiate a new model object.

```
In [29]: ## Define the learning rate schedule
initial_learning_rate = 0.01
lr_schedule = ExponentialDecay(
    initial_learning_rate,
    decay_steps=100,
    decay_rate=0.96,
    staircase=True)

## Instantiate and compile the model
nnsep = create_model() #####
nnsep.compile(optimizer = RMSprop(learning_rate=lr_schedule, weight_decay=0.001),
              loss = 'categorical_crossentropy',
              metrics = ['accuracy', 'top_k_categorical_accuracy'])

## Now fit the model
```

```
history_vnnsep = nnsep.fit(train_images, train_labels,
                           epochs = TEST_EPOCHS or 80, batch_size = 256,
                           validation_data = (test_images, test_labels),
                           callbacks = callbacks_list, # Call backs argument here
                           verbose = 1)
```

Epoch 1/80

235/235 ————— **32s** 129ms/step - accuracy: 0.6914 - loss: 3.6960 - top_k_categorical_accuracy: 0.9514 - val_accuracy: 0.8715 - val_loss: 1.3299 - val_top_k_categorical_accuracy: 0.9931

Epoch 2/80

235/235 ————— **30s** 129ms/step - accuracy: 0.8082 - loss: 1.3991 - top_k_categorical_accuracy: 0.9851 - val_accuracy: 0.8927 - val_loss: 1.1482 - val_top_k_categorical_accuracy: 0.9958

Epoch 3/80

235/235 ————— **30s** 129ms/step - accuracy: 0.8450 - loss: 1.1541 - top_k_categorical_accuracy: 0.9899 - val_accuracy: 0.9129 - val_loss: 0.8262 - val_top_k_categorical_accuracy: 0.9973

Epoch 4/80

235/235 ————— **30s** 127ms/step - accuracy: 0.8606 - loss: 1.0151 - top_k_categorical_accuracy: 0.9913 - val_accuracy: 0.9260 - val_loss: 0.7449 - val_top_k_categorical_accuracy: 0.9979

Epoch 5/80

235/235 ————— **32s** 137ms/step - accuracy: 0.8736 - loss: 0.9012 - top_k_categorical_accuracy: 0.9922 - val_accuracy: 0.9412 - val_loss: 0.6339 - val_top_k_categorical_accuracy: 0.9982

Epoch 6/80

235/235 ————— **32s** 137ms/step - accuracy: 0.8841 - loss: 0.8075 - top_k_categorical_accuracy: 0.9937 - val_accuracy: 0.9564 - val_loss: 0.5054 - val_top_k_categorical_accuracy: 0.9989

Epoch 7/80

235/235 ————— **31s** 133ms/step - accuracy: 0.8933 - loss: 0.7256 - top_k_categorical_accuracy: 0.9941 - val_accuracy: 0.9570 - val_loss: 0.4689 - val_top_k_categorical_accuracy: 0.9991

Epoch 8/80

235/235 ————— **30s** 130ms/step - accuracy: 0.9004 - loss: 0.6561 - top_k_categorical_accuracy: 0.9950 - val_accuracy: 0.9562 - val_loss: 0.4760 - val_top_k_categorical_accuracy: 0.9991

Epoch 9/80

235/235 ————— **33s** 138ms/step - accuracy: 0.9072 - loss: 0.6031 - top_k_categorical_accuracy: 0.9959 - val_accuracy: 0.9636 - val_loss: 0.3569 - val_top_k_categorical_accuracy: 0.9994

Epoch 10/80

235/235 ————— **34s** 144ms/step - accuracy: 0.9120 - loss: 0.5514 - top_k_categorical_accuracy: 0.9954 - val_accuracy: 0.9668 - val_loss: 0.3407 - val_top_k_categorical_accuracy: 0.9995

Epoch 11/80

235/235 ————— **31s** 130ms/step - accuracy: 0.9172 - loss: 0.5065 - top_k_categorical_accuracy: 0.9962 - val_accuracy: 0.9646 - val_loss: 0.3096 - val_top_k_categorical_accuracy: 0.9996

Epoch 12/80

235/235 ————— **32s** 137ms/step – accuracy: 0.9200 – loss: 0.4733 – top_k_categorical_accuracy: 0.9960 – val_accuracy: 0.9706 – val_loss: 0.2747 – val_top_k_categorical_accuracy: 0.9997
Epoch 13/80

235/235 ————— **33s** 139ms/step – accuracy: 0.9221 – loss: 0.4403 – top_k_categorical_accuracy: 0.9969 – val_accuracy: 0.9671 – val_loss: 0.2677 – val_top_k_categorical_accuracy: 0.9992
Epoch 14/80

235/235 ————— **32s** 135ms/step – accuracy: 0.9277 – loss: 0.4146 – top_k_categorical_accuracy: 0.9966 – val_accuracy: 0.9709 – val_loss: 0.2585 – val_top_k_categorical_accuracy: 0.9996
Epoch 15/80

235/235 ————— **32s** 136ms/step – accuracy: 0.9294 – loss: 0.3989 – top_k_categorical_accuracy: 0.9974 – val_accuracy: 0.9710 – val_loss: 0.2431 – val_top_k_categorical_accuracy: 0.9996
Epoch 16/80

235/235 ————— **31s** 131ms/step – accuracy: 0.9315 – loss: 0.3815 – top_k_categorical_accuracy: 0.9973 – val_accuracy: 0.9670 – val_loss: 0.2510 – val_top_k_categorical_accuracy: 0.9993
Epoch 17/80

235/235 ————— **31s** 133ms/step – accuracy: 0.9326 – loss: 0.3637 – top_k_categorical_accuracy: 0.9974 – val_accuracy: 0.9697 – val_loss: 0.2143 – val_top_k_categorical_accuracy: 0.9998
Epoch 18/80

235/235 ————— **28s** 119ms/step – accuracy: 0.9339 – loss: 0.3441 – top_k_categorical_accuracy: 0.9975 – val_accuracy: 0.9725 – val_loss: 0.2102 – val_top_k_categorical_accuracy: 0.9999
Epoch 19/80

235/235 ————— **28s** 118ms/step – accuracy: 0.9366 – loss: 0.3364 – top_k_categorical_accuracy: 0.9976 – val_accuracy: 0.9749 – val_loss: 0.2010 – val_top_k_categorical_accuracy: 0.9995
Epoch 20/80

235/235 ————— **29s** 124ms/step – accuracy: 0.9401 – loss: 0.3173 – top_k_categorical_accuracy: 0.9975 – val_accuracy: 0.9745 – val_loss: 0.1849 – val_top_k_categorical_accuracy: 0.9999
Epoch 21/80

235/235 ————— **28s** 120ms/step – accuracy: 0.9416 – loss: 0.2994 – top_k_categorical_accuracy: 0.9981 – val_accuracy: 0.9763 – val_loss: 0.1735 – val_top_k_categorical_accuracy: 1.0000
Epoch 22/80

235/235 ————— **29s** 122ms/step – accuracy: 0.9445 – loss: 0.2881 – top_k_categorical_accuracy: 0.9984 – val_accuracy: 0.9787 – val_loss: 0.1636 – val_top_k_categorical_accuracy: 0.9999
Epoch 23/80

235/235 ————— **31s** 130ms/step – accuracy: 0.9454 – loss: 0.2780 – top_k_categorical_accuracy: 0.9980 – val_accuracy: 0.9791 – val_loss: 0.1515 – val_top_k_categorical_accuracy: 0.9996
Epoch 24/80

235/235 ————— **31s** 130ms/step – accuracy: 0.9464 – loss: 0.2671 – top_k_categorical_accuracy: 0.9983 – val_accuracy: 0.9794 – val_loss: 0.1475 – val_top_k_categorical_accuracy: 0.9998
Epoch 25/80

235/235 ————— **33s** 138ms/step – accuracy: 0.9488 – loss: 0.2564 – top_k_categorical_accuracy: 0.9982 – val_accuracy: 0.9810 – val_loss: 0.1372 – val_top_k_categorical_accuracy: 0.9997
Epoch 26/80

235/235 ————— **31s** 132ms/step – accuracy: 0.9505 – loss: 0.2437 – top_k_categorical_accuracy: 0.9984 – val_accuracy: 0.9787 – val_loss: 0.1412 – val_top_k_categorical_accuracy: 0.9997
Epoch 27/80

235/235 ————— **31s** 130ms/step – accuracy: 0.9493 – loss: 0.2395 – top_k_categorical_accuracy: 0.9984 – val_accuracy: 0.9827 – val_loss: 0.1283 – val_top_k_categorical_accuracy: 1.0000
Epoch 28/80

235/235 ————— **31s** 133ms/step – accuracy: 0.9508 – loss: 0.2340 – top_k_categorical_accuracy: 0.9984 – val_accuracy: 0.9821 – val_loss: 0.1217 – val_top_k_categorical_accuracy: 0.9999
Epoch 29/80

235/235 ————— **33s** 140ms/step – accuracy: 0.9512 – loss: 0.2260 – top_k_categorical_accuracy: 0.9987 – val_accuracy: 0.9811 – val_loss: 0.1213 – val_top_k_categorical_accuracy: 0.9999
Epoch 30/80

235/235 ————— **31s** 132ms/step – accuracy: 0.9525 – loss: 0.2218 – top_k_categorical_accuracy: 0.9988 – val_accuracy: 0.9814 – val_loss: 0.1153 – val_top_k_categorical_accuracy: 0.9999
Epoch 31/80

235/235 ————— **34s** 144ms/step – accuracy: 0.9553 – loss: 0.2152 – top_k_categorical_accuracy: 0.9987 – val_accuracy: 0.9804 – val_loss: 0.1130 – val_top_k_categorical_accuracy: 0.9999
Epoch 32/80

235/235 ————— **34s** 144ms/step – accuracy: 0.9547 – loss: 0.2126 – top_k_categorical_accuracy: 0.9985 – val_accuracy: 0.9830 – val_loss: 0.1076 – val_top_k_categorical_accuracy: 0.9999
Epoch 33/80

235/235 ————— **32s** 134ms/step – accuracy: 0.9554 – loss: 0.2046 – top_k_categorical_accuracy: 0.9989 – val_accuracy: 0.9807 – val_loss: 0.1047 – val_top_k_categorical_accuracy: 1.0000
Epoch 34/80

235/235 ————— **31s** 133ms/step – accuracy: 0.9566 – loss: 0.1943 – top_k_categorical_accuracy: 0.9986 – val_accuracy: 0.9829 – val_loss: 0.1004 – val_top_k_categorical_accuracy: 0.9997
Epoch 35/80

235/235 ————— **32s** 134ms/step – accuracy: 0.9579 – loss: 0.1883 – top_k_categorical_accuracy: 0.9988 – val_accuracy: 0.9823 – val_loss: 0.0954 – val_top_k_categorical_accuracy: 0.9999
Epoch 36/80

235/235 ————— **31s** 132ms/step – accuracy: 0.9582 – loss: 0.1849 – top_k_categorical_accuracy: 0.9988 – val_accuracy: 0.9825 – val_loss: 0.0956 – val_top_k_categorical_accuracy: 1.0000
Epoch 37/80

235/235 ————— **32s** 135ms/step – accuracy: 0.9588 – loss: 0.1786 – top_k_categorical_accuracy: 0.9989 – val_accuracy: 0.9843 – val_loss: 0.0910 – val_top_k_categorical_accuracy: 0.9997
Epoch 38/80

235/235 ————— **30s** 127ms/step – accuracy: 0.9587 – loss: 0.1801 – top_k_categorical_accuracy: 0.9988 – val_accuracy: 0.9828 – val_loss: 0.0903 – val_top_k_categorical_accuracy: 0.9998
Epoch 39/80

235/235 ————— **30s** 128ms/step – accuracy: 0.9612 – loss: 0.1720 – top_k_categorical_accuracy: 0.9989 – val_accuracy: 0.9850 – val_loss: 0.0844 – val_top_k_categorical_accuracy: 1.0000
Epoch 40/80

235/235 ————— **32s** 137ms/step – accuracy: 0.9612 – loss: 0.1724 – top_k_categorical_accuracy: 0.9991 – val_accuracy: 0.9829 – val_loss: 0.0843 – val_top_k_categorical_accuracy: 0.9998
Epoch 41/80

235/235 ————— **34s** 144ms/step – accuracy: 0.9595 – loss: 0.1712 – top_k_categorical_accuracy: 0.9990 – val_accuracy: 0.9843 – val_loss: 0.0824 – val_top_k_categorical_accuracy: 0.9999
Epoch 42/80

235/235 ————— **33s** 139ms/step – accuracy: 0.9601 – loss: 0.1704 – top_k_categorical_accuracy: 0.9988 – val_accuracy: 0.9837 – val_loss: 0.0827 – val_top_k_categorical_accuracy: 0.9999
Epoch 43/80

235/235 ————— **31s** 133ms/step – accuracy: 0.9608 – loss: 0.1663 – top_k_categorical_accuracy: 0.9991 – val_accuracy: 0.9833 – val_loss: 0.0806 – val_top_k_categorical_accuracy: 0.9999
Epoch 44/80

235/235 ————— **33s** 139ms/step – accuracy: 0.9623 – loss: 0.1609 – top_k_categorical_accuracy: 0.9991 – val_accuracy: 0.9841 – val_loss: 0.0784 – val_top_k_categorical_accuracy: 1.0000
Epoch 45/80

235/235 ————— **33s** 140ms/step – accuracy: 0.9620 – loss: 0.1583 – top_k_categorical_accuracy: 0.9992 – val_accuracy: 0.9847 – val_loss: 0.0784 – val_top_k_categorical_accuracy: 0.9997
Epoch 46/80

235/235 ————— **33s** 142ms/step – accuracy: 0.9640 – loss: 0.1549 – top_k_categorical_accuracy: 0.9990 – val_accuracy: 0.9847 – val_loss: 0.0744 – val_top_k_categorical_accuracy: 0.9999
Epoch 47/80

235/235 ————— **32s** 137ms/step – accuracy: 0.9616 – loss: 0.1620 – top_k_categorical_accuracy: 0.9988 – val_accuracy: 0.9845 – val_loss: 0.0737 – val_top_k_categorical_accuracy: 0.9997
Epoch 48/80

235/235 ————— **32s** 135ms/step – accuracy: 0.9648 – loss: 0.1492 – top_k_categorical_accuracy: 0.9991 – val_accuracy: 0.9848 – val_loss: 0.0728 – val_top_k_categorical_accuracy: 0.9999
Epoch 49/80

235/235 ————— **31s** 130ms/step – accuracy: 0.9665 – loss: 0.1467 – top_k_categorical_accuracy: 0.9991 – val_accuracy: 0.9840 – val_loss: 0.0724 – val_top_k_categorical_accuracy: 0.9999
Epoch 50/80

235/235 ————— **30s** 126ms/step – accuracy: 0.9636 – loss: 0.1496 – top_k_categorical_accuracy: 0.9991 – val_accuracy: 0.9850 – val_loss: 0.0711 – val_top_k_categorical_accuracy: 1.0000
Epoch 51/80

235/235 ————— **31s** 130ms/step – accuracy: 0.9636 – loss: 0.1477 – top_k_categorical_accuracy: 0.9989 – val_accuracy: 0.9846 – val_loss: 0.0711 – val_top_k_categorical_accuracy: 1.0000
 Epoch 52/80

235/235 ————— **29s** 125ms/step – accuracy: 0.9628 – loss: 0.1499 – top_k_categorical_accuracy: 0.9993 – val_accuracy: 0.9851 – val_loss: 0.0692 – val_top_k_categorical_accuracy: 0.9999
 Epoch 53/80

235/235 ————— **29s** 124ms/step – accuracy: 0.9655 – loss: 0.1439 – top_k_categorical_accuracy: 0.9992 – val_accuracy: 0.9848 – val_loss: 0.0687 – val_top_k_categorical_accuracy: 1.0000
 Epoch 54/80

235/235 ————— **29s** 121ms/step – accuracy: 0.9647 – loss: 0.1464 – top_k_categorical_accuracy: 0.9989 – val_accuracy: 0.9846 – val_loss: 0.0686 – val_top_k_categorical_accuracy: 1.0000
 Epoch 55/80

235/235 ————— **28s** 120ms/step – accuracy: 0.9659 – loss: 0.1449 – top_k_categorical_accuracy: 0.9990 – val_accuracy: 0.9853 – val_loss: 0.0672 – val_top_k_categorical_accuracy: 1.0000
 Epoch 56/80

235/235 ————— **30s** 126ms/step – accuracy: 0.9651 – loss: 0.1410 – top_k_categorical_accuracy: 0.9994 – val_accuracy: 0.9850 – val_loss: 0.0677 – val_top_k_categorical_accuracy: 1.0000
 Epoch 57/80

235/235 ————— **30s** 127ms/step – accuracy: 0.9667 – loss: 0.1341 – top_k_categorical_accuracy: 0.9990 – val_accuracy: 0.9855 – val_loss: 0.0661 – val_top_k_categorical_accuracy: 0.9998
 Epoch 58/80

235/235 ————— **29s** 124ms/step – accuracy: 0.9645 – loss: 0.1415 – top_k_categorical_accuracy: 0.9991 – val_accuracy: 0.9858 – val_loss: 0.0655 – val_top_k_categorical_accuracy: 0.9998
 Epoch 59/80

235/235 ————— **32s** 134ms/step – accuracy: 0.9656 – loss: 0.1403 – top_k_categorical_accuracy: 0.9992 – val_accuracy: 0.9854 – val_loss: 0.0647 – val_top_k_categorical_accuracy: 0.9999
 Epoch 60/80

235/235 ————— **31s** 130ms/step – accuracy: 0.9655 – loss: 0.1394 – top_k_categorical_accuracy: 0.9994 – val_accuracy: 0.9854 – val_loss: 0.0640 – val_top_k_categorical_accuracy: 0.9999
 Epoch 61/80

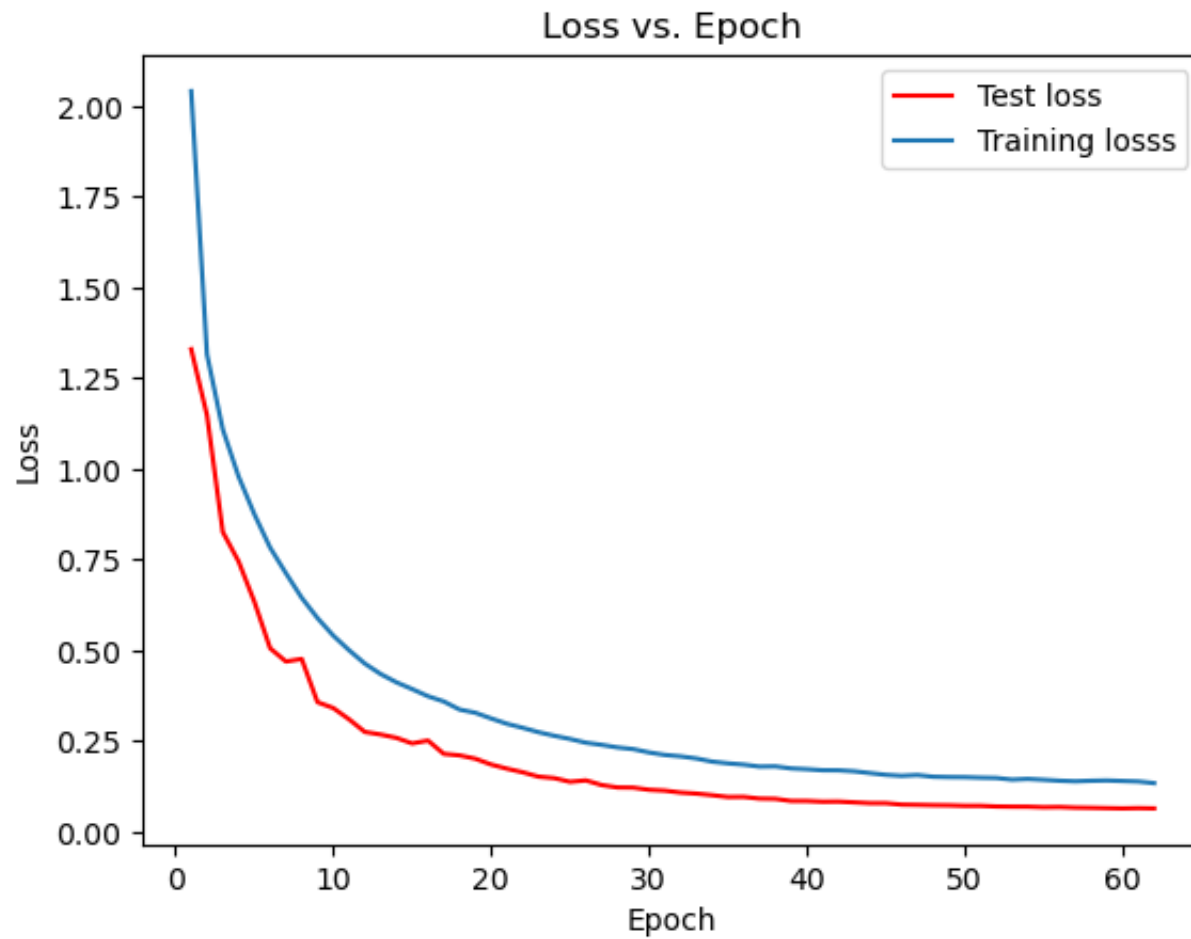
235/235 ————— **31s** 131ms/step – accuracy: 0.9648 – loss: 0.1379 – top_k_categorical_accuracy: 0.9990 – val_accuracy: 0.9851 – val_loss: 0.0647 – val_top_k_categorical_accuracy: 0.9999
 Epoch 62/80

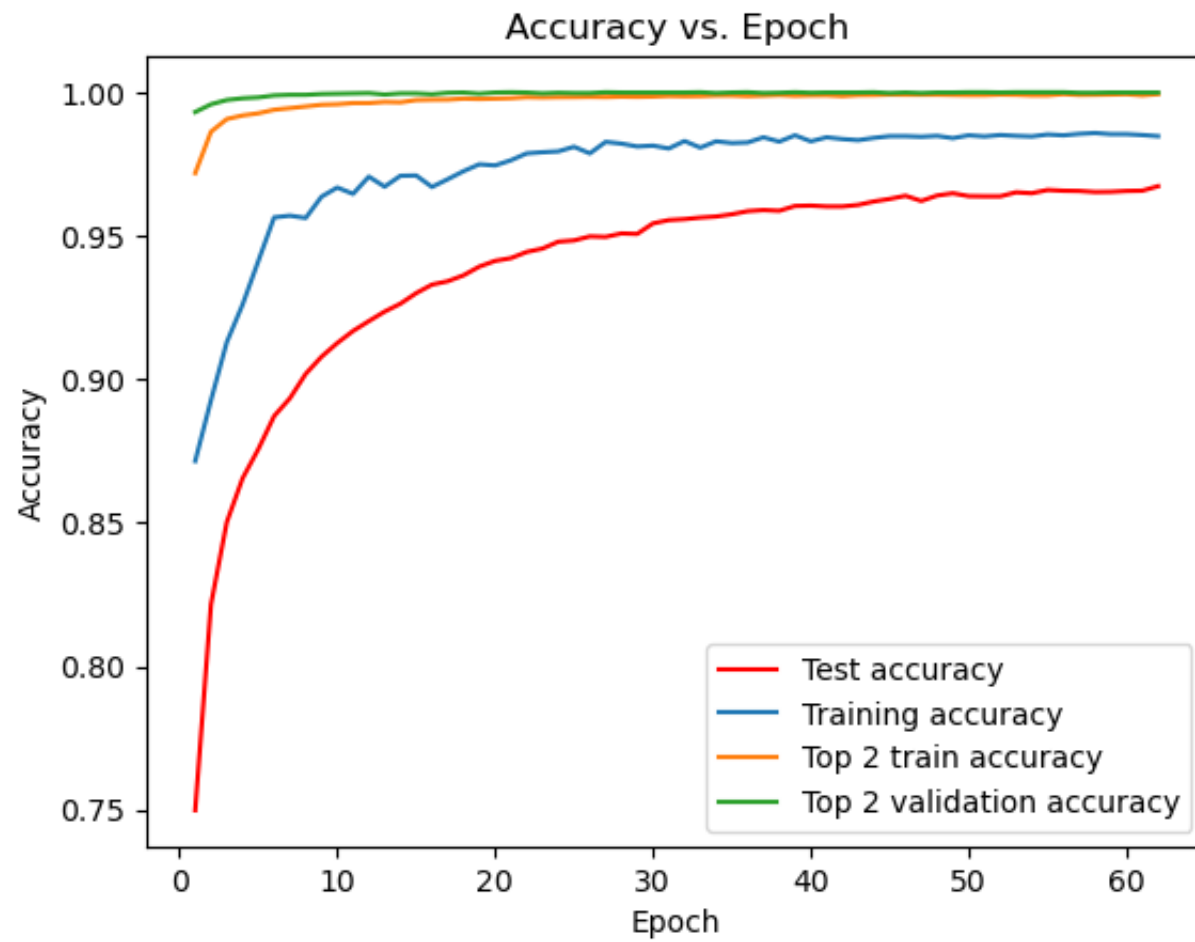
235/235 ————— **29s** 125ms/step – accuracy: 0.9668 – loss: 0.1366 – top_k_categorical_accuracy: 0.9994 – val_accuracy: 0.9847 – val_loss: 0.0640 – val_top_k_categorical_accuracy: 0.9999

Next execute the code in the cell below to evaluate the model just trained.

```
In [30]: plot_loss(history_vnnsep)
         plot_accuracy(history_vnnsep)

## Display accuracy netrics
         evaluate_model(nnsep, test_images, test_labels)
```





313/313  **3s** 9ms/step

Categorical accuracy = 0.9847

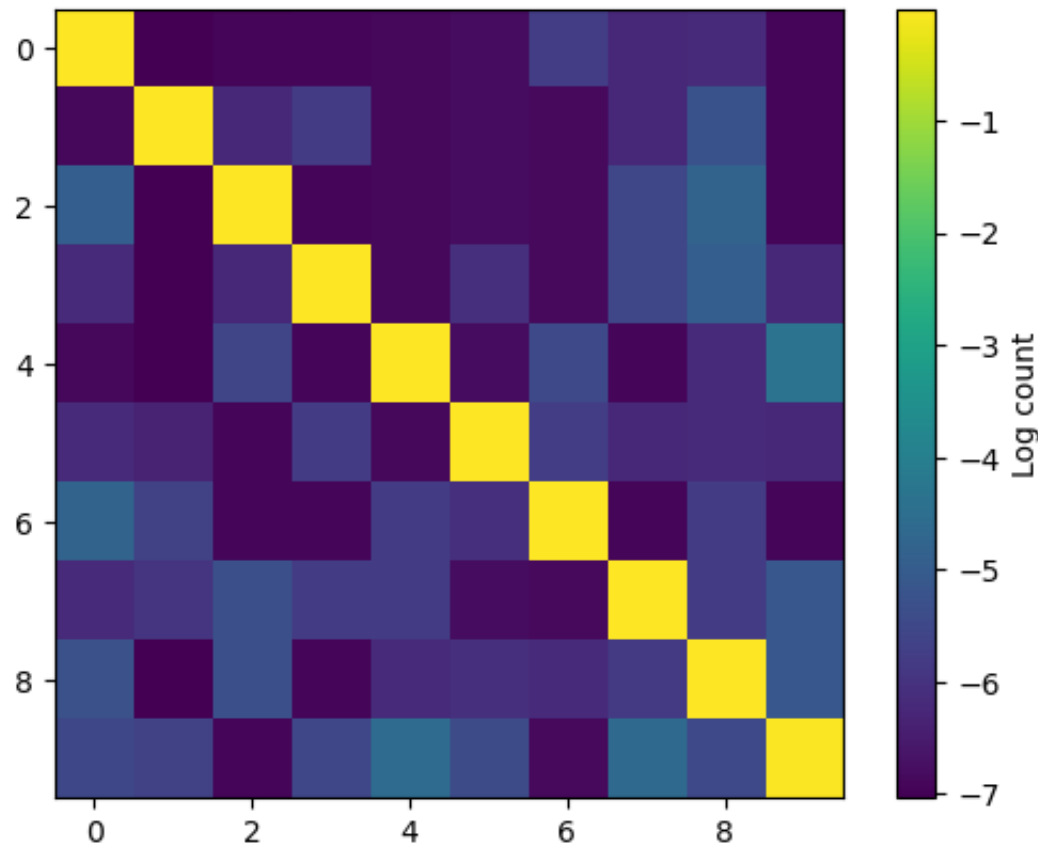
Top 2 accuracy = 0.9974

Average precision = 0.9847

Average recall = 0.9847

Confusion matrix

	0	1	2	3	4	5	6	7	8	9
0	976	0	0	0	0	0	2	1	1	0
1	0	1127	1	2	0	0	0	1	4	0
2	6	0	1016	0	0	0	0	3	7	0
3	1	0	1	997	0	1	0	3	6	1
4	0	0	3	0	963	0	3	0	1	12
5	1	1	0	2	0	883	2	1	1	1
6	7	3	0	0	2	1	943	0	2	0
7	1	2	4	2	2	0	0	1010	2	5
8	4	0	4	0	1	1	1	2	956	5
9	3	3	0	3	9	3	0	9	3	976



? Examine the results and compare them to those created for Exercise 6-3.

1. Compare the number of epochs that occur before reaching the early stopping criteria. What does this difference tell you about the effect of using a decaying learning rate?
2. How can you explain the improved model performance given the decaying learning rate?

✓ **Answers:**

1. The number of epochs that occur before reaching the early stopping criteria for the decaying rate model is ~10x times higher. In current case early stopping wasn't even triggered, but based on the

loss reduction rate it would be very close should the maximum epoch count be higher.

2. The improved model performance given the decaying learning rate makes total sense. Smaller learning rate helps gradient descending algorithm without overjumping smaller, but still improving accuracy local minimas.

Model with Residual Blocks

To train very deep models one needs to do more than just stack sequentially evaluated convolutional layers. Doing so, results in diminishing gradients during back propagation as the depth of the network is increases. In this case, we are working with relatively shallow networks, which should not encounter problems with vanishing gradients. None the less, we can demonstrate the principle.

The code below implements a model with two residual blocks. This code uses another method for specifying Keras models. The output of each layer is set to a named variable. The input of each layer is explicitly specified using these named variables. Using this approach allows us to create non-sequential models. In this case, we must use an addition operation to combine the residual from the convolution layers with the **identity shortcut** branch.

The residual blocks use separable convolutional layers. The first residual block has 32 channels and the second residual block has 64 channels. A 1×1 convolutional layer is used to transform the 32 channel identity shortcut from the first residual block to the required 64 channels for the second residual block. Each residual block is followed by a max pooling layer.

Execute the code in the cell below and examine the model summary.

```
In [31]: CNN_input = layers.Input(shape=(28, 28, 1, ))
u = layers.SeparableConv2D(32, (3, 3), padding = 'same', activation = 'relu')(CNN_input)

## A residual block
v = layers.SeparableConv2D(32, (3, 3), padding = 'same', activation = 'relu')(u)
v = layers.SeparableConv2D(32, (3, 3), padding = 'same', activation = 'relu')(v)
v = layers.Add()(v,u) # Combine the output of the pervious layer with the output of the convoluti
v = layers.MaxPooling2D((2, 2))(v)
```

```

## Another residual block
r = layers.Conv2D(filters=64, kernel_size=(1,1))(v) # 1x1 convolution layers to transform 32 to
w = layers.SeparableConv2D(64, (3, 3), padding = 'same', activation = 'relu')(v)
w = layers.SeparableConv2D(64, (3, 3), padding = 'same', activation = 'relu')(w)
w = layers.Add()(w,r) # Combine the output of the previous layer with the output of the convolution
w = layers.MaxPooling2D((2, 2))(w)

## Flatten and classify
x = layers.Flatten()(w)
x = layers.Dense(64, activation = 'relu',
                  kernel_regularizer=regularizers.l2(0.1))(x)
x = Dropout(0.5)(x)
CNN_output = layers.Dense(10, activation = 'softmax')(x)

CNN_model = models.Model(CNN_input, CNN_output)
CNN_model.summary()

```

Model: "functional_15"

Layer (type)	Output Shape	Param #	Connected to
input_layer_3 (InputLayer)	(None, 28, 28, 1)	0	–
separable_conv2d_3 (SeparableConv2D)	(None, 28, 28, 32)	73	input_layer_3[0]...
separable_conv2d_4 (SeparableConv2D)	(None, 28, 28, 32)	1,344	separable_conv2d...
separable_conv2d_5 (SeparableConv2D)	(None, 28, 28, 32)	1,344	separable_conv2d...
add (Add)	(None, 28, 28, 32)	0	separable_conv2d... separable_conv2d...
max_pooling2d_6 (MaxPooling2D)	(None, 14, 14, 32)	0	add[0][0]
separable_conv2d_6 (SeparableConv2D)	(None, 14, 14, 64)	2,400	max_pooling2d_6[...

separable_conv2d_7 (SeparableConv2D)	(None , 14, 14, 64)	4,736	separable_conv2d...
conv2d_6 (Conv2D)	(None , 14, 14, 64)	2,112	max_pooling2d_6[...
add_1 (Add)	(None , 14, 14, 64)	0	separable_conv2d... conv2d_6[0][0]
max_pooling2d_7 (MaxPooling2D)	(None , 7, 7, 64)	0	add_1[0][0]
flatten_3 (Flatten)	(None , 3136)	0	max_pooling2d_7[...
dense_6 (Dense)	(None , 64)	200,768	flatten_3[0][0]
dropout_12 (Dropout)	(None , 64)	0	dense_6[0][0]
dense_7 (Dense)	(None , 10)	650	dropout_12[0][0]

Total params: 213,427 (833.70 KB)

Trainable params: 213,427 (833.70 KB)

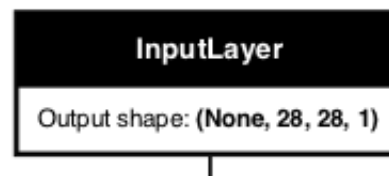
Non-trainable params: 0 (0.00 B)

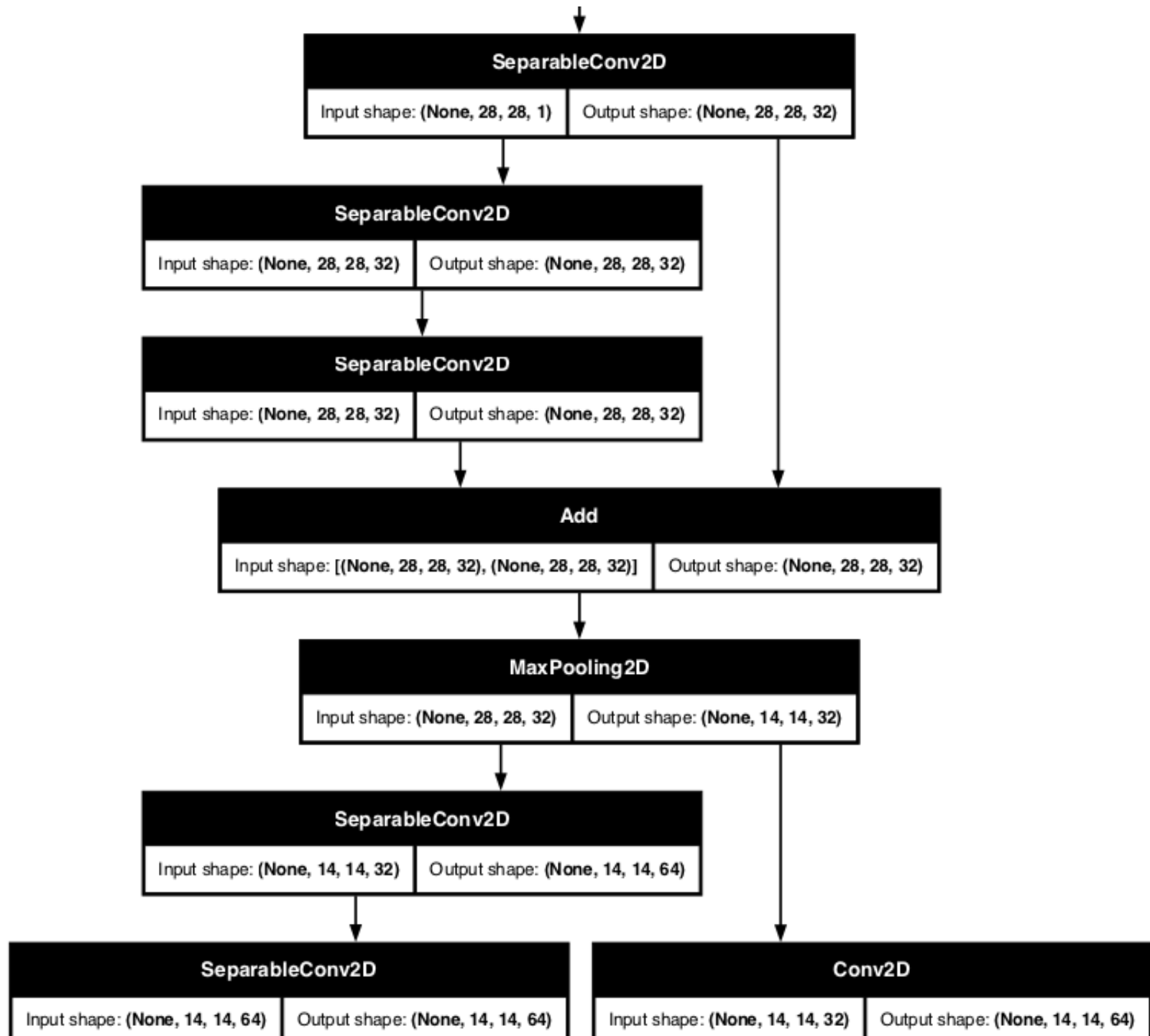
While this model has an additional separable convolution layer the total number of parameters that must be learned has increased only slightly compared to the sequential model we have been working with.

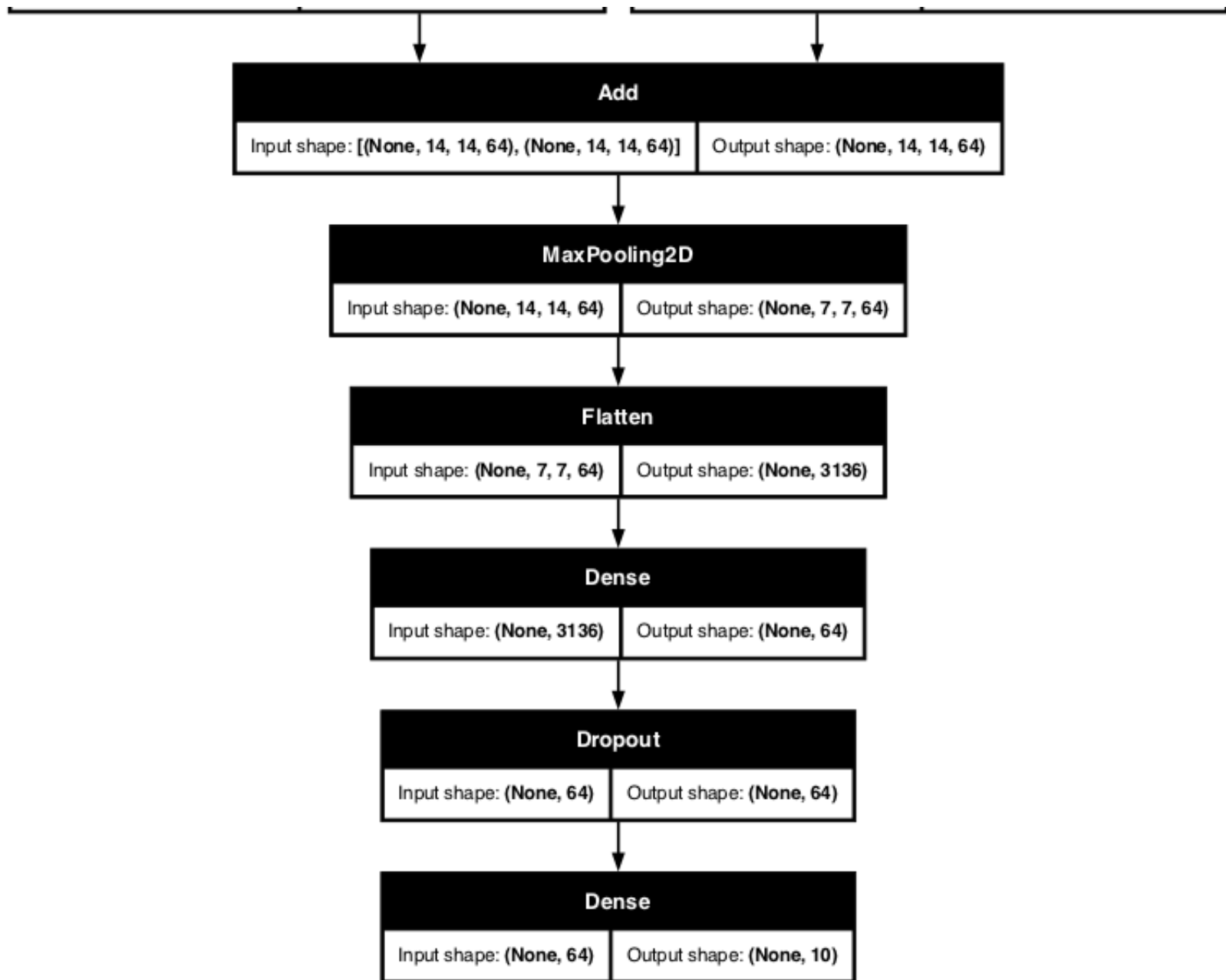
Since the model structure is not sequential, it will be useful to visualize how the layers are connected. Display a graph of the model by executing the code in the cell below which uses the [keras.utils.plot_model](#) function.

```
In [32]: plot_model(CNN_model, "mini_resnet.png", show_shapes=True, dpi=60)
```

Out[32]:







With the model defined it is time to train and evaluate it. The same process and hyperparameters are used here. Execute the code below and examine the results.

```

In [33]: ## Define the learning rate schedule
initial_learning_rate = 0.01
lr_schedule = ExponentialDecay(
    initial_learning_rate,
    decay_steps=100,
    decay_rate=0.96,
    staircase=True)

## compile the model
CNN_model.compile(optimizer = RMSprop(learning_rate=lr_schedule, weight_decay=0.01),
                  loss = 'categorical_crossentropy',
                  metrics = ['accuracy', 'top_k_categorical_accuracy'])

## Now fit the model
history_CNN_model = CNN_model.fit(train_images, train_labels,
                                   epochs = TEST_EPOCHS or 80, batch_size = 128,
                                   validation_data = (test_images, test_labels),
                                   callbacks = callbacks_list, # Call backs argument here
                                   verbose = 1)

plot_loss(history_CNN_model)
plot_accuracy(history_CNN_model)

## Display accuracy netrics
evaluate_model(CNN_model, test_images, test_labels)

```

Epoch 1/80

469/469 ————— **71s** 144ms/step - accuracy: 0.4780 - loss: 2.4057 - top_k_categorical_accuracy: 0.8464 - val_accuracy: 0.7450 - val_loss: 1.5178 - val_top_k_categorical_accuracy: 0.9898

Epoch 2/80

469/469 ————— **72s** 154ms/step - accuracy: 0.7695 - loss: 1.2719 - top_k_categorical_accuracy: 0.9870 - val_accuracy: 0.9008 - val_loss: 0.6124 - val_top_k_categorical_accuracy: 0.9975

Epoch 3/80

469/469 ————— **69s** 146ms/step - accuracy: 0.8249 - loss: 0.9329 - top_k_categorical_accuracy: 0.9916 - val_accuracy: 0.8938 - val_loss: 0.6274 - val_top_k_categorical_accuracy: 0.9978

Epoch 4/80

469/469 ————— **72s** 153ms/step - accuracy: 0.8449 - loss: 0.8080 - top_k_categorical_accuracy: 0.9923 - val_accuracy: 0.9129 - val_loss: 0.4895 - val_top_k_categorical_accuracy: 0.9978

Epoch 5/80

469/469 ————— **66s** 141ms/step – accuracy: 0.8572 – loss: 0.6959 – top_k_categorical_accuracy: 0.9945 – val_accuracy: 0.9196 – val_loss: 0.4462 – val_top_k_categorical_accuracy: 0.9975
Epoch 6/80

469/469 ————— **68s** 145ms/step – accuracy: 0.8752 – loss: 0.5912 – top_k_categorical_accuracy: 0.9952 – val_accuracy: 0.9253 – val_loss: 0.4272 – val_top_k_categorical_accuracy: 0.9983
Epoch 7/80

469/469 ————— **72s** 154ms/step – accuracy: 0.8875 – loss: 0.5265 – top_k_categorical_accuracy: 0.9953 – val_accuracy: 0.9373 – val_loss: 0.3287 – val_top_k_categorical_accuracy: 0.9983
Epoch 8/80

469/469 ————— **69s** 147ms/step – accuracy: 0.8995 – loss: 0.4774 – top_k_categorical_accuracy: 0.9959 – val_accuracy: 0.9415 – val_loss: 0.3122 – val_top_k_categorical_accuracy: 0.9983
Epoch 9/80

469/469 ————— **66s** 141ms/step – accuracy: 0.9116 – loss: 0.4404 – top_k_categorical_accuracy: 0.9968 – val_accuracy: 0.9487 – val_loss: 0.3214 – val_top_k_categorical_accuracy: 0.9984
Epoch 10/80

469/469 ————— **67s** 142ms/step – accuracy: 0.9249 – loss: 0.3832 – top_k_categorical_accuracy: 0.9975 – val_accuracy: 0.9634 – val_loss: 0.2439 – val_top_k_categorical_accuracy: 0.9988
Epoch 11/80

469/469 ————— **71s** 151ms/step – accuracy: 0.9341 – loss: 0.3458 – top_k_categorical_accuracy: 0.9977 – val_accuracy: 0.9669 – val_loss: 0.2140 – val_top_k_categorical_accuracy: 0.9993
Epoch 12/80

469/469 ————— **69s** 148ms/step – accuracy: 0.9398 – loss: 0.3125 – top_k_categorical_accuracy: 0.9982 – val_accuracy: 0.9691 – val_loss: 0.2034 – val_top_k_categorical_accuracy: 0.9986
Epoch 13/80

469/469 ————— **68s** 144ms/step – accuracy: 0.9473 – loss: 0.2803 – top_k_categorical_accuracy: 0.9981 – val_accuracy: 0.9757 – val_loss: 0.1734 – val_top_k_categorical_accuracy: 0.9994
Epoch 14/80

469/469 ————— **64s** 137ms/step – accuracy: 0.9514 – loss: 0.2565 – top_k_categorical_accuracy: 0.9988 – val_accuracy: 0.9750 – val_loss: 0.1530 – val_top_k_categorical_accuracy: 0.9995
Epoch 15/80

469/469 ————— **66s** 142ms/step – accuracy: 0.9568 – loss: 0.2280 – top_k_categorical_accuracy: 0.9985 – val_accuracy: 0.9790 – val_loss: 0.1390 – val_top_k_categorical_accuracy: 0.9996
Epoch 16/80

469/469 ————— **70s** 150ms/step – accuracy: 0.9602 – loss: 0.2066 – top_k_categorical_accuracy: 0.9991 – val_accuracy: 0.9810 – val_loss: 0.1289 – val_top_k_categorical_accuracy: 0.9999
Epoch 17/80

469/469 ————— **56s** 119ms/step – accuracy: 0.9619 – loss: 0.1955 – top_k_categorical_accuracy: 0.9990 – val_accuracy: 0.9801 – val_loss: 0.1217 – val_top_k_categorical_accuracy: 0.9999
Epoch 18/80

469/469 ————— **57s** 122ms/step – accuracy: 0.9632 – loss: 0.1813 – top_k_categorical_accuracy: 0.9990 – val_accuracy: 0.9809 – val_loss: 0.1144 – val_top_k_categorical_accuracy: 0.9996
Epoch 19/80

469/469 ————— **63s** 133ms/step – accuracy: 0.9676 – loss: 0.1640 – top_k_categorical_accuracy: 0.9991 – val_accuracy: 0.9832 – val_loss: 0.1042 – val_top_k_categorical_accuracy: 0.9998
Epoch 20/80

469/469 ————— **62s** 131ms/step – accuracy: 0.9676 – loss: 0.1580 – top_k_categorical_accuracy: 0.9994 – val_accuracy: 0.9818 – val_loss: 0.0996 – val_top_k_categorical_accuracy: 1.0000
Epoch 21/80

469/469 ————— **57s** 121ms/step – accuracy: 0.9697 – loss: 0.1443 – top_k_categorical_accuracy: 0.9995 – val_accuracy: 0.9827 – val_loss: 0.0924 – val_top_k_categorical_accuracy: 1.0000
Epoch 22/80

469/469 ————— **58s** 123ms/step – accuracy: 0.9708 – loss: 0.1381 – top_k_categorical_accuracy: 0.9993 – val_accuracy: 0.9828 – val_loss: 0.0912 – val_top_k_categorical_accuracy: 0.9998
Epoch 23/80

469/469 ————— **58s** 124ms/step – accuracy: 0.9718 – loss: 0.1345 – top_k_categorical_accuracy: 0.9995 – val_accuracy: 0.9844 – val_loss: 0.0848 – val_top_k_categorical_accuracy: 1.0000
Epoch 24/80

469/469 ————— **58s** 124ms/step – accuracy: 0.9714 – loss: 0.1315 – top_k_categorical_accuracy: 0.9994 – val_accuracy: 0.9844 – val_loss: 0.0810 – val_top_k_categorical_accuracy: 1.0000
Epoch 25/80

469/469 ————— **57s** 122ms/step – accuracy: 0.9738 – loss: 0.1228 – top_k_categorical_accuracy: 0.9996 – val_accuracy: 0.9840 – val_loss: 0.0794 – val_top_k_categorical_accuracy: 0.9999
Epoch 26/80

469/469 ————— **59s** 127ms/step – accuracy: 0.9741 – loss: 0.1198 – top_k_categorical_accuracy: 0.9996 – val_accuracy: 0.9851 – val_loss: 0.0757 – val_top_k_categorical_accuracy: 1.0000
Epoch 27/80

469/469 ————— **58s** 123ms/step – accuracy: 0.9743 – loss: 0.1188 – top_k_categorical_accuracy: 0.9994 – val_accuracy: 0.9850 – val_loss: 0.0740 – val_top_k_categorical_accuracy: 1.0000
Epoch 28/80

469/469 ————— **59s** 125ms/step – accuracy: 0.9755 – loss: 0.1115 – top_k_categorical_accuracy: 0.9997 – val_accuracy: 0.9848 – val_loss: 0.0733 – val_top_k_categorical_accuracy: 1.0000
Epoch 29/80

469/469 ————— **61s** 131ms/step – accuracy: 0.9755 – loss: 0.1110 – top_k_categorical_accuracy: 0.9995 – val_accuracy: 0.9843 – val_loss: 0.0723 – val_top_k_categorical_accuracy: 1.0000
Epoch 30/80

469/469 ————— **59s** 126ms/step – accuracy: 0.9761 – loss: 0.1083 – top_k_categorical_accuracy: 0.9997 – val_accuracy: 0.9848 – val_loss: 0.0706 – val_top_k_categorical_accuracy: 0.9999
Epoch 31/80

469/469 ————— **61s** 130ms/step – accuracy: 0.9773 – loss: 0.1070 – top_k_categorical_accuracy: 0.9998 – val_accuracy: 0.9853 – val_loss: 0.0696 – val_top_k_categorical_accuracy: 1.0000
Epoch 32/80

469/469 ————— **60s** 128ms/step – accuracy: 0.9770 – loss: 0.1045 – top_k_categorical_accuracy: 0.9998 – val_accuracy: 0.9860 – val_loss: 0.0691 – val_top_k_categorical_accuracy: 1.0000
Epoch 33/80

469/469 ————— **60s** 128ms/step – accuracy: 0.9778 – loss: 0.1024 – top_k_categorical_accuracy: 0.9997 – val_accuracy: 0.9852 – val_loss: 0.0686 – val_top_k_categorical_accuracy: 1.0000
Epoch 34/80

469/469 ————— **59s** 126ms/step – accuracy: 0.9779 – loss: 0.1021 – top_k_categorical_accuracy: 0.9996 – val_accuracy: 0.9856 – val_loss: 0.0679 – val_top_k_categorical_accuracy: 1.0000
Epoch 35/80

469/469 ————— **59s** 127ms/step – accuracy: 0.9791 – loss: 0.0992 – top_k_categorical_accuracy: 0.9998 – val_accuracy: 0.9860 – val_loss: 0.0677 – val_top_k_categorical_accuracy: 1.0000
Epoch 36/80

469/469 ————— **60s** 128ms/step – accuracy: 0.9780 – loss: 0.1020 – top_k_categorical_accuracy: 0.9994 – val_accuracy: 0.9859 – val_loss: 0.0675 – val_top_k_categorical_accuracy: 1.0000
Epoch 37/80

469/469 ————— **60s** 128ms/step – accuracy: 0.9785 – loss: 0.0984 – top_k_categorical_accuracy: 0.9997 – val_accuracy: 0.9856 – val_loss: 0.0672 – val_top_k_categorical_accuracy: 1.0000
Epoch 38/80

469/469 ————— **60s** 127ms/step – accuracy: 0.9781 – loss: 0.1018 – top_k_categorical_accuracy: 0.9994 – val_accuracy: 0.9864 – val_loss: 0.0666 – val_top_k_categorical_accuracy: 1.0000
Epoch 39/80

469/469 ————— **60s** 128ms/step – accuracy: 0.9778 – loss: 0.1005 – top_k_categorical_accuracy: 0.9997 – val_accuracy: 0.9858 – val_loss: 0.0666 – val_top_k_categorical_accuracy: 1.0000
Epoch 40/80

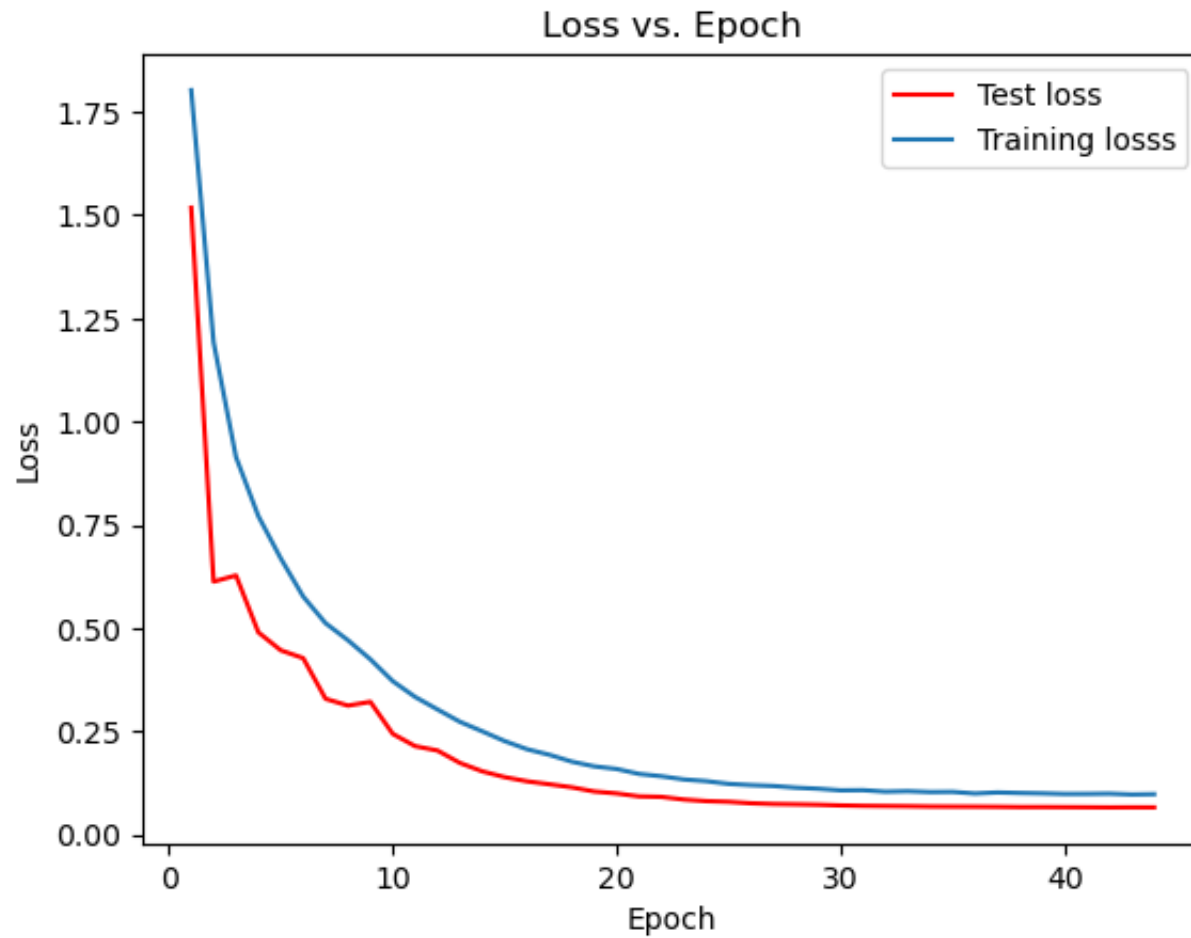
469/469 ————— **59s** 126ms/step – accuracy: 0.9782 – loss: 0.0991 – top_k_categorical_accuracy: 0.9997 – val_accuracy: 0.9861 – val_loss: 0.0664 – val_top_k_categorical_accuracy: 1.0000
Epoch 41/80

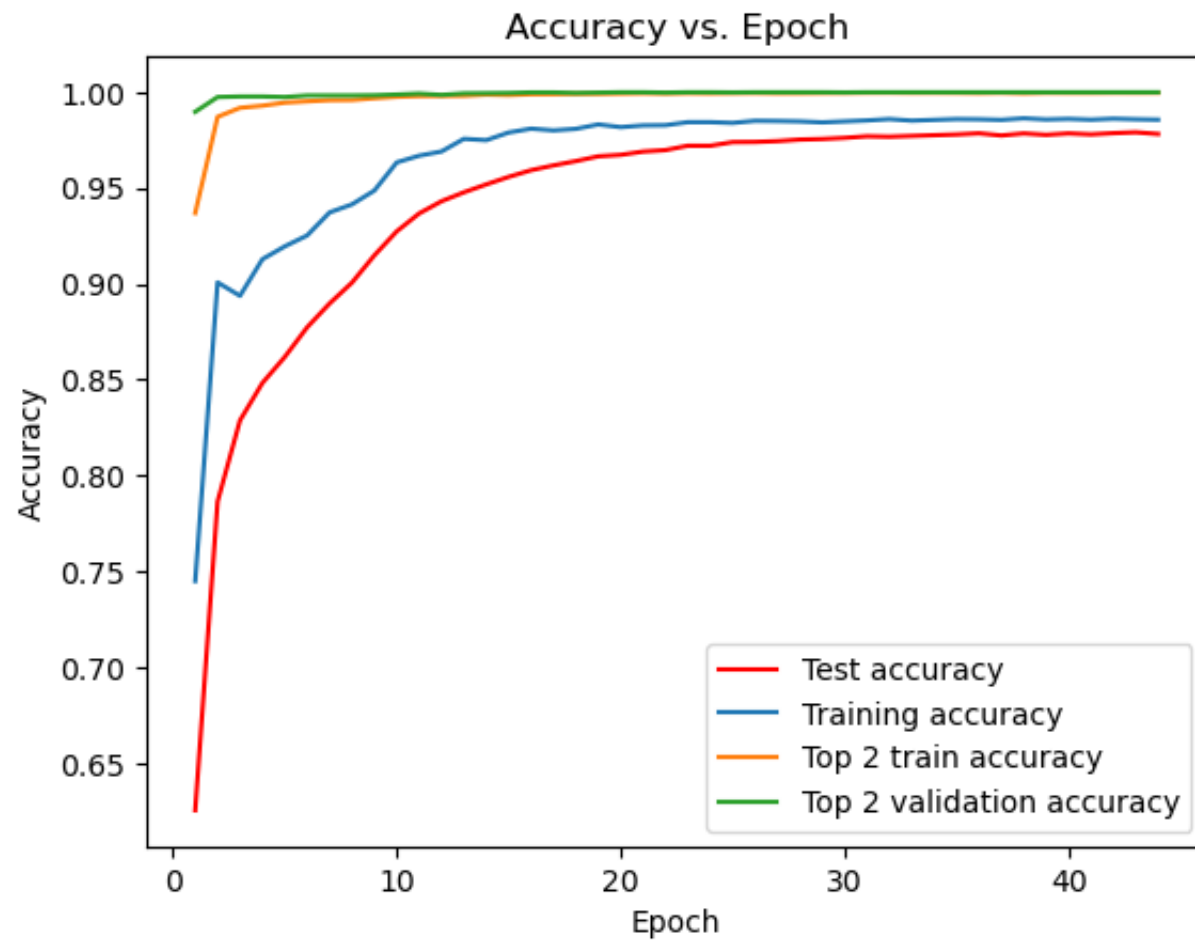
469/469 ————— **59s** 127ms/step – accuracy: 0.9773 – loss: 0.0997 – top_k_categorical_accuracy: 0.9997 – val_accuracy: 0.9857 – val_loss: 0.0661 – val_top_k_categorical_accuracy: 1.0000
Epoch 42/80

469/469 ————— **60s** 127ms/step – accuracy: 0.9784 – loss: 0.0982 – top_k_categorical_accuracy: 0.9997 – val_accuracy: 0.9862 – val_loss: 0.0658 – val_top_k_categorical_accuracy: 1.0000
Epoch 43/80

469/469 ————— **59s** 126ms/step – accuracy: 0.9786 – loss: 0.0982 – top_k_categorical_accuracy: 0.9997 – val_accuracy: 0.9859 – val_loss: 0.0659 – val_top_k_categorical_accuracy: 1.0000
Epoch 44/80

469/469 ————— **59s** 125ms/step – accuracy: 0.9782 – loss: 0.0967 – top_k_categorical_accuracy: 0.9997 – val_accuracy: 0.9857 – val_loss: 0.0658 – val_top_k_categorical_accuracy: 1.0000





313/313  **4s** 14ms/step

Categorical accuracy = 0.9857

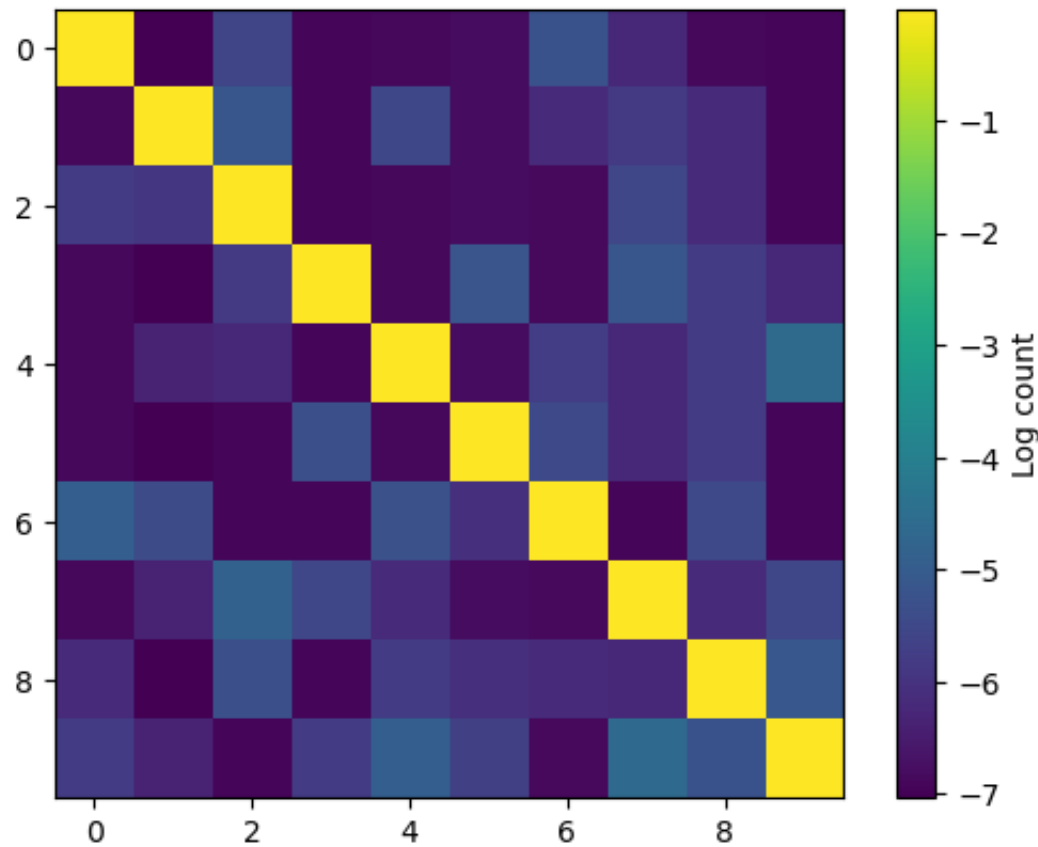
Top 2 accuracy = 0.9973

Average precision = 0.9857

Average recall = 0.9857

Confusion matrix

	0	1	2	3	4	5	6	7	8	9
0	972	0	3	0	0	0	4	1	0	0
1	0	1123	5	0	3	0	1	2	1	0
2	2	2	1024	0	0	0	0	3	1	0
3	0	0	2	996	0	4	0	5	2	1
4	0	1	1	0	966	0	2	1	2	9
5	0	0	0	4	0	882	3	1	2	0
6	6	4	0	0	4	1	940	0	3	0
7	0	1	7	3	1	0	0	1012	1	3
8	1	0	4	0	2	1	1	1	959	5
9	2	1	0	2	6	2	0	9	4	983



► ? **Exercise 6-10:** Answer the following questions based on the results shown above.

1. Are the accuracy results of the model with residual blocks comparable to the sequential model evaluated in Exercise 6-9 and why should this result be expected.
2. Is there a significant difference in the number of training epochs required between the sequential model used in Exercise 3-9 and the model with residual blocks and why would this behavior be expected?
3. Why is a 1×1 convolution layer required in the identity loop of the second residual block.

✓ **Answers:**

1. The residual model's accuracy is better, which isn't surprising. One of its paths has 5 consecutive convolutional layers, making this network deeper, with residual connections compensating for the vanishing gradient.
2. The residual model requires twice fewer epochs to train due to improved gradient propagation across layers provided by the residual connections. To be fair, I must notice, that an average per epoch train time grew up twice as well. So the overall model training didn't change drastically.
3. The 1x1 convolution layer was required in the identity loop of the second residual block to adjust the feature map count to the following Add layer input, which has 64 feature maps.

Summary

In these exercises you have applied convolutional neural networks (CNNs) to an image classification problem. Some key points of this lesson are:

1. CNNs learn powerful feature maps from images.
2. Convolutional feature maps have multiple channels with a spatial dimensions.
3. Multi-layer CNNs create highly abstract features at deeper layers. Deeper architectures can learn more powerful features.
4. The convolutional feature map is used as an input to a classifier.
5. Evaluation of multi-class classifiers can be evaluated by both top-1 and top-k accuracy metrics.

Copyright 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, Stephen F Elston. All rights reserved.

In []: